ODT COMPLETE REFERENCE

On-Device Training Library

FQT + RigL Sparse Training on STM32 Nucleo-F746ZG

39 Chapters | Source-Level Documentation | Complete RigL Implementation

Paper 1: Deutel et al. "On-Device Training of Fully Quantized DNNs on Cortex-M MCUs" (arXiv 2407.10734v2)

Paper 2: Evci et al. "Rigging the Lottery: Making All Tickets Winners" (ICML 2020)

Dataset: UCI HAR (Human Activity Recognition) - 30 subjects, 6 activities, 50 Hz IMU

Target: STM32 Nucleo-F746ZG (Cortex-M7, 216 MHz, 320 KB SRAM)

INTRODUCTION

This introduction covers three foundational pillars: (1) FQT from Deutel et al. (arXiv 2407.10734v2), (2) RigL from Evci et al. (ICML 2020), and (3) the UCI HAR dataset. Each topic is covered in depth to prepare for the per-file chapters.

Part 1: Fully Quantized Training (FQT)

1.1 Why On-Device Training Matters

Modern deep learning sends personal sensor data to cloud servers for training, creating three problems: privacy violations, latency, and infrastructure dependence. On-device training resolves all three by keeping data on the device.

The FQT paper (Deutel, Knoll, Schreyer, Mutschler; FAU Erlangen-Nuremberg; arXiv 2407.10734v2) demonstrates full integer training on a Cortex-M7 MCU with only 320 KB SRAM -- including the backward pass.

CONCEPT	On-device training enables privacy-preserving, low-latency adaptation. A factory machine learns its own vibration signature without transmitting proprietary data to any external server.

1.2 The Four Memory Challenges

Training requires four data categories simultaneously in SRAM. On 320 KB MCU this is extreme:

Category	Description	128x128 float32	Solution
Weights	Parameters W (learned knowledge)	64 KB	SYM_INT32: 4x less
Activations	Intermediate outputs for backward pass	Batch x 512 B/layer	Micro-batch=1
Gradients	dL/dW update signal per weight	64 KB	INT32 accumulator
Optimizer state	Adam: momentum+variance = 2x weights	128 KB	SGD: zero extra

WARNING	A single 128x128 float32 linear layer: 64 KB weights + 64 KB gradients = 128 KB, nearly 40% of total SRAM. FQT INT8 cuts this to 16 KB per layer.

1.3 Symmetric Integer Quantization

ODT uses symmetric quantization: real_value = mantissa * scale. The scale is one float32 per tensor. SYM_INT32 stores 32-bit mantissas with qBits=12 (range [-2047, 2047]).

```
real_value = mantissa * scale // Decode integer to float
mantissa = round(real_value/scale) // Encode float to integer
```

```
// Example: weight=0.05, scale=0.004
mantissa = round(0.05/0.004) = round(12.5) = 13   // half-away rounding
decoded = 13*0.004 = 0.052 (error=0.002, 4%)
// Matmul: Y_mantissa = W_mantissa * X_mantissa
// Y_scale = W_scale * X_scale (automatic, tracked by SYM_INT32)
```

CONCEPT ODT dtypes: FLOAT32, INT32, BOOL (1-bit mask), SYM_INT32 (32-bit mantissa + scale, FQT workhorse), SYM (packed N-bit), ASYM (with zero-point). SYM_INT32 handles all FQT forward and backward computations.

1.4 The Integer Backward Pass

QAT simulates quantization only in the forward pass -- backward runs in float32. FQT runs ALL three matrix multiplications as integer operations: forward Y=W*X, backward E_prev=W^T*E_next, weight gradient dW=E_next*X^T.

```
// 1. Forward: Y_m = W_m x X_m; Y_scale = W_scale*X_scale
// 2. Error propagation (dL/dX):
// E_prev_m = W_m^T x E_next_m; scale = W_scale*E_next_scale
// 3. Weight gradient (dL/dW = outer product):
// dW_m = E_next_m x X_m^T; scale = E_next_scale*X_scale
// All via matmulSymInt32Tensors():
// outputQC->scale = aQC->scale * bQC->scale (automatic)
// int32 accumulator for inner products (exact, no overflow)
```

1.5 Stochastic Rounding

Dead zone: gradient magnitude smaller than scale/2 rounds to zero permanently. For scale=0.004, gradients smaller than 0.002 are lost forever. SR escapes this by adding uniform noise before rounding.

```
// Dead zone example: gradient=0.0003, scale=0.004
mantissa = round(0.075) = 0 -> GRADIENT PERMANENTLY LOST

// SR: noise = rngNextFloat()-0.5 in [-0.5, 0.5)
P(round(0.075+noise)=1) = 7.5%   // low gradient = low probability
P(round(0.075+noise)=0) = 92.5%
E[SR(0.075)] = 0.075 = exact gradient   // UNBIASED - E[SR(x)]=x for all x
```

TIP SR: roundSRHalfAway() in Rounding.c. noise=rngNextFloat()-0.5 (XorShift32, period 2^32-1). Subtract 0.5 to center noise: without subtraction, noise in [0,1) would bias every rounding upward.

1.6 FQT Results

```
FQT results on Cortex-M7 @ 216 MHz (UCI HAR):
Memory: 34.8% less SRAM vs float32 training
Latency: 49% lower per training step
```

Speedup: 8.7x wall-clock for same accuracy	
Accuracy gap: less than 1% vs float32 baseline	
INT8 weights: 4x size reduction vs float32	
Cortex-M7 SMLAL 64-bit DSP used for int32 matmul inner loop	

Part 2: RigL -- Sparse Training That Matches Dense

2.1 Why Sparse Training and What RigL Solves

Standard pruning requires 2-3x compute. Lottery Ticket needs full dense training. RigL trains sparse from scratch in one pass by using gradient information to grow the right connections.

// Gradient magnitude for GROW decisions:	
// |grad[i]|: loss reduction if inactive weight i were updated	
// High |grad[i]| for inactive weight => good candidate to activate	
// Weight magnitude for DROP decisions:	
// Low |w[i]| for active weight => small contribution => safe to prune	
// Conservation: DROP K = GROW K => sparsity constant throughout	

2.2 The RigL Algorithm

RIGL (per sparse layer, every T training steps):	
alpha(t) = (0.3/2)*(1+cos(pi*t/T_end)) // cosine decay	
K = floor(alpha(t)*(1-s)*N) // weights to swap	
DROP K active weights with smallest |W|:	
thresh = K-th smallest |W[i]| in active set	
For M[i]=1 and |W[i]|<=thresh: M[i]=0; W[i]=0; G[i]=0	
GROW K inactive weights with largest |G|:	
thresh = K-th largest |G[i]| in inactive set	
For M[i]=0 and |G[i]|>=thresh: M[i]=1; W[i]=0	
Zero gradients of remaining inactive weights: G[i]=0	

CONCEPT Conservation: exactly K dropped = K grown per update. Total active weights stays at (1-s)*N. The mask evolves from low-magnitude to high-gradient positions -- learning optimal sparse connectivity.

2.3 Cosine Schedule and LR Synergy

alpha(t) = (alpha_init/2)*(1+cos(pi*t/T_end))	
// alpha_init=0.3, T_end=0.8*total_steps	
// HAR: 50 epochs, 10350 steps, RigL every T=100 steps	
// step 0: alpha=0.30, K=30% of active	// explore

```
// step 4140: alpha=0.15, K=15%                                  // moderate
// step 8280: alpha=0.00, K=0 (mask frozen)                      // refine
// Synergy with CosineAnnealingLR:
// High LR + high alpha: explore weight space and connectivity
// Low LR + zero alpha: refine stable sparse structure
```

2.4 RigL Results and ODT Status

ResNet-50 ImageNet: Dense=76.8%, RigL 80% sparse 1x=74.6%, RigL 80% sparse 5x=76.4% (matches dense). HAR estimate: RigL 90% sparse + FQT should achieve ~90-93% test accuracy.

RigL: ODT Gap BOOL tensor, tensorBoolGet/Set, bernoulliFillMask: EXIST. Missing: weightMask in linearConfig_t, mask-aware matmul, mask-aware SGD, findAbsKthSmallestActive, findAbsKthLargestInactive, rigLStep, serializeSparsity. Chapter 39 provides all seven missing implementations.

Part 3: HAR Dataset -- Human Activity Recognition

3.1 Dataset Overview

UCI HAR (Anguita et al. 2013): 30 volunteers aged 19-48, Samsung Galaxy S II on waist, 50 Hz IMU, 6 activity classes. Butterworth low-pass filter at 0.3 Hz separates gravity from body acceleration. Windows: 128 samples (2.56s), 50% overlap (stride=64).

```
// 9 channels: body_acc_xyz, body_gyro_xyz, total_acc_xyz
// Train: 6617 samples, Val: 735, Test: 2947 (total: 10299)
// Per sample: 9*128*4B = 4608 bytes = 4.5 KB (fits on MCU!)
// Full train set: 30.5 MB (does NOT fit: stream from SD card)
```

3.2 Activity Classes and IMU Signatures

Class	Activity	IMU Signature
0	WALKING	Regular 1-2 Hz oscillations in body_acc; periodic leg swing in gyro; ~0.5g amplitude
1	WALKING_UPSTAIRS	Higher vertical acc; irregular step timing; asymmetric push-off visible
2	WALKING_DOWNSTAIRS	Heel-strike impact spikes in total_acc; braking deceleration on body_acc_z
3	SITTING	Near-zero body_acc; gravity on z-axis; gyro near zero throughout window
4	STANDING	Similar to sitting with subtle postural sway (0.1-0.5 Hz); hardest pair
5	LAYING	Gravity shifted to one axis; body_acc near zero throughout 2.56-second window

3.3 NPY File Format and SD Card Streaming

On STM32, HAR data is stored as NumPy .npy files on an SD card. NPYLoader reads the binary header (magic, version, shape, dtype) and provides random access to samples by computing byte offsets in the file.

```
// NumPy .npy v1.0 file format:
// Bytes 0-5: magic 0x93+NUMPY
// Bytes 6-7: version 1, 0
// Bytes 8-9: header_len u16 LE
// Bytes 10+: Python dict
{'descr':'<f4','shape':(6617,9,128),...}
// Bytes 10+header_len: raw float32 data (row-major)
// Sample i offset: 10 + header_len + i * 4608
// fseek(fp, offset, SEEK_SET); fread(buf, 1, 4608, fp);
```

WARNING Never load the full 30.5 MB training set into MCU SRAM. DataLoader shuffles index array [0..6616] at epoch start (Fisher-Yates via rngShuffleIndices), reads samples in random order from SD card. SRAM overhead: 1 sample (4.5 KB) + indices (26 KB) = 31 KB total.

3.4 Model Architecture and Expected Performance

```
// HAR class balance (train set, 6617 total):
// WALKING: 1226 SITTING: 1286 LAYING: 672
// UPSTAIRS: 1073 STANDING: 1374 DOWNSTAIRS: 986
// Near-balanced -- no class weighting needed

// MCU-feasible model:
// Input [1,9,128] -> Conv1d(9->16,k=5,s=2) -> [1,16,64]
// -> Conv1d(16->16,k=5,s=2) -> [1,16,32] -> Flatten ->
[1,512]
// -> Linear(512->64, RigL 90%) -> ReLU -> Linear(64->6) ->
Softmax
// Weights: Conv ~8KB + Linear1 sparse ~5KB + mask 4KB = 17KB
total
// vs dense float32 Linear1 alone: 512*64*4 = 131 KB
```

CONCEPT RigL 90% + FQT INT8 achieves ~90x memory reduction for linear layers vs dense float32. A 512->64 weight matrix shrinks from 131 KB to ~1.5 KB active weights + 4 KB mask = 5.5 KB. This is the key enabler for on-device training within 320 KB SRAM.

End of Introduction. The 39 following chapters provide source-level documentation for every ODT library file: usage examples, dependency analysis, line-by-line explanation, and RigL gap analysis. Chapter 39 provides the complete RigL implementation covering all seven missing components.

Chapter 1

Common.h

Logging, Debug Levels, and Compiler Infrastructure

1.1 Usage of This File

Common.h is the foundational header included by virtually every C file in the ODT library. It provides three families of functionality: (1) conditional debug logging macros that compile to zero overhead in release builds; (2) byte-array printing for low-level diagnostics; (3) the include of stdbool.h, stdio.h, and string.h, making these available to all consumers without redundant includes.

Functions and Macros Exposed

`PRINT_DEBUG(str, ...)`	`// Logs [FILE: FUNC] + message if DLEVEL >= 3`
`PRINT_INFO(str, ...)`	`// Logs [FILE: FUNC] + message if DLEVEL >= 2`
`PRINT_ERROR(str, ...)`	`// Logs [FILE: FUNC] + message (red) if DLEVEL >= 1`
`PRINT_BYTE_ARRAY(prefix, byteArray, n)`	`// Hex-dumps n bytes for debugging`
`Compile-time knobs:`	
`-DDEBUG_MODE_DEBUG -> DLEVEL=3 (all logging)`	
`-DDEBUG_MODE_INFO -> DLEVEL=2 (info + error)`	
`-DDEBUG_MODE_ERROR -> DLEVEL=1 (error only)`	
`(none) -> DLEVEL=0 (silent)`	`// Release / MCU default`
`SOURCE_FILE macro: define before #include "Common.h"`	
`#define SOURCE_FILE "MYTENSOR"`	`// Names this file in log output`

Who Calls Common.h?

Every single source file in the ODT library includes Common.h. It is the first header in almost every .c file after the local #define SOURCE_FILE. Files that use PRINT_ERROR (which is the majority) require Common.h to even compile.

C Usage Example

`#define SOURCE_FILE "MY_LAYER"`	`// Must come BEFORE including Common.h`
`#include "Common.h"`	`// Gets logging macros + stdbool/stdio/string`
`void myFunction(int x) {`	
`PRINT_DEBUG("entering myFunction x=%d", x);`	`// Only prints if DLEVEL>=3`
`if (x < 0) {`	
`PRINT_ERROR("negative x: %d", x);`	`// Always prints if DLEVEL>=1`

```
exit(1);                                    // Fail-fast on MCU (no exception handling)
}
PRINT_INFO("x is valid: %d", x);            // Prints if DLEVEL>=2
}
```

1.2 Interactions with Other Files and STM32

Common.h has no dependencies of its own (it only includes standard library headers). Every other ODT file depends on it.

Hardware Context: STM32 Nucleo-F746ZG

On the STM32 Nucleo-F746ZG, printf() routes through a UART or SWO (Serial Wire Output) debug interface. PRINT_ERROR calls printf() unconditionally in DLEVEL>=1 builds. In a production MCU build with DLEVEL=0, ALL macro calls compile to empty do-while(false) blocks — zero code size, zero runtime cost.

```
// In release MCU build (DLEVEL=0):
PRINT_DEBUG("x=%d", x);                     // Compiles to: do {} while(false) -- 0 bytes
PRINT_ERROR("fail!", x);                    // Compiles to: do {} while(false) -- 0 bytes

// In debug build (-DDEBUG_MODE_DEBUG):
PRINT_DEBUG("x=%d", x);                     // Calls printf -- uses UART/SWO
```

WARNING If SOURCE_FILE is not defined before including Common.h, the fallback string "no Source file defined!" is used. Always define SOURCE_FILE at the top of each .c file to get meaningful log output.

CONCEPT The do { ... } while(false) pattern for macros is a classic C idiom. It ensures the macro behaves as a single statement in all contexts (including if-else chains without braces) and enables the compiler to optimize away the entire block when the condition (DLEVEL >= N) is a compile-time constant false.

1.3 Line-by-Line Explanation

Header Guard

```
#ifndef ODT_COMMON_H                        // Include guard: only process file once
#define ODT_COMMON_H                        // Define the guard sentinel
...
#endif                                      // End of guard
```

The include guard prevents multiple inclusion. If Common.h is included twice (e.g., A.h includes it, B.h includes it, and C.c includes both), only the first inclusion is processed. Without this, the macro redefinitions would cause compiler warnings or errors.

Standard Library Includes

```
#include <stdbool.h>          // Provides bool, true, false (C99)
#include <stdio.h>            // Provides printf(), FILE*, fprintf()
#include <string.h>           // Provides memcpy(), strlen(), memset()
```

By including these in Common.h, every ODT file that includes Common.h gets them for free. This reduces boilerplate but creates an implicit dependency: any file that uses memcpy() may rely on string.h being pulled in by Common.h rather than including it explicitly. This is fine for a tightly coupled embedded library.

SOURCE_FILE Macro

```
#ifndef SOURCE_FILE
#define SOURCE_FILE "no Source file defined!"
#endif

Usage in DTypes.c:
#define SOURCE_FILE "DTYPES"
#include "Common.h"
```

DLEVEL Computation

```
#ifdef DEBUG_MODE_DEBUG
#define DLEVEL 3
#elif defined(DEBUG_MODE_INFO)
#define DLEVEL 2
#elif defined(DEBUG_MODE_ERROR)
#define DLEVEL 1
#else
#define DLEVEL 0
#endif
```

DLEVEL is an integer compile-time constant. The macros compare against it with a compile-time if. Modern compilers (GCC, Clang, ARMCC) completely eliminate dead branches when DLEVEL is 0, resulting in zero code size overhead for logging in production builds.

PRINT_DEBUG Macro

```
#define PRINT_DEBUG(str, ...) do {
if (DLEVEL >= 3) {
printf("[%s: %s] ", SOURCE_FILE, __FUNCTION__);
printf(str, ##__VA_ARGS__);
printf(ANSI_RESET_NEWLINE);
}
} while (false)
```

##__VA_ARGS__ is the GCC/Clang extension that handles zero variadic arguments. If str has no format arguments, ##__VA_ARGS__ expands to nothing (the ## eats the trailing comma). Without ##, PRINT_DEBUG("msg") would expand to printf("msg",) which is a syntax error.

The ANSI escape code \033[0;33m colors terminal output yellow for DEBUG messages. \033[0;31m is red for ERROR. \033[0m resets to default. These codes work on Linux terminals and the STM32 SWO viewer if configured. In environments without ANSI support (raw UART), they appear as literal characters.

PRINT_BYTE_ARRAY Macro

```
#define PRINT_BYTE_ARRAY(prefix, byteArray, n) do {
printf("[%s: %s] ", SOURCE_FILE, __FUNCTION__);
printf(prefix);                          // User-provided label
for (size_t i = 0; i < n; i++)
printf("0x%02X ", byteArray[i]);         // Each byte as hex, e.g. 0x3F
printf("newline");
} while (false)
```

Example output: [TENSOR: loadWeights] raw bytes: 0x3F 0x80 0x00 0x00. This is how you would inspect the raw bytes of a float32 tensor on the MCU. The value 0x3F800000 is IEEE 754 for 1.0f.

> **TIP** PRINT_BYTE_ARRAY is not guarded by DLEVEL. It always prints when called. This is intentional: it is a diagnostic tool that the developer calls explicitly, not an ambient logging macro.

1.4 Missing RigL Code

Common.h itself does not need RigL-specific changes. It is infrastructure. However, any new RigL files (RigL.h, RigL.c) must follow the same SOURCE_FILE pattern:

```
// In RigL.c:
#define SOURCE_FILE "RIGL"               // Name for log messages
#include "Common.h"
#include "RigL.h"
#include "Tensor.h"
#include "MinMax.h"
#include "Layer.h"
#include <math.h>                        // For cosf(), fabsf()
#include <stdlib.h>                      // For malloc(), free()

// Key RigL logging examples:
PRINT_INFO("rigLStep: step=%zu K=%zu", step, K);
PRINT_DEBUG("drop thresh=%.6f", dropThresh);
PRINT_ERROR("rigLStep: NULL mask on LINEAR layer %zu", l);
```

Common.h provides the complete logging infrastructure that rigLStep() will need. The PRINT_INFO calls during mask updates are critical for debugging sparse training convergence — the developer needs to see K and alpha values to verify the cosine decay is working correctly.

Chapter 2

DTypes.h / DTypes.c

Type-Safe Byte-Level Serialization and Deserialization

2.1 Usage of This File

DTypes.h/c provides the low-level serialization layer: functions to convert between raw byte arrays (uint8_t*) and typed values (int32_t, float). On embedded systems, data is streamed as flat byte arrays over UART, SPI, I2C, or loaded from flash. DTypes bridges the gap between these raw bytes and the typed C variables used in computation.

Functions Exposed

`int32_t readBytesAsInt32(uint8_t *bytes)`	`// Read 4 bytes as int32_t`
`int32_t readNumberOfBytesAsInt32(uint8_t *data, size_t n)`	`// Read n bytes as int32_t (zero-padded)`
`void readBytesAsInt32Array(size_t n, uint8_t *bytes, int32_t *out)`	`// Deserialize array of int32`
`float readBytesAsFloat(uint8_t *bytes)`	`// Read 4 bytes as float`
`void readBytesAsFloatArray(size_t n, uint8_t *bytes, float *out)`	`// Deserialize array of floats`
`void writeInt32ToByteArray(int32_t value, uint8_t *bytes)`	`// Serialize int32_t to 4 bytes`
`void writeInt32ArrayToByteArray(size_t n, int32_t *arr, uint8_t *bytes)`	`// Serialize int32 array`
`void writeFloatToByteArray(float value, uint8_t *bytes)`	`// Serialize float to 4 bytes`
`void writeFloatArrayToByteArray(size_t n, float *arr, uint8_t *bytes)`	`// Serialize float array`

Who Calls DTypes.h?

DTypes is called by: Matmul.c (reading individual int32/float values during inner loop), Tensor.c (reading/writing tensor element data), Serialize.c (reading model weights from binary files), NPYLoader.c (reading .npy file data), and ExecuteOp.c (accessing operand values). It is a fundamental utility used throughout the library.

C Usage Example

`#include "DTypes.h"`	
`// Reading a float from a byte buffer (e.g. received over UART)`	
`uint8_t buf[4] = {0x3F, 0x80, 0x00, 0x00};`	`// IEEE 754 for 1.0f`
`float val = readBytesAsFloat(buf);`	`// val == 1.0f`
`// Reading an int32 from a byte buffer`	
`uint8_t ibuf[4] = {0x0C, 0x00, 0x00, 0x00};`	`// Little-endian 12`

```
int32_t mantissa = readBytesAsInt32(ibuf);          // mantissa == 12

// Writing a float to a byte buffer (for transmission)
float weight = 0.048f;
uint8_t out[4];
writeFloatToByteArray(weight, out);                 // out = {0x3D, 0x44, 0x9B, 0xA6}
```

2.2 Interactions with Other Files and STM32

Dependency Graph

DTypes.h depends on: stddef.h (size_t), stdint.h (uint8_t, int32_t). DTypes.c additionally depends on: string.h (memcpy), Common.h (logging), Tensor.h (calcNumberOfElementsByTensor). Files that call DTypes: Matmul.c, Tensor.c, Serialize.c, NPYLoader.c, ExecuteOp.c, and most arithmetic files.

Hardware Context: Endianness and STM32

The STM32F7 (Cortex-M7) is a little-endian processor. When you store int32_t x = 12, the bytes in memory are: 0x0C 0x00 0x00 0x00 (least significant byte first). readBytesAsInt32 uses memcpy which preserves this byte order exactly. If communicating with a big-endian host (some network protocols), byte swapping would be needed — but within the ODT ecosystem, all data is little-endian.

```
// STM32F7 is little-endian:
int32_t x = 12; // 0x0000000C
// Memory layout at &x:
// Address+0: 0x0C (least significant byte)
// Address+1: 0x00
// Address+2: 0x00
// Address+3: 0x00 (most significant byte)

// memcpy reads/writes these bytes in order:
memcpy(&x, bytes, 4); // safe on any alignment (MCU may fault on unaligned access)
```

WARNING Direct pointer casting (*(int32_t*)bytes) causes undefined behavior via strict aliasing and may cause hardware faults on architectures requiring alignment. DTypes.c correctly uses memcpy() throughout to avoid both issues. The STM32F7 can handle some unaligned accesses in software, but at a performance penalty.

2.3 Line-by-Line Explanation

readBytesAsInt32()

```
int32_t readBytesAsInt32(uint8_t *bytes) {
int32_t x;                                  // Stack-allocated 4-byte variable
memcpy(&x, bytes, sizeof(int32_t));         // Copy 4 bytes from bytes -> x
return x;                                   // Return the interpreted integer
}
```

```
Example: bytes = {0x0C, 0x00, 0x00, 0x00}
After memcpy: x = 0x0000000C = 12 (little-endian)
```

sizeof(int32_t) is always 4 on any conforming C implementation. The memcpy reads exactly 4 bytes. The key guarantee is type-safe: the raw bit pattern of the 4 bytes is reinterpreted as int32_t without any arithmetic conversion.

readNumberOfBytesAsInt32()

```
int32_t readNumberOfBytesAsInt32(uint8_t *data, size_t n) {
int32_t output = 0;                    // Zero-initialize the 4-byte container
memcpy(&output, data, n);              // Copy only n bytes (n <= 4)
return output;
}

Example with n=2: data = {0xFF, 0x7F}
output = 0x00000000 initially
After memcpy(2 bytes): output = 0x00007FFF = 32767   // Zero-extended int16
```

This function handles sub-word reads. n=1 reads a uint8 as int32, n=2 reads a int16, n=3 reads a 3-byte integer, n=4 is the same as readBytesAsInt32. The zero-initialization ensures unused high bytes stay zero. This works correctly on little-endian systems (STM32F7) because the lowest-address bytes are the least significant.

WARNING readNumberOfBytesAsInt32 assumes LITTLE-ENDIAN memory layout. On a big-endian system, 2-byte copy into output=0 would populate the MOST significant bytes, giving a wrong result 256x too large.

readBytesAsFloat()

```
float readBytesAsFloat(uint8_t *bytes) {
float x;                               // 4-byte IEEE 754 single precision
memcpy(&x, bytes, sizeof(float));      // Copy raw bit pattern
return x;
}

Example: bytes = {0x00, 0x00, 0x80, 0x3F}   // Little-endian IEEE 754
Bit pattern = 0x3F800000
IEEE 754: sign=0, exp=127-127=0, mantissa=1.0 * 2^0 = 1.0
Result: x = 1.0f
```

CONCEPT Using memcpy to type-pun float<->bytes is the only standard-compliant way in C. Alternative: union {float f; uint32_t i;} -- valid in C99/C11. Alternative: *(float*)bytes -- undefined behavior due to strict aliasing. DTypes.c correctly uses memcpy.

readBytesAsFloatArray()

```
void readBytesAsFloatArray(size_t n, uint8_t *bytes, float
*out) {
for (size_t i = 0; i < n; i++) {
```

```
size_t byteIndex = i * sizeof(float);                    // 4 bytes per float
out[i] = readBytesAsFloat(&bytes[byteIndex]);
}
}

Example: n=3, bytes = [float1_bytes | float2_bytes |
float3_bytes]
i=0: byteIndex=0, reads bytes[0..3] -> out[0]
i=1: byteIndex=4, reads bytes[4..7] -> out[1]
i=2: byteIndex=8, reads bytes[8..11] -> out[2]
```

Optimization note: If bytes and out are guaranteed aligned and endianness matches, this entire function is equivalent to memcpy(out, bytes, n*sizeof(float)). However, the loop form is safer in the general case and the compiler typically vectorizes it with SIMD instructions on the Cortex-M7 FPU.

writeFloatToByteArray() and writeInt32ToByteArray()

```
void writeFloatToByteArray(float value, uint8_t *bytes) {
memcpy(bytes, &value, sizeof(float));                    // Copy float bit pattern to bytes
}

Example: value = 1.0f (0x3F800000)
bytes[0] = 0x00 (LSB, little-endian)
bytes[1] = 0x00
bytes[2] = 0x80
bytes[3] = 0x3F (MSB)
```

TIP writeFloat and readFloat are exact inverses. writeFloatToByteArray(x, buf) followed by readBytesAsFloat(buf) returns exactly x with no loss of precision. This is critical for model checkpoint serialization: weights must round-trip exactly.

2.4 Missing RigL Code

DTypes.h/c does not need RigL-specific changes. However, the RigL implementation will use DTypes functions to access weight and gradient values in MinMax operations. Specifically, findAbsKthSmallestActive() reads float weights using direct pointer access (float*)weights->data since SYM_INT32 grad tensors store int32 mantissas. The relevant context:

```
// In rigLStep() and MinMax helper functions:
float *w = (float *)weights->data;                       // Direct float* access for FLOAT32
weights
float *g = (float *)grads->data;                         // Direct float* access for FLOAT32
grads

// For SYM_INT32 weights, read mantissa as int32:
int32_t m = readBytesAsInt32(&weights->data[i *
sizeof(int32_t)]);
float real_w = (float)m * scale;                         // Reconstruct real value for
comparison
```

```
// Note: RigL threshold comparisons work on absolute values of REAL weights,
// not mantissas. Scale matters for the DROP/GROW threshold.
```

For the initial RigL implementation, weights are assumed to be FLOAT32 (direct float* access). Extending to SYM_INT32 weights requires reconstructing real values from mantissa * scale before threshold comparisons. This is a planned enhancement.

Chapter 3

Tensor.h / Tensor.c

The Central Data Structure for All Neural Network Computations

3.1 Usage of This File

Tensor.h defines the core data structure of the ODT library. Everything - weights, activations, gradients, masks - is a tensor_t. The struct wraps a raw byte array (data) with shape metadata (shape_t) and quantization information (quantization_t).

Key Structs

`shape_t { numberOfDimensions, *dimensions, *orderOfDimensions }`	`// Rank and layout`
`tensor_t { *data, *shape, *quantization, *sparsity }`	`// The universal container`
`parameter_t { *param, *grad }`	`// Weight + gradient pair`
`sparsity_t { type, *config }`	`// Future sparse storage config`

Key Functions

`void setTensorValues(t,data,shape,q,sparsity)`	`// Wire up tensor struct (no alloc)`
`size_t calcNumberOfElementsByTensor(t)`	`// Product of all dimensions`
`size_t calcBytesPerTensor(t)`	`// Total bytes needed for data buffer`
`size_t calcBytesPerElement(q)`	`// Bytes per element (dtype-dependent)`
`void transposeTensor(t, dim0, dim1)`	`// O(1) logical transpose - no copy`
`bool tensorBoolGet(t, flatIndex)`	`// Read bit from BOOL tensor (RigL mask)`
`void tensorBoolSet(t, flatIndex, value)`	`// Write bit to BOOL tensor (RigL mask)`
`void copyTensor(dest, src)`	`// Deep copy all tensor data`
`void printTensor(t)`	`// Debug print (uses printf)`

C Usage Example

`// Create a 128x64 float32 weight tensor:`	
`uint8_t weightData[128*64*4];`	`// 32768 bytes on stack/SRAM`
`size_t dims[2] = {128, 64}; size_t order[2];`	
`shape_t wShape; quantization_t wQ; tensor_t wTensor;`	
`setOrderOfDimsForNewTensor(2, order);`	`// Sets row-major order: {0,1}`
`setShape(&wShape, dims, 2, order);`	
`initFloat32Quantization(&wQ);`	`// FLOAT32 type, qConfig=NULL`
`setTensorValues(&wTensor, weightData, &wShape, &wQ, NULL);`	
`size_t n = calcNumberOfElementsByTensor(&wTensor);`	`// n = 8192`

```
size_t b = calcBytesPerTensor(&wTensor);          // b = 32768
```

3.2 Interactions with Other Files and STM32

Tensor.h depends only on Quantization.h. Every other ODT file depends on Tensor.h. It is the universal language of the library.

Hardware Context: STM32F746ZG Memory Map

```
// STM32F746ZG memory regions for tensor data:
0x20000000: DTCM-RAM 64 KB -- fast, for scratch buffers
0x20010000: SRAM1 240 KB -- main weight/activation storage
0x20050000: SRAM2 16 KB -- small auxiliary tensors
0x08000000: Flash 1 MB -- read-only inference weights

// tensor_t struct itself: ~32 bytes (4 pointers)
// data buffer: the heavy allocation (e.g. 32768 bytes for
128x64 float)
// Cortex-M7 FPU requires 4-byte alignment for float access
```

CONCEPT tensor_t is a SHALLOW structure. It holds pointers, not copies. setTensorValues() does NOT allocate memory - the caller manages all buffers. This enables static allocation on MCU without a heap allocator.

WARNING Never free the data buffer while a tensor_t pointing to it is still in use. The tensor holds a raw pointer with no reference counting.

3.3 Line-by-Line Explanation

transposeTensor() -- O(1) Logical Transpose

```
void transposeTensor(tensor_t *tensor, size_t dim0, size_t
dim1) {
// Swap dimension sizes
size_t tmp = tensor->shape->dimensions[dim0];
tensor->shape->dimensions[dim0] =
tensor->shape->dimensions[dim1];
tensor->shape->dimensions[dim1] = tmp;
// Swap order entries
size_t otmp = tensor->shape->orderOfDimensions[dim0];
tensor->shape->orderOfDimensions[dim0] =
tensor->shape->orderOfDimensions[dim1];
tensor->shape->orderOfDimensions[dim1] = otmp;
}

Example: 3x4 matrix (dims={3,4}, order={0,1})
After transposeTensor(t,0,1): dims={4,3}, order={1,0}   // Data bytes unchanged!
```

This is used constantly in linearForwardFloat(): transposeTensor(w,0,1) before matmul, then transposeTensor(w,0,1) again after. The double-transpose restores the original shape. No data is moved - only 4 integers are swapped.

tensorBoolGet() and tensorBoolSet() -- RigL Mask Interface

```
// BOOL tensor packs 8 bits per byte:
// Element i is at: byte[i/8], bit position [i%8]

bool tensorBoolGet(tensor_t const *t, size_t i) {
return (t->data[i/8] >> (i%8)) & 1;                    // Extract bit i%8 from byte i/8
}

void tensorBoolSet(tensor_t *t, size_t i, bool v) {
if (v) t->data[i/8] |= (1u << (i%8));                  // Set bit
else t->data[i/8] &= ~(1u << (i%8));                   // Clear bit
}

Memory: 8192 weights -> 8192/8 = 1024 bytes for BOOL mask   // vs 32768 for float mask
```

CONCEPT Bit-packed BOOL tensors save 32x memory vs float32 masks. For a 128x64 layer, the RigL mask is 1 KB vs 32 KB. This is critical on 320 KB MCU where every kilobyte counts.

calcBytesPerElement() and calcBitsPerElement()

```
size_t calcBitsPerElement(quantization_t *q) {
switch(q->type) {
case FLOAT32: return 32;                               // 4 bytes per float
case INT32: return 32;                                 // 4 bytes per int32
case SYM_INT32:return 32;                              // 4 bytes per mantissa
case BOOL: return 1;                                   // 1 bit per mask element
case SYM: return ((symQConfig_t*)q->qConfig)->qBits;   // e.g. 8 bits
case ASYM: return ((asymQConfig_t*)q->qConfig)->qBits;  // e.g. 8 bits
}
}
```

byteConversion() -- Sub-Byte Packing

byteConversion() handles conversion between different bit widths. For example, converting from 32-bit float representation to 4-bit packed storage. Each element is read at dataInBits granularity and written at dataOutBits granularity.

```
void byteConversion(uint8_t *in, size_t inBits,
uint8_t *out, size_t outBits, size_t n)
// Example: convert 8 int32 mantissas to 4-bit packed
// inBits=32, outBits=4, n=8
// Input: [0x00000003, 0x00000007, ...] (8 x 32-bit)
// Output: [0x73, ...] (4 x 8-bit, 2 nibbles per byte)
```

3.4 Missing RigL Code

Tensor.h already has tensorBoolGet/Set. The missing piece is the BOOL tensor initialization pattern for the weight mask. Complete pattern:

```
// Complete RigL mask creation for 128x64 linear layer:
static uint8_t maskData[128*64/8]; // 1024 bytes
static size_t maskDims[2]={128,64}; size_t maskOrd[2];
static shape_t maskShape; static quantization_t maskQ;
static tensor_t weightMask;
memset(maskData, 0, sizeof(maskData));          // All inactive initially
setOrderOfDimsForNewTensor(2, maskOrd);
setShape(&maskShape, maskDims, 2, maskOrd);
initBoolQuantization(&maskQ);
setTensorValues(&weightMask, maskData, &maskShape, &maskQ,
NULL);
bernoulliFillMask(&weightMask, 0.10f);          // 10% weights active at start
```

RigL: Mask Memory Cost	A 128x64 RigL mask needs 1024 bytes. A full 4-layer network (128x64, 64x32, 32x16, 16x6) needs (8192+2048+512+96)/8 = 1356 bytes total for all masks. This is negligible on 320 KB SRAM.

Chapter 4

Quantization.h / Quantization.c

Quantization Types, Scale Factors, and Config Initialization

4.1 Usage of This File

Quantization.h defines the quantization type system of the ODT library. Six types are supported: FLOAT32 (IEEE 754, no compression), INT32 (raw signed integers), BOOL (1-bit packed, for RigL masks), SYM_INT32 (symmetric int32 with scale, for integer training), SYM (symmetric packed sub-byte), and ASYM (asymmetric with zero-point).

`typedef enum qtype { INT32, FLOAT32, SYM_INT32, SYM, ASYM, BOOL } qtype_t;`	
`symInt32QConfig_t { float scale; roundingMode_t rm; uint8_t qMaxBits; }`	`// Integer training config`
`symQConfig_t { float scale; uint8_t qBits; roundingMode_t rm; }`	`// Symmetric packed config`
`asymQConfig_t { float scale; int32_t zeroPoint; uint8_t qBits; rm; }`	`// Asymmetric config`
`quantization_t { qtype_t type; void *qConfig; }`	`// Polymorphic wrapper`

Init Functions

`void initFloat32Quantization(quantization_t *q)`	`// type=FLOAT32, qConfig=NULL`
`void initInt32Quantization(quantization_t *q)`	`// type=INT32, qConfig=NULL`
`void initBoolQuantization(quantization_t *q)`	`// type=BOOL, qConfig=NULL`
`void initSymInt32QConfig(roundingMode_t, symInt32QConfig_t *c)`	`// scale=1, qMaxBits=12`
`void initSymInt32Quantization(symInt32QConfig_t *, quantization_t *)`	`// Wire config to quant`
`void initSymQConfig(uint8_t qBits, roundingMode_t, symQConfig_t *)`	`// Packed symmetric`
`void initAsymQConfig(uint8_t qBits, roundingMode_t, asymQConfig_t *)`	`// Asymmetric`

C Usage Example

`// Create SYM_INT32 quantization for integer training:`	
`symInt32QConfig_t wQCfg;`	
`initSymInt32QConfig(SR_HALF_AWAY, &wQCfg);`	`// scale=1.0, qMaxBits=12, SR rounding`
`quantization_t wQ;`	
`initSymInt32Quantization(&wQCfg, &wQ);`	`// type=SYM_INT32, qConfig=&wQCfg`
`// Create BOOL quantization for RigL mask:`	

```
quantization_t maskQ;
initBoolQuantization(&maskQ);  // type=BOOL, qConfig=NULL
```

4.2 Interactions with Other Files and STM32

Quantization.h depends on: Rounding.h (roundingMode_t). Files that depend on Quantization.h: Tensor.h (quantization_t is a field of tensor_t), TensorConversion.c (conversionMatrix uses qtype_t to select conversion function), ExecuteOp.c (checks qtype_t for prologue/epilogue dispatch), Matmul.c (reads scale from symInt32QConfig_t), Sgd.c, AdamW.c.

> **CONCEPT** ODT_SYM_OPERAND_QMAXBITS defaults to 12. This means SYM_INT32 operands in matmul have mantissas in range [-2^11, 2^11-1] = [-2048, 2047]. The product 2048*2047 = 4,192,256 fits in int32 with headroom (INT32_MAX = 2,147,483,647). This prevents overflow in the matmul inner loop accumulator.

4.3 Line-by-Line Explanation

initSymInt32QConfig() -- The Most Important Init

```
void initSymInt32QConfig(roundingMode_t rm, symInt32QConfig_t *c) {
    c->roundingMode = rm;  // SR_HALF_AWAY for training, HALF_AWAY for inference
    c->scale = 1.f;  // Will be updated by calibration or matmul
    c->qMaxBits = ODT_SYM_OPERAND_QMAXBITS;  // Default: 12 bits
}

// WithQMaxBits variant adds a safety check:
if (qMaxBits > 31) { PRINT_ERROR(...); exit(1); }  // qMaxBits=32 would cause int32 cast UB
```

The qMaxBits=12 default (ODT_SYM_OPERAND_QMAXBITS) is a critical correctness contract. If two int12 mantissas are multiplied, the product fits in int32: 2^12 * 2^12 = 2^24 = 16,777,216 << 2^31 = 2,147,483,648. With qMaxBits=16, the product could be 2^32 which overflows int32.

initAsymQConfig() -- Range Check

```
void initAsymQConfig(uint8_t qBits, roundingMode_t rm, asymQConfig_t *c) {
    if (qBits == 0 || qBits > 30) { PRINT_ERROR(...); exit(1); }  // Prevent UB on cast
    c->qBits = qBits;
    c->roundingMode = rm;
    c->scale = 1.f;  // Updated during calibration
    c->zeroPoint = 0;  // Updated during calibration
}
```

WARNING	qBits > 30 for ASYM causes (int32_t)(pow(2, qBits)-1) to overflow int32 in the quantizer. The 30-bit limit is a safety guard documented in bug #246. Always use initAsymQConfig() -- never set qBits directly.

The polymorphic void *qConfig pattern

quantization_t uses void *qConfig as a type-erased pointer. The consumer casts it based on qtype_t. This is a classic C polymorphism pattern:

```
// In Matmul.c:
if (q->type == SYM_INT32) {
symInt32QConfig_t *qc = (symInt32QConfig_t *)q->qConfig;  // Safe cast
float scale = qc->scale;
}
// For FLOAT32/INT32/BOOL: qConfig is NULL -- never dereference!
```

4.4 Missing RigL Code

Quantization.h is complete for RigL. The BOOL type already exists and initBoolQuantization() sets it up correctly. The only addition needed is ensuring that rigLStep() and serializeSparsity() check for BOOL type correctly:

```
// In rigLStep(), check that mask is BOOL type:
if (mask->quantization->type != BOOL) {
PRINT_ERROR("rigLStep: mask must be BOOL tensor");
exit(1);
}

// In serializeSparsity() (Serialize.c fix):
if (mask == NULL || mask->quantization->type != BOOL) {
uint8_t noMask = 0; fwrite(&noMask, 1, 1, fp);
} else {
uint8_t hasMask = 1; fwrite(&hasMask, 1, 1, fp);
size_t n = calcNumberOfElementsByTensor(mask);
fwrite(mask->data, 1, (n+7)/8, fp);  // Write bit-packed mask
}
```

Chapter 5

Rounding.h / Rounding.c

Stochastic Rounding and Scale Rescaling for Integer Training

5.1 Usage of This File

Rounding.h/c provides the mathematical core of FQT: the ability to round floats to integers using either standard half-away rounding or stochastic rounding. Stochastic rounding is critical for escaping the quantization dead zone in integer training. Additionally, rescaleIntoAccumulatorScale() handles the bias rescaling needed when fusing bias addition into matmul.

```
typedef enum roundingMode { HALF_AWAY, SR_HALF_AWAY } roundingMode_t;
int32_t roundByMode(float input, roundingMode_t mode)  // Dispatch to HALF_AWAY or SR
int32_t rescaleIntoAccumulatorScale(int32_t paramQ, float paramScale,
float accumulatorScale, roundingMode_t)  // Bias rescaling
int32_t clampInt32(int32_t input, int32_t min, int32_t max)  // Clamp integer
float clamp(float input, float min, float max)  // Clamp float
```

C Usage Example

```
// Standard rounding (inference, deterministic):
int32_t m = roundByMode(0.075f, HALF_AWAY);  // m = 0 (dead zone)

// Stochastic rounding (training, escapes dead zone):
rngSetSeed(42);  // Set RNG seed for reproducibility
int32_t m = roundByMode(0.075f, SR_HALF_AWAY);  // m = 0 or 1 probabilistically
// P(m=1) = 7.5%, P(m=0) = 92.5%, E[m] = 0.075 (unbiased)
```

5.2 Interactions with Other Files and STM32

Rounding.h depends on: stdint.h. Rounding.c depends on: math.h (round(), fabsf()), RNG.h (rngNextFloat()), Common.h. Files that call rounding: TensorConversion.c (every quantize operation), ExecuteOp.c (the OUT_WRITE epilogue), Sgd.c (via optimizer write-back), AdamW.c.

Hardware: FPU and the round() Function

The Cortex-M7 FPU handles float operations in hardware. The C standard round() function computes half-away-from-zero rounding. On Cortex-M7, this compiles to a sequence of FPU instructions. For SR, rngNextFloat() adds a float in [-0.5,0.5) before calling round() - this is a single extra FADD instruction.

5.3 Line-by-Line Explanation

roundHalfAway() and roundSRHalfAway()

```
int32_t roundHalfAway(float input) {
return round(input);                                    // C standard round(): half-away from zero (C17 7.12.9.6)
}

int32_t roundSRHalfAway(const float input) {
return roundHalfAway(input + randfloat() - 0.5f);
}

float randfloat() {
return rngNextFloat();                                  // Returns uniform float in [0, 1)
}

// Noise = rngNextFloat() - 0.5 is in [-0.5, 0.5)
// P(round(0.075 + noise) = 1) = P(noise > 0.425) = P(U > 0.925) = 7.5%
```

The XorShift32 RNG in RNG.c produces uniform integers which are scaled to [0,1) by dividing by 2^24. This gives approximately 16 million distinct noise values. The noise is fine-grained enough to make SR statistically smooth over typical batch sizes.

CONCEPT Why rngNextFloat()-0.5 and not rngNextFloat()? The noise must be centered at zero to keep the rounding unbiased. If noise were uniform in [0,1), then round(x+noise) would always round UP for any x > 0, biasing the quantization.

rescaleIntoAccumulatorScale()

```
int32_t rescaleIntoAccumulatorScale(int32_t paramQ, float paramScale,
float accumulatorScale, roundingMode_t rm) {
float rescaled = (float)paramQ * paramScale / accumulatorScale;
return roundByMode(rescaled, rm);
}

// Example: bias mantissa = 100, bias scale = 0.01
// Accumulator scale = scale_W * scale_X = 0.004 * 0.002 = 0.000008
// rescaled = 100 * 0.01 / 0.000008 = 125000
// seed = round(125000) = 125000 (added to matmul accumulator)
```

This function is used in matmulSymInt32TensorsWithBias() to convert the bias (at its own scale) into the accumulator scale (which is scale_A * scale_B). The result is an integer that can be added directly to the int32 accumulator in the matmul inner loop.

clampInt32() and clamp()

```
int32_t clampInt32(int32_t input, int32_t min, int32_t max) {
if (input < min) return min;
```

```
if (input > max) return max;
return input;
}
// Used in TensorConversion.c to prevent quantized values from exceeding
// the representable range of the target dtype.
// Example: for 8-bit SYM: clamp to [-127, 127]
```

5.4 Missing RigL Code

Rounding.h/c is complete for RigL. The rigLStep() function does not use rounding: threshold comparisons use raw float absolute values, not quantized mantissas. However, the GROW step sets w[newly_active]=0.0f -- if weights are SYM_INT32, the corresponding mantissa is set to 0 (the integer zero, which decodes to real value 0*scale=0). This is handled by:

```
// In rigLStep() GROW step for SYM_INT32 weights:
// Set mantissa of newly active weight to 0:
int32_t zero = 0;
writeInt32ToByteArray(zero, &weights->data[i * sizeof(int32_t)]);  // mantissa = 0
// Real value: 0 * scale = 0.0 (correct: new connections start at zero)
```

Chapter 6

ExecuteOp.h / ExecuteOp.c

The Universal Operation Funnel: Prologue, Kernel, Epilogue

6.1 Usage of This File

ExecuteOp is the single conversion funnel through which every operation in the ODT library passes. It solves the mixed-precision problem: an operation may have inputs in one dtype and an output in another. ExecuteOp handles conversion automatically so kernels work in one arithmetic type.

The three-phase design: (1) PROLOGUE: convert inputs to the arithmetic dtype; (2) KERNEL: compute in the arithmetic dtype, writing a raw intermediate; (3) EPILOGUE: convert intermediate to the target dtype (OUT_WRITE) or accumulate into it (OUT_ACC_*).

```
typedef enum {
OUT_WRITE, // Overwrite target in its own dtype
OUT_ACC_DYNAMIC_RESCALE, // Accumulate: dequant+add+fresh scale
OUT_ACC_FIXED_SCALE, // Accumulate: rescale into existing scale
} outputMode_t;

typedef struct opSpec {
opKernelFn_t kernel; // Computation: operands->rawOut
const void *ctx; // Kernel config (geometry, etc)
tensor_t **inputs; // Operand tensors
size_t nInputs;
arithmetic_t arithmetic; // ARITH_FLOAT32 or ARITH_SYM_INT32
outputMode_t mode; // How to write the output
bool writesInPlaceSafe; // Kernel reads i before writing i
} opSpec_t;

void executeOp(const opSpec_t *spec, tensor_t *target);
```

Who Uses ExecuteOp?

Every layer forward and backward function: linearForward(), linearBackward(), sgdStepM(), adamWStep(), conv1dForward(), layerNormForward(), etc. ExecuteOp is the backbone of the computation graph.

C Usage Example: Linear Forward

```
// Linear forward pass via executeOp:
executeOp(
&(opSpec_t){
.kernel = linearForwardKernelFloat,  // Calls matmulFloat32TensorsWithBias
```

```
.inputs = (tensor_t*[]){input, weights, bias},
.nInputs = 3,
.arithmetic = {ARITH_FLOAT32, HALF_AWAY},
.mode = OUT_WRITE,
},
output);                                    // Target tensor (gets written in its own dtype)
```

6.2 Interactions with Other Files and STM32

ExecuteOp.h depends on: ArithmeticType.h (arithmetic_t), Tensor.h, Common.h. ExecuteOp.c additionally depends on: TensorConversion.c (conversionMatrix), Add.c (accumulateTensorIntoFloat32Inplace), Rounding.c (rescaleIntoAccumulatorScale). It is depended upon by every layer and optimizer file.

CONCEPT ExecuteOp solves the N^2 conversion problem. Without it, each layer would need to implement its own dtype dispatch (float->int, int->float, sym->float, etc.) - that is N^2 conversions for N dtypes. With ExecuteOp, each layer implements ONE kernel in its arithmetic type, and the funnel handles all conversions.

6.3 Line-by-Line Explanation

The Three Output Modes

```
OUT_WRITE: overwrite target
Used for: forward passes, first gradient write
Epilogue: convertTensor(rawOut -> target)
Rounding: uses arithmetic.roundingMode (NOT target qConfig rm)

OUT_ACC_DYNAMIC_RESCALE: Strategy A accumulation
Used for: weight gradient accumulation across mini-batch
Epilogue: dequant both -> float add -> re-derive absmax scale

OUT_ACC_FIXED_SCALE: Strategy B accumulation
Used for: bias gradient accumulation
Epilogue: rescale increment into targets existing scale, int-add
```

The Prologue: Converting Inputs

When operand dtype != arithmetic dtype, ExecuteOp creates a stack-allocated scratch tensor and converts the operand into it. This means the kernel always receives inputs in the correct arithmetic dtype. The original tensor is never modified.

```
// Simplified prologue logic:
for each input operand:
if (operand->quantization->type == arithmetic.type):
```

```
pass operand directly to kernel (no copy)
else:
allocate scratch[nElements * bytesPerElem] on stack
convertTensor(operand -> scratch)
pass scratch to kernel
```

> **WARNING** The scratch tensors are stack-allocated via C99 VLAs (variable-length arrays). For large tensors, this can overflow the MCU stack (typically 8 KB on STM32). The writesInPlaceSafe=true optimization avoids this by allowing the kernel to write directly to the target.

executeOpIdentityKernel()

```
void executeOpIdentityKernel(tensor_t **ops, size_t n,
tensor_t *rawOut, ...)
// Copies operand 0 into rawOut:
size_t bytes = calcNumberOfBytesForData(src->quantization,
n_elem);
memcpy(rawOut->data, src->data, bytes);
// For SYM_INT32: also copy scale:
rawOut_scale = src_scale;
```

6.4 Missing RigL Code

ExecuteOp.h does not need changes for RigL. The RigL mask-aware matmul operates at a level below executeOp - it modifies the kernel (matmulFloatCore) rather than the funnel. However, rigLStep() itself could be structured as an executeOp call if desired:

```
// Alternative: rigLStep as an executeOp call (not required):
// The simpler approach is a direct function call since
// rigLStep operates on the model structure, not individual
tensors.
// rigLStep(model, numLayers, sparsityTarget, step,
totalSteps);
// This is a model-level operation, not a tensor-level
operation.
```

Chapter 7

MinMax.h / MinMax.c

Finding Extrema in Tensors — Core of Quantization Calibration and RigL

7.1 Usage of This File

MinMax.h provides functions for finding minimum and maximum values in tensor data. These are used in quantization calibration (deriving the scale factor from the data range) and will be extended for RigL (finding the K-th smallest/largest values for mask updates).

`float absFloat32(float a)`	`// Returns |a| using fabsf()`
`float maxFloat32s(float a, float b)`	`// Returns max(a, b)`
`float findAbsMaxFloat(uint8_t *bytes, size_t n)`	`// Find max |value| in float array`
`float findMaxFloat(uint8_t *bytes, size_t n)`	`// Find max value in float array`
`float findMinFloat(uint8_t *bytes, size_t n)`	`// Find min value in float array`
`int32_t findMaxInt32(uint8_t *bytes, size_t n)`	`// Find max in int32 array`
`int32_t findMinInt32(uint8_t *bytes, size_t n)`	`// Find min in int32 array`

Who Calls MinMax?

TensorConversion.c (deriving scale from absmax for SYM quantization), ExecuteOp.c (dynamic rescale accumulation uses absmax to set new scale), and -- once implemented -- rigLStep() will call the new K-selection functions.

7.2 Interactions with Other Files and STM32

MinMax.c depends on: math.h (fabsf()), DTypes.h (readBytesAsFloat, readBytesAsInt32). It has no dependencies on Tensor.h itself -- all functions take raw uint8_t* pointers and element counts.

WARNING MinMax functions take raw uint8_t* pointers, NOT tensor_t*. The caller must extract tensor->data and pass calcNumberOfElementsByTensor(). This is intentional: MinMax is a low-level utility that works on raw byte arrays.

7.3 Line-by-Line Explanation

findAbsMaxFloat() -- Used in SYM Calibration

`float findAbsMaxFloat(uint8_t *bytes, size_t n) {`	
`float *values = (float *)bytes;`	`// Reinterpret bytes as float array`
`float max = fabsf(values[0]);`	`// Start with first element`
`for (size_t i = 1; i < n; i++) {`	
`if (fabsf(values[i]) > max) max = fabsf(values[i]);`	`// Track running max`

```
}
return max;
}

// Calibration usage:
float absMax = findAbsMaxFloat(weights->data, n);
scale = absMax / (float)((1 << (qBits-1)) - 1);   // e.g. absMax/127 for INT8
```

For SYM quantization, the scale is derived as absMax / (2^(qBits-1) - 1). If absMax=1.0 and qBits=8, scale = 1.0/127 = 0.00787. Then a weight of 0.5 maps to mantissa = round(0.5/0.00787) = 63.

findMaxFloat() and findMinFloat()

```
float findMaxFloat(uint8_t *bytes, size_t n) {
float max = readBytesAsFloat(&bytes[0]);   // Use DTypes for safe read
for (size_t i = 1; i < n; i++) {
float cur = readBytesAsFloat(&bytes[i * sizeof(float)]);
if (cur > max) max = cur;
}
return max;
}

// ASYM calibration: need both min and max:
float mn = findMinFloat(w->data, n);
float mx = findMaxFloat(w->data, n);
float scale = (mx - mn) / ((1 << qBits) - 1);   // e.g. range/255 for uint8
int32_t zp = round(-mn / scale);   // Zero-point offset
```

7.4 Missing RigL Code — K-Selection Functions

Two critical functions are missing from MinMax.c for RigL. These implement partial selection sort to find the K-th smallest/largest value in O(n*K) time, which is acceptable since K << n and called only every T steps.

RigL: findAbsKthSmallestActive	Finds the K-th smallest |w| among ACTIVE (mask=1) weights. Used in DROP step to set the threshold.

```
float findAbsKthSmallestActive(tensor_t *weights, tensor_t *mask, size_t K) {
size_t n = calcNumberOfElementsByTensor(weights);
float *w = (float *)weights->data;
// Count active weights
size_t count = 0;
for (size_t i = 0; i < n; i++) if (tensorBoolGet(mask, i)) count++;
if (K >= count) return 1e38f; // keep all: threshold = infinity
```

```
float *vals = malloc(count * sizeof(float));
if (!vals) { exit(1); }
size_t idx = 0;
for (size_t i = 0; i < n; i++)
if (tensorBoolGet(mask, i)) vals[idx++] = fabsf(w[i]);
// Partial selection sort: find K-th smallest (0-indexed)
for (size_t i = 0; i <= K && i < count; i++) {
size_t minIdx = i;
for (size_t j = i+1; j < count; j++)
if (vals[j] < vals[minIdx]) minIdx = j;
float tmp = vals[i]; vals[i] = vals[minIdx]; vals[minIdx] = tmp;
}
float thresh = vals[K < count ? K : count-1];
free(vals);
return thresh;
}
```

RigL: findAbsKthLargestInactive Finds the K-th largest |grad| among INACTIVE (mask=0) weights. Used in GROW step.

```
float findAbsKthLargestInactive(tensor_t *grads, tensor_t *mask, size_t K) {
size_t n = calcNumberOfElementsByTensor(grads);
float *g = (float *)grads->data;
size_t count = 0;
for (size_t i = 0; i < n; i++) if (!tensorBoolGet(mask, i)) count++;
if (K >= count) return 0.0f; // grow all: threshold = zero
float *vals = malloc(count * sizeof(float));
if (!vals) { exit(1); }
size_t idx = 0;
for (size_t i = 0; i < n; i++)
if (!tensorBoolGet(mask, i)) vals[idx++] = fabsf(g[i]);
// Partial selection sort descending: find K-th largest
for (size_t i = 0; i <= K && i < count; i++) {
size_t maxIdx = i;
for (size_t j = i+1; j < count; j++)
if (vals[j] > vals[maxIdx]) maxIdx = j;
float tmp = vals[i]; vals[i] = vals[maxIdx]; vals[maxIdx] = tmp;
```

```
}
float thresh = vals[K < count ? K : count-1];
free(vals);
return thresh;
}
```

Both functions use malloc/free because the size is not known at compile time. An MCU-safe alternative is a fixed-size stack buffer, but this requires knowing the maximum layer size at compile time. For the prototype, malloc is acceptable since rigLStep() is called infrequently (every T=100 steps).

WARNING	O(n*K) partial selection sort is O(n^2) in the worst case (K=n). For a 128x64 layer (n=8192) and K=819 (10% of active), this is 8192*819 = 6.7M comparisons per layer per RigL step. With a 216 MHz Cortex-M7, this takes <1 ms -- acceptable since RigL steps are rare.

Chapter 8

Matmul.h / Matmul.c

Matrix Multiplication: INT32, FLOAT32, SYM_INT32

8.1 Usage of This File

Matmul.c implements 2D matrix multiplication for three arithmetic types: FLOAT32 (standard single-precision FP), SYM_INT32 (integer mantissa with scale propagation), and INT32 (raw integer without scale). It is the computational core of every linear layer forward pass (Y = X * W^T) and backward pass (dW = E^T * X, dX = E * W).

The public API exposes five functions covering all combinations of bias fusion and arithmetic type. Internal helper functions matmulFloatCore() and matmulIntCore() implement the inner loop, shared by all public functions through executeOp(). This architecture allows the same kernel to be used with different prologue/epilogue conversions.

```
void matmulFloat32Tensors(A,B,out);                    // FLOAT32 C=A*B
void matmulFloat32TensorsWithBias(A,B,out,bias);       // FLOAT32 C=A*B+bias
void matmulSymInt32Tensors(A,B,out);                   // SYM_INT32: int matmul + scale multiply
void matmulSymInt32TensorsWithBias(A,B,out,bias);      // SYM_INT32 + bias seed
void matmulInt32Tensors(A,B,out);                      // Raw INT32 matmul
size_t getMatmulInstructionCounter();                  // Profiling: count inner loop executions
```

C Usage Example -- Linear Layer Forward Pass

In linearForwardFloat32(), the weight matrix is transposed O(1) before the matmul, allowing the matmul to operate row-by-row on the weight rows. The transpose is restored after the call. The bias is passed as a seed to matmulFloat32TensorsWithBias(), which initializes the accumulator with the bias value, fusing bias addition into the inner loop.

```
// Linear layer forward: Y = X * W^T + b

transposeTensor(weights, 0, 1);                        // O(1) logical transpose
matmulFloat32TensorsWithBias(input, weights, output, bias);  // Fused: Y=X*W^T+b
transposeTensor(weights, 0, 1);                        // Restore: avoids memory copy

// Dimensions: X=[batch,in], W=[out,in], W^T=[in,out]
// Output Y=[batch,out]
```

8.2 Interactions with Other Files and STM32

Matmul.c depends on: Arithmetic.h (calcElementIndexByIndices for stride computation), Common.h (PRINT_ERROR, debug macros), DTypes.h (readBytesAsFloat, readBytesAsInt32, writeFloat/Int32ToByteArray), Mul.h (mulInt32s), Rounding.h (roundByMode for SYM output), Tensor.h

(getDimensionsByIndex). Called by: Linear.c, LayerNorm.c.

On the Cortex-M7 FPU at 216 MHz, the inner loop VFMA (Fused Multiply-Add) instruction executes in 1 cycle after a 2-cycle latency. For a 64x1152 weight matmul (64 outputs, 1152 inputs, batch=1): 64*1152 = 73728 VFMA operations = ~73728 cycles = 341 microseconds. With 90% sparsity (RigL), only 7373 active weights: ~34 microseconds -- a 10x speedup.

CONCEPT	Scale propagation in SYM_INT32: after matmulIntCore(), the output mantissa is exact (no rounding). The output scale is set to scale_A * scale_B. This is mathematically exact: (mantissa_A * scale_A) * (mantissa_B * scale_B) = (mantissa_A * mantissa_B) * (scale_A * scale_B). The real-valued result is obtained by multiplying the output mantissa by the output scale.

WARNING	Integer overflow risk: matmulIntCore() accumulates int32_t values. For a 1152-element inner product with input mantissa up to ODT_SYM_OPERAND_QMAXBITS=12 bits (4096) and weight mantissa up to 12 bits (4096), the maximum single product is 4096*4096=16M. Accumulating 1152 such products: 1152*16M = 18.87 billion, which exceeds int32 max (2.1 billion). Use ODT_SYM_OPERAND_QMAXBITS=12 only for layers with < 128 inputs.

8.3 Line-by-Line Explanation

matmulIntCore() -- The Integer Inner Loop

matmulIntCore() is the shared kernel for matmulInt32Tensors() and matmulSymInt32Tensors(). It takes an optional seed array (the bias pre-converted to integer units) to initialize the accumulator, fusing bias addition into the multiply-accumulate loop.

```
static void matmulIntCore(A, B, out, int32_t *seed) {
aRows = getDimensionsByIndex(A, 0);                    // Batch size (or output features)
aColumns = getDimensionsByIndex(A, 1);                 // Input features
bColumns = getDimensionsByIndex(B, 1);                 // Output features
for (size_t row = 0; row < aRows; row++) {
for (size_t col = 0; col < bColumns; col++) {
int32_t result = seed ? seed[col] : 0;                 // Bias seed
for (size_t i = 0; i < aColumns; i++) {                // Inner dot product
int32_t a = readBytesAsInt32(A[row,i]);
int32_t b = readBytesAsInt32(B[i,col]);
result += mulInt32s(a, b);                             // MAC: no overflow for small inputs
}
writeInt32ToByteArray(result, out[row,col]);
}
}
}
```

matmulSymInt32Tensors() -- Scale Propagation

matmulSymInt32Tensors() calls matmulIntCore() for the integer computation, then sets the output quantization scale to scale_A * scale_B. This is the only post-computation step needed -- no rounding, no conversion. The output is a valid SYM_INT32 tensor with the correct scale.

```
void matmulSymInt32Tensors(A, B, out) {
matmulInt32Tensors(A, B, out);                    // Integer multiply-accumulate
// Set output scale: output_scale = scale_A * scale_B
symInt32QConfig_t *aQC = getSymInt32QConfig(A);
symInt32QConfig_t *bQC = getSymInt32QConfig(B);
symInt32QConfig_t *outQC = getSymInt32QConfig(out);
outQC->scale = aQC->scale * bQC->scale;           // Automatic scale tracking
}
// Example: scale_A=0.004, scale_B=0.002
// Product: 1000 * 500 = 500000 (integer)
// outQC->scale = 0.000008
// Real value: 500000 * 0.000008 = 4.0 (exact)
```

matmulFloatCore() -- FPU Inner Loop

matmulFloatCore() uses readBytesAsFloat() and VFMA instructions. GCC -O2 with -mfpu=fpv5-d16 unrolls the inner loop and generates VLDR+VFMA sequences. The instruction counter getMatmulInstructionCounter() allows profiling to verify that the expected number of multiply-accumulate operations are being executed.

```
static void matmulFloatCore(A, B, out, float *bias) {
for (size_t row = 0; row < aRows; row++) {
for (size_t col = 0; col < bColumns; col++) {
float result = bias ? bias[col] : 0.0f;
for (size_t i = 0; i < aColumns; i++) {           // Hot inner loop
float a = readBytesAsFloat(&A->data[...]);
float b = readBytesAsFloat(&B->data[...]);
result += a * b;                                  // VFMA on Cortex-M7
matmulInstructionCounter++;                       // Profiling
}
writeFloatToByteArray(result, &out->data[...]);
}
}
}
```

8.4 Missing RigL Code -- Mask-Aware Inner Loop

The critical RigL addition: check the weight mask before each multiply in the inner loop. Inactive weights (mask bit = 0) contribute zero and must be skipped. This is the change that makes 90% sparsity translate to ~10x computation speedup.

The mask index flatIdx = rowIndex * aColumns + col corresponds to element (row, i) in the weight matrix. This matches the layout because the weight matrix is stored as [out_neurons, in_neurons] and transposed (O(1)) before the matmul, making row=output_neuron index and i=input_neuron index.

RigL: Mask-Aware Matmul	Add tensorBoolGet() check in inner loop. For 90% sparsity: only 10% of inner loop iterations execute, giving ~10x speedup on the hottest code path.

```
// Modified matmulFloatCore() inner loop:
for (size_t i = 0; i < aColumns; i++) {
size_t flatIdx = rowIndex * aColumns + i;  // Weight position
if (weightMask != NULL &&
!tensorBoolGet(weightMask, flatIdx)) continue;  // Skip inactive
float aVal = readBytesAsFloat(&A->data[aByteIdx]);
float bVal = readBytesAsFloat(&B->data[bByteIdx]);
result += aVal * bVal;  // Only active weights contribute
}
```

The weightMask parameter must be threaded through from matmulFloat32TensorsWithMask() (a new function to add) all the way to matmulFloatCore(). Alternatively, the mask can be a global context variable set before calling the existing function -- but threaded parameters are cleaner and safer in re-entrant code.

TIP	Performance verification: after adding the mask check, verify that getMatmulInstructionCounter() returns approximately (sparsity fraction) * (dense count). For 90% sparse 64x1152 matmul: dense count = 73728, sparse count should be ~7373. If it returns 73728 still, the mask check is being optimized away or not reached.

Chapter 9

RNG.h / RNG.c + Bernoulli.h / Bernoulli.c

XorShift32 PRNG, Stochastic Rounding, and Bernoulli Mask Initialization

9.1 Usage of This File

RNG.h/c provides a XorShift32 pseudo-random number generator that is the foundation for stochastic rounding (SR_HALF_AWAY), dropout mask generation, and DataLoader shuffling. Bernoulli.h/c uses RNG to fill BOOL tensors with random binary patterns -- the exact operation needed to initialize RigL sparsity masks and dropout activity masks.

The XorShift32 algorithm was chosen for MCU deployment because it requires only one 32-bit state variable (4 bytes of SRAM), executes in exactly 5 instructions on Cortex-M7, and has a period of 2^32-1 = 4.29 billion, sufficient for all ODT use cases. It is not cryptographically secure, but that is not required for ML training.

`void rngSetSeed(uint32_t seed);`	`// Set global RNG state (0 maps to UINT32_MAX)`
`uint32_t rngNextUint32(void);`	`// Next raw 32-bit value from global state`
`float rngNextFloat(void);`	`// Next float in [0,1) from global state`
`float rngNextFloatCtx(rng32_t *rng);`	`// Thread-safe: caller-owned state`
`void rngShuffleIndices(size_t *indices, size_t n);`	`// Fisher-Yates in-place shuffle`
`void bernoulliFillMask(tensor_t *mask, float p);`	`// Fill BOOL tensor: bit=1 with prob p`
`void bernoulliFillMaskReference(tensor_t *mask, float p);`	`// Reference implementation`
`void bernoulliSetFillMaskFn(bernoulliFillMaskFn_t fn);`	`// Swap optimized implementation`

9.2 Interactions with Other Files and STM32

RNG.c has no dependencies beyond stdint.h (included via RNG.h). Bernoulli.c depends on RNG.h and Tensor.h. RNG is called by: Rounding.c (rngNextFloat() for SR noise), DataLoader.c (rngShuffleIndices() for epoch shuffling), Bernoulli.c (bernoulliFillMask() for masks), and PpcaReplay.c (rngNextFloatCtx() with private context).

On the STM32F746ZG, the global RNG state fits in a single CPU register during execution of rngNextUint32(). The 5-instruction sequence executes in 5 cycles at 216 MHz = 23 nanoseconds per random value. For stochastic rounding of a 1152-element gradient vector: 1152 SR calls = 26,600 cycles = 123 microseconds.

WARNING	The global RNG (rngNextFloat/rngNextUint32) is NOT thread-safe. In a multi-threaded MCU application (e.g., FreeRTOS with separate training and inference tasks), each task must use its own rng32_t context with rngNextFloatCtx(). Sharing the global RNG between tasks would cause race conditions and biased randomness.

CONCEPT	Why XorShift32 is sufficient for SR: stochastic rounding requires uniform noise in (-0.5, 0.5). XorShift32 produces uniform integers in [1, 2^32-1] (never 0). Converting to float: (x >> 8) / 2^24 gives values in [0.0, 1.0) with 16M distinct levels. Subtracting 0.5 gives (-0.5, 0.5). The 16M resolution is more than sufficient for float32 SR unbiasedness.

9.3 Line-by-Line Explanation

XorShift32 State Transition -- 5 Instructions

The XorShift32 algorithm is a 3-step XOR shift sequence. The magic constants (13, 17, 5) were found by George Marsaglia to produce maximal-period sequences passing basic statistical tests. The state is never zero because x^=x<<13 on x=0 returns 0 -- rngSetSeed(0) maps to UINT32_MAX to avoid the degenerate all-zero state.

```
static uint32_t rngNext(rng32_t *rng) {
uint32_t x = rng->state;                    // Load current state
x ^= x << 13;                               // XOR with left shift 13
x ^= x >> 17;                               // XOR with right shift 17
x ^= x << 5;                                // XOR with left shift 5
rng->state = x;                             // Store new state
return x;                                   // Return new state as random value
}
// Period: 2^32-1 (all states except 0 are visited)
// 5 instructions: 3x XOR, 2x shift (on Cortex-M7 EORS+LSL)
```

rngSetSeed() -- Safe Initialization

```
void rngSetSeed(uint32_t seed) {
globalRng.state = (seed == 0) ? UINT32_MAX : seed;
}
// rngSetSeed(0) -> state = 0xFFFFFFFF (not 0)
// This prevents the degenerate state where all subsequent
// values would be 0 (0 XOR 0 = 0 forever)
```

rngNextFloat() -- Float Conversion

The conversion from uint32 to float [0,1) uses bit manipulation: take the top 24 bits (right-shifting by 8), then divide by 2^24. This gives 16,777,216 distinct float values evenly spaced in [0, 1). The top 24 bits are used (not bottom) because XorShift32 has better statistical properties in high-order bits.

```
float rngNextFloat(void) {
return rngNextFloatCtx(&globalRng);
}
float rngNextFloatCtx(rng32_t *rng) {       // Thread-safe variant
uint32_t x = rngNext(rng);
// 0x1p24f = 2^24 = 16777216 as float literal
```

```
return (float)(x >> 8) / 0x1p24f;                    // Top 24 bits / 2^24
}
// Range: [0.0, 1.0) with 16M discrete values
// x >> 8 range: [0, 2^24-1] (since state != 0)
```

bernoulliFillMaskReference() -- Mask Initialization

bernoulliFillMask() is the primary function for initializing RigL weight masks. For a 1152-element weight vector with 10% target density (p=0.1), it sets approximately 115 bits to 1 (active) and 1037 bits to 0 (inactive). The exact count is Binomial(1152, 0.1) distributed -- not exactly 115.

```
static void bernoulliFillMaskReference(tensor_t *mask, float p) {
size_t n = calcNumberOfElementsByTensor(mask);       // Total bits
for (size_t i = 0; i < n; i++) {
bool active = (rngNextFloat() < p);                  // Bernoulli(p) draw
tensorBoolSet(mask, i, active);                      // Set bit i
}
}
// For p=0.1, n=1152: E[active] = 115.2, Std = 10.2
// bernoulliFillMask() calls this or a registered optimized fn
```

rngShuffleIndices() -- Fisher-Yates

Fisher-Yates produces a uniformly random permutation of indices. It is called by DataLoader.c before each epoch to shuffle the order in which training samples are presented. Presenting samples in random order prevents the model from memorizing sample ordering patterns.

```
void rngShuffleIndices(size_t *indices, size_t n) {
for (size_t i = n - 1; i > 0; i--) {                 // Backward iteration
size_t j = rngBounded(&globalRng, i + 1);            // j in [0, i]
// Swap indices[i] and indices[j]:
size_t tmp = indices[i];
indices[i] = indices[j];
indices[j] = tmp;
}
}
// rngBounded() uses rejection sampling to avoid modulo bias:
// biased: j = rngNextUint32() % (i+1) -- wrong for non-power-of-2
// unbiased: reject values >= floor(UINT32_MAX/(i+1)) * (i+1)
```

9.4 Missing RigL Code

RNG.c and Bernoulli.c are complete for RigL. The initial mask creation uses bernoulliFillMask(). During training, SR uses rngNextFloat() implicitly via roundByMode(). The RigL mask UPDATE (DROP/GROW)

uses deterministic threshold comparisons -- no random draws are needed during mask evolution.

```
// Complete RigL mask initialization sequence:
rngSetSeed(42);                                  // Reproducible mask + SR
bernoulliFillMask(&weightMask, 0.10f);           // 10% active at initialization
zeroInactiveWeights(cfg);                        // Enforce mask on weight tensor
// During training:
// SR (stochastic rounding) uses rngNextFloat() in Rounding.c
// rigLStep() uses findAbsKthSmallest/Largest -- deterministic
```

TIP For reproducible RigL experiments: always set the seed before bernoulliFillMask() AND before the training loop begins. The mask initialization consumes some RNG state, so subsequent SR draws depend on the order of operations. A fixed seed guarantees identical masks and identical SR sequences across runs.

Chapter 10

Layer.h / Layer.c

Polymorphic Layer System and Virtual Dispatch Vtable

10.1 Usage of This File

Layer.h defines the polymorphic layer system at the heart of the ODT neural network runtime. Every neural network layer is represented as a layer_t struct with a layerType_t enum and a layerConfig_t union pointer. This allows the training loop to treat all layers uniformly through a vtable without type-specific dispatch in the caller.

The design is C-friendly: no heap allocation is required for the vtable, and all layer configs can be statically allocated. On the STM32 with only 320 KB SRAM, avoiding runtime malloc() in the hot training loop is essential for reliability.

```
typedef enum layerType {                    // Wire-format tag serialized as uint8
LINEAR = 0,                                 // Fully-connected: Y = W*X + b
RELU,                                       // Element-wise max(0, x)
CONV1D,                                     // 1D temporal convolution
CONV1D_TRANSPOSED,                          // Transpose conv for upsampling
MAXPOOL1D,                                  // Max pooling over time axis
AVGPOOL1D,                                  // Average pooling
SOFTMAX,                                    // exp(x_i)/sum(exp(x_j))
FLATTEN,                                    // Reshape [C,T] to [C*T]
ADAPTIVE_AVGPOOL1D,                         // Output-size-specified avgpool
DROPOUT,                                    // Training-only stochastic zeroing
LAYERNORM,                                  // Normalize over feature dimension
GROUPNORM                                   // Normalize over channel groups
} layerType_t;                              // NEVER reorder: breaks serialized
                                            checkpoints
```

layer_t and layerConfig_t Union

Each layer in a model is an instance of layer_t. The type field determines which union member to use for config and which row of layerFunctions[] to call. The config union is a classic C type-erasure pattern: one pointer, many interpretations.

```
typedef struct layer {
layerType_t type;                           // Determines vtable index
layerConfig_t *config;                      // Layer-specific config struct
} layer_t;

typedef union layerConfig {                 // Type-erased config union
linearConfig_t *linear;                     // Linear layer
```

```
reluConfig_t *relu;                    // ReLU with threshold
conv1dConfig_t *conv1d;                // 1D conv
softmaxConfig_t *softmax;              // Softmax axis
dropoutConfig_t *dropout;              // Dropout rate
layerNormConfig_t *layerNorm;          // Norm config
} layerConfig_t;
```

Sequential Forward and Backward Pass

```
layer_t *model[3] = {&linear1, &relu1, &linear2};   // 3-layer model
tensor_t *cur = input;
for (size_t i = 0; i < 3; i++) {
layerFunctions[model[i]->type].forward(
model[i], cur, activations[i]);                     // Virtual dispatch
cur = activations[i];
}
for (int i = 2; i >= 0; i--) {                      // Backward: reverse order
layerFunctions[model[i]->type].backward(
model[i], fwdIn[i], lossGrad[i+1], lossGrad[i]);
}
```

10.2 Interactions with Other Files and STM32

Layer.h is the most widely included header in ODT. Layer.c populates the layerFunctions[] array with pointers from all layer implementation files. The vtable array is placed in Flash as const static data, so it consumes no SRAM.

Function pointer dispatch through the vtable adds 1-2 CPU cycles compared to direct calls. This overhead is completely negligible compared to the O(n^2) matmul inside each linear layer forward pass.

WARNING WIRE FORMAT: layerType_t enum values are serialized as uint8 tags in .odts checkpoints. The values LINEAR=0, RELU=1, CONV1D=2 are PERMANENT. Never insert or reorder enum members -- only append new types at the end. Reordering corrupts all saved model files.

CONCEPT The C vtable pattern avoids C++ overhead while achieving polymorphism. The vtable is a statically allocated array indexed by layerType_t -- O(1) dispatch with zero per-object overhead. Each layer_t is exactly 2 words: the type enum and the config pointer.

10.3 Line-by-Line Explanation

layerFunctions[] -- The Global Vtable

Layer.c defines a global array of layerFunctions_t structs indexed by layerType_t. Each struct has three function pointers: forward, backward (NULL for in-place activations like softmax), and calcOutputShape.

```
typedef struct layerFunctions {
```

```
void (*forward)(layer_t*, tensor_t*, tensor_t*);
void (*backward)(layer_t*, tensor_t*, tensor_t*, tensor_t*);
shape_t (*calcOutputShape)(layer_t*, shape_t);
} layerFunctions_t;

layerFunctions_t layerFunctions[] = {
[LINEAR] = { linearForward, linearBackward,
linearCalcOutputShape },
[RELU] = { reluForward, reluBackward, reluCalcOutputShape },
[CONV1D] = { conv1dForward, conv1dBackward,
conv1dCalcOutputShape },
[SOFTMAX] = { softmaxForward, NULL, softmaxCalcOutputShape }, // No separate backward
[DROPOUT] = { dropoutForward, dropoutBackward,
dropoutCalcOutputShape },
[LAYERNORM] = { lnForward, lnBackward, lnCalcOutputShape },
};
```

initLayer() -- Initialization Dispatch

```
void initLayer(layer_t *layer) {
switch (layer->type) {
case LINEAR:
linearInitConfig(layer->config->linear); // Set defaults
break;
case CONV1D:
conv1dInitConfig(layer->config->conv1d);
break;
default:
PRINT_ERROR('unknown type'); exit(1);
}
}
```

calcOutputShape() -- Static Shape Inference

calcOutputShape() pre-computes output tensor shapes at model build time, allowing all activation buffers to be allocated once before training starts. No malloc() is called during forward or backward passes.

```
shape_t inShape = input->shape;
for (size_t i = 0; i < numLayers; i++) {
shape_t out = layerFunctions[model[i]->type]
.calcOutputShape(model[i], inShape); // Infer output shape
activations[i] = allocateTensorByShape(out); // Pre-allocate buffer
inShape = out;
}
```

10.4 Missing RigL Code

Layer.h does not need changes for RigL. rigLStep() uses the type enum to find LINEAR layers and access their config->linear->weightMask. Non-linear layers are skipped. The vtable and layer_t struct are sufficient as-is.

```
for (size_t l = 0; l < numLayers; l++) {
if (model[l]->type != LINEAR) continue;  // Skip non-linear
linearConfig_t *cfg = model[l]->config->linear;
if (cfg->weightMask == NULL) continue;  // Skip unmasked layers
// Apply DROP and GROW to this layer's mask
}
```

TIP Future: Conv1d layers can benefit from RigL too. Add kernelMask to conv1dConfig_t and modify Conv1dKernel.c inner loop. The Layer.h vtable and layerType_t enum need no changes.

Chapter 11

Linear.h / Linear.c

Fully-Connected Layer: Forward, Backward, and Weight Gradient

11.1 Usage of This File

Linear.c implements the fully-connected (dense) layer: Y = X * W^T + b. It is the most important layer in ODT for the HAR use case, as the classification head uses linear layers to map feature vectors to class probabilities. Linear.c supports both FLOAT32 and SYM_INT32 arithmetic, and is the primary target for RigL weight sparsification.

The linear layer has three learnable parameter sets: weights W (shape [out_features, in_features]), bias b (shape [out_features]), and their respective gradients. For HAR with a 1152->64->6 architecture: W1 has 1152*64=73,728 weights (288 KB float32), W2 has 64*6=384 weights. With 90% RigL sparsity on W1, only 7,373 active weights need storage and computation.

`typedef struct linearConfig {`	
`parameter_t *weights; // [out_features, in_features]`	
`parameter_t *bias; // [out_features], optional`	
`bool hasBias;`	
`arithmetic_t arithType; // FLOAT32 or SYM_INT32`	
`tensor_t *weightMask; // BOOL [out*in bits] -- RigL mask`	`// MISSING: must add`
`} linearConfig_t;`	
`void linearForward(layer_t*, tensor_t *in, tensor_t *out);`	`// Y = X*W^T + b`
`void linearBackward(layer_t*, tensor_t *fwdIn, tensor_t *loss, tensor_t *propLoss);`	
`void linearCalcWeightGrads(layer_t*, tensor_t *fwdIn, tensor_t *loss);`	`// dW = E^T * X`
`void linearCalcBiasGrads(layer_t*, tensor_t *loss);`	`// dB = sum_batch(E)`
`shape_t linearCalcOutputShape(layer_t*, shape_t in);`	`// [batch, out_features]`
`void linearInitConfig(linearConfig_t *cfg);`	`// Set defaults, validate`

11.2 Interactions with Other Files and STM32

Linear.c depends on Matmul.c (matmulFloat32Tensors, matmulSymInt32Tensors), Tensor.h (transposeTensor), DTypes.h, ArithmeticType.h, ExecuteOp.h (writesInPlaceSafe check), Rounding.h (roundByMode for SGD write-back), and Common.h. It is called through the Layer.c vtable.

The linearForwardFloat32() function is the hottest code path in HAR training. For a 1152->64 layer with batch_size=1: 1152*64 = 73,728 VFMA operations + 64 bias adds = 341 microseconds at 216 MHz. With a 64->6 second layer: 384 additional VFMA = 1.8 microseconds. Total forward pass: ~343 microseconds for the two-layer head.

CONCEPT	transposeTensor() is O(1) because Tensor.h stores the transpose as a metadata swap, not a data copy. linearForwardFloat32() calls transposeTensor(W, 0, 1) to get W^T in [in,out] order, calls matmul(X, W^T, Y), then restores W to [out,in] with another transposeTensor(). Two O(1) metadata ops, zero data copies.

11.3 Line-by-Line Explanation

linearForwardFloat32() -- Forward Pass

```
void linearForwardFloat32(linearConfig_t *cfg,
tensor_t *input, tensor_t *output) {
// Step 1: Transpose weights O(1): [out,in] -> [in,out]
transposeTensor(cfg->weights->param, 0, 1);
// Step 2: Y = X * W^T (+ bias fused)
if (cfg->hasBias)
matmulFloat32TensorsWithBias(
input, cfg->weights->param, output, cfg->bias->param);
else
matmulFloat32Tensors(input, cfg->weights->param, output);
// Step 3: Restore weights: [in,out] -> [out,in]
transposeTensor(cfg->weights->param, 0, 1);
}
```

linearCalcWeightGradsFloat32() -- Weight Gradient

The weight gradient dW is computed as: dW = E^T * X, where E is the loss gradient at the output (shape [batch, out_features]) and X is the forward input (shape [batch, in_features]). The result dW has shape [out_features, in_features] -- same as the weight matrix.

```
void linearCalcWeightGradsFloat32(linearConfig_t *cfg,
tensor_t *fwdInput, tensor_t *lossGrad) {
// dW = E^T * X: [out,batch] * [batch,in] = [out,in]
transposeTensor(lossGrad, 0, 1);          // E: [batch,out] -> E^T: [out,batch]
matmulFloat32Tensors(lossGrad, fwdInput, cfg->weights->grad);  // dW += E^T * X
transposeTensor(lossGrad, 0, 1);          // Restore E
}
// Note: this ADDS to the existing gradient (accumulation
mode)
// Zero cfg->weights->grad before the batch if batch_size > 1
```

linearCalcPropLossFloat32() -- Propagated Loss

The propagated loss (gradient w.r.t. the layer input) is: dX = E * W, where E=[batch,out] and W=[out,in], giving dX=[batch,in]. This is passed to the previous layer's backward() as its incoming loss gradient.

```
void linearCalcPropLossFloat32(linearConfig_t *cfg,
```

```
tensor_t *lossGrad, tensor_t *propLoss) {
// dX = E * W: [batch,out] * [out,in] = [batch,in]
matmulFloat32Tensors(lossGrad, cfg->weights->param,
propLoss);
}
// No transpose needed: E=[batch,out] and W=[out,in]
// are already in the correct orientation for this matmul
```

linearInitConfig() -- Default Initialization

```
void linearInitConfig(linearConfig_t *cfg) {
if (cfg == NULL) { PRINT_ERROR('null cfg'); exit(1); }
// Set arithmetic type from weight quantization:
cfg->arithType = arithmeticFromQuantizationOrDefault(
cfg->weights->param->quantization);
cfg->weightMask = NULL;                    // Dense by default
cfg->hasBias = (cfg->bias != NULL);
// Validate weight dimensions:
if (cfg->hasBias) {
size_t outFeatures =
getDimensionsByIndex(cfg->weights->param, 0);
size_t biasFeatures = getDimensionsByIndex(cfg->bias->param,
0);
if (outFeatures != biasFeatures) {
PRINT_ERROR('bias/weight mismatch'); exit(1);
}
}
}
```

11.4 Missing RigL Code -- weightMask Field

The most critical RigL change is in Linear.h: add a tensor_t *weightMask field to linearConfig_t. This is the single change that enables all downstream RigL functionality -- the matmul mask check, the optimizer mask check, and the rigLStep() mask update all reference this field.

RigL: Add weightMask to linearConfig_t	In Linear.h: add `tensor_t *weightMask;` to linearConfig_t. In linearInitConfig(): add `cfg->weightMask = NULL;`. This null default means dense layers work unchanged. Set weightMask to a BOOL tensor to enable RigL for that layer.

```
// Linear.h change (add one field):
typedef struct linearConfig {
parameter_t *weights;
parameter_t *bias;
bool hasBias;
```

```
arithmetic_t arithType;
tensor_t *weightMask; // NEW: NULL = dense, non-NULL = RigL    // ADD THIS
} linearConfig_t;

// linearInitConfig() addition:
cfg->weightMask = NULL;                                        // Dense by default

// To enable RigL for a layer:
tensor_t *mask = createBoolTensor(outFeatures * inFeatures);
bernoulliFillMask(mask, 1.0f - targetSparsity);                // e.g., 0.1 for 90% sparse
cfg->weightMask = mask;                                        // Attach mask
```

TIP Memory for the weight mask: for a 64x1152 layer (90% sparse), the mask has 73,728 bits = 9,216 bytes = 9 KB. The sparse weights themselves: 10% * 73,728 * 4B = 29 KB (if stored densely with mask, or ~9 KB if using CSR format). Storing dense with mask is simpler and faster for random access.

Chapter 12

Optimizer.h + Sgd.h / Sgd.c

SGD with Momentum: Weight Update Kernel and Stochastic Rounding Write-Back

12.1 Usage of This File

Sgd.c implements Stochastic Gradient Descent with optional momentum. It is the optimizer used for FQT training. The key design: the update kernel works in FLOAT32 arithmetic only (currently), and write-back to quantized weights uses the optimizer writeBackRounding (SR_HALF_AWAY by default) to escape the dead zone.

`typedef struct sgd { float learningRate; float momentumFactor; float weightDecay; arithmetic_t updateMath; } sgd_t;`	
`void sgdInit(sgd_t*, float lr, float momentum, float wd, arithmetic_t)`	`// Initialize SGD`
`void sgdStepM(optimizer_t *optimizer)`	`// Perform one SGD step (handles both plain and momentum)`
`float sgdGetLr(optimizer_t*)`	`// Get current learning rate`
`void sgdSetLr(optimizer_t*, float lr)`	`// Set learning rate (for scheduler)`

Optimizer_t: The Generic Optimizer Wrapper

`typedef struct optimizer {`
`optimizerType_t type; // SGD_M or ADAM_W`
`optimImpl_t *impl; // Union: sgd_t* or adamW_t*`
`parameter_t **parameter; // Array of weight+grad pairs`
`states_t **states; // Momentum buffers (NULL if momentum=0)`
`size_t sizeStates; // Number of parameter sets`
`roundingMode_t writeBackRounding; // SR_HALF_AWAY default`
`} optimizer_t;`

12.2 Interactions with Other Files and STM32

Sgd.c depends on: ArithmeticType.h, Common.h, ExecuteOp.h, Sgd.h, Tensor.h. Called by: userApi/optimizer/SgdApi.c, userApi/training_loop/TrainingBatchDefault.c.

CONCEPT The writeBackRounding field on optimizer_t is what enables FQT dead-zone escape. When SGD writes updated weights back to a SYM_INT32 tensor, the write-back rounds using SR_HALF_AWAY (the default). This means tiny gradient updates that would round to zero in standard rounding instead survive with correct probability.

12.3 Line-by-Line Explanation

sgdUpdateKernel() -- Plain SGD

```
static void sgdUpdateKernel(tensor_t **op, ..., const void *ctxv) {
const sgdUpdateCtx_t *ctx = ctxv;
float *param = (float *)op[0]->data;
float *grad = (float *)op[1]->data;
float *out = (float *)rawOut->data;
for (size_t i = 0; i < n; i++) {
float g = grad[i] + ctx->weightDecay * param[i];          // L2 regularization
out[i] = param[i] - ctx->lr * g;                          // SGD update: param -= lr * (grad + wd*param)
}
}
// Example: param=0.1, grad=0.05, lr=0.01, wd=0.0001
// g = 0.05 + 0.0001*0.1 = 0.05001
// new_param = 0.1 - 0.01*0.05001 = 0.09950
```

sgdStepM() -- Dispatches Plain vs Momentum

```
void sgdStepM(optimizer_t *optim) {
if (sgd->momentumFactor == 0.0f) {                        // Plain SGD path
for each parameter p:
executeOp({kernel=sgdUpdateKernel, ...}, p->param);       // Write back with SR
return;
}
// Momentum path: two executeOp calls per parameter
for each parameter p:
// Step 1: state = momentum*state + grad + wd*param
executeOp({kernel=sgdMStateKernel, ...}, state);
// Step 2: param = param - lr*state
executeOp({kernel=sgdMParamKernel, ...}, p->param);
}
```

WARNING Only ARITH_FLOAT32 update arithmetic is implemented (checked in both sgdInit() and sgdStepM()). Integer-arithmetic SGD updates are not yet designed (#310). The update always runs in float32; the write-back to quantized storage happens in the executeOp epilogue.

12.4 Missing RigL Code -- Mask-Aware SGD

The most important Sgd.c change: skip inactive weights (mask=0) during the parameter update. Currently sgdUpdateKernel() updates ALL weights unconditionally. With a mask, inactive weights must stay at exactly zero.

RigL: Mask-Aware SGD	Add weightMask check in sgdUpdateKernel. Inactive weights: output[i]=0.0f; gradient[i]=0.0f.

```
// Modified sgdUpdateKernel with mask support:
static void sgdUpdateKernelMasked(tensor_t **op, size_t n,
tensor_t *rawOut, tensor_t *aux, const void *ctxv) {
const sgdUpdateCtx_t *ctx = ctxv;
tensor_t *mask = ctx->weightMask; // NULL for dense layers
float *param = (float *)op[0]->data;
float *grad = (float *)op[1]->data;
float *out = (float *)rawOut->data;
size_t nElem = calcNumberOfElementsByTensor(rawOut);
for (size_t i = 0; i < nElem; i++) {
if (mask != NULL && !tensorBoolGet(mask, i)) {   // Check mask
out[i] = 0.0f;                                   // Inactive: force to zero
grad[i] = 0.0f;                                  // Clear gradient too
continue;                                        // Skip update
}
float g = grad[i] + ctx->weightDecay * param[i];
out[i] = param[i] - ctx->lr * g;                 // Normal SGD update
}
}
```

Chapter 13

AdamW.h / AdamW.c

Adam with Weight Decay: Adaptive Moment Estimation Optimizer

13.1 Usage of This File

AdamW.c implements the Adam optimizer with decoupled weight decay (AdamW), as introduced by Loshchilov & Hutter (ICLR 2019). AdamW is the preferred optimizer for neural network training because its adaptive learning rates allow faster convergence than SGD and better generalization than Adam. For HAR training, AdamW with lr=1e-3, beta1=0.9, beta2=0.999, eps=1e-8, and weight_decay=0.01 is a strong baseline.

AdamW maintains two moment buffers per weight: the first moment m (exponential moving average of gradients) and the second moment v (EMA of squared gradients). These buffers have the same shape as the weight tensor, so AdamW requires 3x more memory than SGD (weights + m + v). For a 64x1152 layer: 73,728 weights + 73,728 m values + 73,728 v values = 221,184 floats = 864 KB -- too large for STM32 SRAM without sparsity.

```
typedef struct adamWCtx {
float lr; // Learning rate (e.g., 1e-3)
float beta1; // First moment decay (default 0.9)
float beta2; // Second moment decay (default 0.999)
float eps; // Numerical stability (default 1e-8)
float weightDecay; // Decoupled weight decay (default 0.01)
size_t t; // Step counter (for bias correction)
roundingMode_t writeBackRounding; // SR_HALF_AWAY for FQT
} adamWCtx_t;

void adamWUpdateKernel(float *param, float *grad,
float *m, float *v, adamWCtx_t *ctx, size_t n);  // Update n weights
```

13.2 Interactions with Other Files and STM32

AdamW.c depends on Axpby.h (lerp_ operations for moment updates), Mul.h (element-wise multiplication for v update), DTypes.h, and Rounding.h (for SR write-back in FQT mode). It is called by the training loop after each backward pass.

Memory constraint on STM32: a single AdamW first moment buffer for a 1152->64 layer is 73,728 floats = 288 KB -- nearly the entire SRAM. With 90% RigL sparsity: only 7,373 active weights, meaning active m and v buffers need only 7,373 floats each = 29 KB each. However, in the current ODT implementation, m and v are always the full size -- sparse storage of moment buffers is a TODO for memory-constrained deployment.

CONCEPT	AdamW vs Adam: the key difference is WHERE weight decay is applied. Adam applies weight decay as L2 regularization to the gradient (grad += weight_decay * param), which is then adapted by the moment estimates. AdamW applies weight decay directly to the weight (param -= lr * weight_decay * param) AFTER the Adam update. This decoupling avoids the adaptive scaling interfering with regularization.

13.3 Line-by-Line Explanation

Adam Update Step -- Full Formula

The Adam update for step t: (1) m = beta1*m + (1-beta1)*g, (2) v = beta2*v + (1-beta2)*g^2, (3) m_hat = m/(1-beta1^t), (4) v_hat = v/(1-beta2^t), (5) param -= lr * m_hat / (sqrt(v_hat) + eps). AdamW adds step 6: param -= lr * weight_decay * param (before the Adam update).

```
void adamWUpdateKernel(float *param, float *grad,
float *m, float *v, adamWCtx_t *ctx, size_t n) {
ctx->t++;                                          // Increment step counter
float bc1 = 1.0f - powf(ctx->beta1, (float)ctx->t);  // Bias correction 1
float bc2 = 1.0f - powf(ctx->beta2, (float)ctx->t);  // Bias correction 2
float lrCorrected = ctx->lr * sqrtf(bc2) / bc1;    // Effective lr
for (size_t i = 0; i < n; i++) {
float g = grad[i];                                 // Current gradient
// Update first moment: m = beta1*m + (1-beta1)*g
m[i] = ctx->beta1 * m[i] + (1.0f - ctx->beta1) * g;  // lerp
// Update second moment: v = beta2*v + (1-beta2)*g^2
v[i] = ctx->beta2 * v[i] + (1.0f - ctx->beta2) * g * g;
// Adam update:
float update = lrCorrected * m[i] / (sqrtf(v[i]) + ctx->eps);
// AdamW: separate weight decay (not L2 in grad)
param[i] -= update + ctx->lr * ctx->weightDecay * param[i];
// Zero gradient after use:
grad[i] = 0.0f;
}
}
```

lerp_ Idiom -- Moment Update via Axpby

The moment updates m = beta1*m + (1-beta1)*g and v = beta2*v + (1-beta2)*g^2 are both instances of the lerp (linear interpolation) operation. PyTorch calls this lerp_(): y = (1-t)*y + t*x. In ODT, axpby(1-beta1, grad, beta1, m, m) implements this via the BLAS axpby primitive.

```
// Using axpby for moment update:
axpbyInplace(1.0f-ctx->beta1, gradTensor, ctx->beta1,
mTensor);
// mTensor = (1-beta1)*grad + beta1*m
```

```
// Equivalent to: mTensor = lerp(mTensor, gradTensor,
1-beta1)
// For v update: v = beta2*v + (1-beta2)*g^2
// Need g^2 first: mulTensors(gradTensor, gradTensor,
gradSquared)
axpbyInplace(1.0f-ctx->beta2, gradSquared, ctx->beta2,
vTensor);
```

13.4 Missing RigL Code -- Mask-Aware AdamW

AdamW needs two RigL additions: (1) skip the update for inactive weights (mask=0), and (2) zero the moment buffers m and v for newly-deactivated weights after DROP. If m and v retain non-zero values for inactive weights, the GROW step may activate a weight with incorrect moment history, causing a bad parameter update in the first step after grow.

RigL: AdamW + RigL	Add mask check in adamWUpdateKernel: if (!tensorBoolGet(mask, i)) { m[i]=0; v[i]=0; param[i]=0; grad[i]=0; continue; }. Zeroing m and v for inactive positions prevents stale moment accumulation.

```
// Mask-aware AdamW update:
for (size_t i = 0; i < n; i++) {
if (mask != NULL && !tensorBoolGet(mask, i)) {
// Inactive weight: zero everything
param[i] = 0.0f;
grad[i] = 0.0f;
m[i] = 0.0f;                    // Clear moment history
v[i] = 0.0f;                    // Clear velocity history
continue;                       // Skip Adam update
}
// Active weight: normal Adam update
float g = grad[i];
m[i] = beta1*m[i] + (1-beta1)*g;
v[i] = beta2*v[i] + (1-beta2)*g*g;
param[i] -= lrCorr * m[i] / (sqrtf(v[i]) + eps);
param[i] -= lr * weightDecay * param[i];
grad[i] = 0.0f;
}
```

TIP	Memory note: for HAR with 90% sparsity on the 1152->64 layer, active AdamW moment storage = 7,373 active weights * 3 tensors (param, m, v) * 4 bytes = 88 KB. Adding the mask (9 KB) = 97 KB total for this layer. With the 64->6 dense layer (384 weights, 5 KB) = 102 KB total, well within 320 KB SRAM.

Chapter 14

LrScheduler.h / LrScheduler.c

Learning Rate Schedules: Step, Exponential, Cosine Annealing

14.1 Usage of This File

LrScheduler.c implements three learning rate schedules: StepLR (multiply by gamma every N epochs), ExponentialLR (multiply by gamma every epoch), and CosineAnnealingLR (cosine decay to etaMin over T_max epochs). The scheduler calls optimizerFunctions[].setLr() to update the optimizer learning rate each time lrSchedulerStep() is called.

```
typedef enum { STEP_LR, EXPONENTIAL_LR, COSINE_ANNEALING_LR } lrSchedulerType_t;  // Schedule type
typedef struct lrScheduler {
lrSchedulerType_t type;  // Which schedule
float baseLr; // Initial learning rate
int lastEpoch; // Current epoch counter
union { stepParams; expParams; cosineParams; } params;
optimizer_t *optimizer; // The optimizer to update
} lrScheduler_t;

void stepLrInit(sched, optimizer, baseLr, stepSize, gamma)  // Step decay
void exponentialLrInit(sched, optimizer, baseLr, gamma)  // Exponential decay
void cosineAnnealingLrInit(sched, optimizer, baseLr, tMax, etaMin)  // Cosine decay
void lrSchedulerStep(lrScheduler_t *sched)  // Advance by one epoch
float lrSchedulerGetLr(lrScheduler_t *sched)  // Query current LR
```

C Usage Example

```
lrScheduler_t sched;
cosineAnnealingLrInit(&sched, &optim, 0.01f, 50, 1e-5f);  // baseLr=0.01, T=50

for (int epoch = 0; epoch < 50; epoch++) {
trainOneEpoch(&optim, &loader);  // Forward/backward/step
lrSchedulerStep(&sched);  // Advance LR schedule
printf("epoch %d lr=%.6f\n", epoch, lrSchedulerGetLr(&sched));
}
// epoch 0: lr = 0.01000 (start high)
// epoch 25: lr = 0.00501 (midpoint)
// epoch 49: lr = 0.00001 (etaMin reached)
```

14.2 Interactions and Math

All three schedules share the same interface: init sets baseLr and hyperparameters, step() increments lastEpoch and calls setLr(). The math for each schedule differs only in the lr computation function.

StepLR: Epoch-Milestone Decay

```
// StepLR: lr = baseLr * gamma^floor(epoch / stepSize)          // Integer division
// Hyperparameters: stepSize=10, gamma=0.5, baseLr=0.01
// epoch 0: floor(0/10)=0, lr = 0.01 * 0.5^0 = 0.0100
// epoch 10: floor(10/10)=1, lr = 0.01 * 0.5^1 = 0.0050
// epoch 20: floor(20/10)=2, lr = 0.01 * 0.5^2 = 0.0025
// epoch 30: floor(30/10)=3, lr = 0.01 * 0.5^3 = 0.0013
static float stepLrComputeLr(const lrScheduler_t *sched) {
int steps = sched->lastEpoch / sched->params.stepLr.stepSize;
return sched->baseLr * powf(sched->params.stepLr.gamma,
(float)steps);
}
```

CONCEPT StepLR is the simplest schedule: the LR is held constant for stepSize epochs, then dropped by gamma. This creates a "staircase" LR curve. It is easy to reason about but produces sharp LR drops that can destabilize training if gamma is too small. Common choices: gamma=0.1 (aggressive) or gamma=0.5 (moderate).

ExponentialLR: Smooth Geometric Decay

```
// ExponentialLR: lr = baseLr * gamma^epoch                     // Every epoch
// Hyperparameters: gamma=0.95, baseLr=0.01
// epoch 0: lr = 0.01 * 0.95^0 = 0.010000
// epoch 10: lr = 0.01 * 0.95^10 = 0.005987
// epoch 20: lr = 0.01 * 0.95^20 = 0.003585
// epoch 50: lr = 0.01 * 0.95^50 = 0.000769
static float exponentialLrComputeLr(const lrScheduler_t
*sched) {
return sched->baseLr * powf(sched->params.expLr.gamma,
(float)sched->lastEpoch);
}
```

CONCEPT ExponentialLR decays smoothly every epoch rather than in steps. For training on HAR (short epochs over 6617 samples, batch 32 = 207 iterations/epoch), exponential decay with gamma=0.95 over 50 epochs reduces LR to ~7.7% of start. This is gentler than StepLR(stepSize=10, gamma=0.1) which reduces to 0.01% over 50 epochs.

CosineAnnealingLR: Smooth Cosine Decay

```
// CosineAnnealing: lr = etaMin + (baseLr - etaMin) * (1 +
cos(pi*t/T)) / 2
```

```
// Hyperparameters: baseLr=0.01, T_max=50, etaMin=1e-5
// t=0: lr = 1e-5 + (0.01-1e-5)*(1+cos(0))/2 = 0.01000
// t=25: lr = 1e-5 + (0.01-1e-5)*(1+cos(pi/2))/2 = 0.00501
// t=50: lr = 1e-5 + (0.01-1e-5)*(1+cos(pi))/2 = 0.00001
static float cosineAnnealingLrComputeLr(const lrScheduler_t *sched) {
double etaMin = (double)sched->params.cosineAnnealingLr.etaMin;
return (float)(etaMin + ((double)sched->baseLr - etaMin) *
(1.0 + cos(M_PI * (double)sched->lastEpoch /
(double)sched->params.cosineAnnealingLr.tMax)) / 2.0);
}                                                    // double intermediates match PyTorch numerics exactly
```

WARNING Note that double is used for intermediate computations in cosineAnnealingLrComputeLr(). The reason: for small etaMin (e.g. 1e-5) and large baseLr (e.g. 0.01), the subtraction (baseLr - etaMin) loses precision in float32 when baseLr >> etaMin. Using double preserves the full range. The final result is cast back to float.

14.3 Line-by-Line Explanation

lrSchedulerStep() -- One Epoch Advance

lrSchedulerStep() increments the epoch counter, recomputes the learning rate using the schedule-specific function, and calls setLr() on the optimizer. The optimizer's setLr updates the lr field used in the next sgdStepM() or adamWStepM() call.

```
void lrSchedulerStep(lrScheduler_t *sched) {
sched->lastEpoch++;                                  // Advance epoch count
float newLr;
switch (sched->type) {
case STEP_LR: newLr = stepLrComputeLr(sched); break;
case EXPONENTIAL_LR: newLr = exponentialLrComputeLr(sched); break;
case COSINE_ANNEALING_LR: newLr = cosineAnnealingLrComputeLr(sched); break;
default: PRINT_ERROR("unknown schedule"); return;
}
optimizerFunctions[sched->optimizer->type].setLr(sched->optimizer, newLr);  // Update optimizer
}
// Note: lastEpoch starts at 0. After first call, lastEpoch=1.
// StepLR with stepSize=10: first decay happens at lastEpoch=10.
```

lrSchedulerInit() Common Setup

```
// Common to all three init functions:
void _lrSchedulerBaseInit(lrScheduler_t *sched, optimizer_t *optim, float baseLr) {
sched->optimizer = optim;                    // Link to optimizer
sched->baseLr = baseLr;                      // Store initial LR
sched->lastEpoch = 0;                        // Epoch counter starts at 0
// Set optimizer to baseLr immediately:
optimizerFunctions[optim->type].setLr(optim, baseLr);
}
```

14.4 Missing RigL Code

LrScheduler.c is complete and does not need RigL changes. The cosine schedule can be used as-is alongside the RigL alpha(t) cosine decay. Both decays share the same cosine math but apply independently: one controls learning rate, the other controls mask update frequency.

```
// Recommended setup for RigL training:
cosineAnnealingLrInit(&sched, &optimizer, 0.01f, totalEpochs, 1e-5f);  // LR cosine decay
// RigL alpha cosines: alpha(step) = (0.3/2)*(1+cos(pi*step/T_end))    // alpha_init=0.3
// Both decrease together -- natural synergy:
// Early training: high LR (explore) + high alpha (aggressive mask swap)
// Late training: low LR (refine) + low alpha (stable mask)
```

RigL: LR + Alpha Synergy Run CosineAnnealingLR and RigL with the same T_max. The high LR early in training is compatible with aggressive mask swapping (high alpha): both encourage exploration. As training progresses, both decrease, allowing the sparse network to converge on a good solution with a stable mask.

Concrete RigL Training Schedule

```
// Example: 50 epochs, 207 iter/epoch = 10350 total steps
// RigL runs every 100 steps: 103 mask updates total
// T_end for alpha cosine = 80% of total steps = 8280
// step=0: alpha = 0.3*(1+cos(0))/2 = 0.300          // Full swapping
// step=4140: alpha = 0.3*(1+cos(pi/2))/2 = 0.150    // Half-way
// step=8280: alpha = 0.3*(1+cos(pi))/2 = 0.000      // No more swapping
// step>8280: mask frozen, only weight magnitudes change
// LR at epoch 50 (step 10350): lr = etaMin = 1e-5   // Fine-tuning
```

Chapter 15

CrossEntropy.h / CrossEntropy.c + LossFunction.h

Cross-Entropy Loss and Softmax-Backward for Classification

15.1 Usage of This File

CrossEntropy.c implements the cross-entropy loss function for multi-class classification. It expects the input to be a softmax probability distribution (not raw logits). The backward pass computes the combined gradient of cross-entropy through softmax, which simplifies to (predictions - targets) -- one of the most elegant results in deep learning.

```
float crossEntropyForward(tensor_t *softmaxOutput, tensor_t *target,
reduction_t reduction);                                    // Returns scalar loss
void crossEntropyBackward(tensor_t *softmaxOutput, tensor_t *target,
tensor_t *grad);                                           // Gradient wrt softmax input
// reduction_t: REDUCTION_MEAN (default) or REDUCTION_SUM
// LossFunction.h provides lossFunction_t struct for model integration
```

HAR: 6-Class Classification

```
// HAR activity classes (6 total):
// 0: WALKING 1: WALKING_UPSTAIRS 2: WALKING_DOWNSTAIRS
// 3: SITTING 4: STANDING 5: LAYING
// Cross-entropy example for a correct WALKING prediction:
// softmax output: p = [0.80, 0.10, 0.05, 0.02, 0.02, 0.01]
// one-hot target: y = [1.0, 0.0, 0.0, 0.0, 0.0, 0.0]
// loss = -log(0.80) = 0.223 (low: confident + correct)

// Incorrect prediction example:
// softmax output: p = [0.10, 0.70, 0.10, 0.05, 0.03, 0.02]
// one-hot target: y = [1.0, 0.0, 0.0, 0.0, 0.0, 0.0]
// loss = -log(0.10) = 2.303 (high: confident + wrong)
```

15.2 The Beautiful Backward Formula

The softmax-cross-entropy gradient has a strikingly simple form: grad = p - y. This is derived by applying the chain rule through the softmax and log operations simultaneously.

```
// Forward: L = -sum_i(y_i * log(p_i)) where p = softmax(z)
// softmax: p_i = exp(z_i) / sum_j(exp(z_j))

// Gradient dL/dz_i (derivation):
// dL/dp_k = -y_k / p_k (from log)
// dp_k/dz_i = p_k*(delta_ik - p_i) (from softmax Jacobian)
// dL/dz_i = sum_k(dL/dp_k * dp_k/dz_i)                    // Chain rule
// = sum_k(-y_k/p_k * p_k*(delta_ik - p_i))                // Substitute
// = sum_k(-y_k*(delta_ik - p_i))                          // Simplify
// = -y_i + p_i * sum_k(y_k)                               // Expand
// = p_i - y_i (since sum_k(y_k) = 1 for one-hot)          // Final result

// HAR example (WALKING, correct=0):
// grad = p - y = [0.80-1, 0.10-0, 0.05-0, 0.02-0, 0.02-0, 0.01-0]
// = [-0.20, 0.10, 0.05, 0.02, 0.02, 0.01]
// Interpretation: push z[0] down (-0.20), push all others up
```

CONCEPT The p - y gradient is exact only for softmax + cross-entropy combined. Applying cross-entropy directly to raw logits requires a different gradient derivation. ODT's CrossEntropy.c expects softmax probabilities as input (after a Softmax layer), NOT raw logits. Always apply Softmax before CrossEntropy in the forward pass.

15.3 Interactions and Hardware

CrossEntropy.c depends on: Common.h, math.h (logf()), TensorConversion.h. It is called by the training loop after Softmax.c. For FLOAT32 tensors, raw float pointer access is used. The clamp to [1e-7, 1-1e-7] prevents log(0) = -infinity which would produce NaN gradients.

```
// Dependencies:
// math.h: logf() -- Cortex-M7 computes ln(x) via FPU + library
// Softmax.c must run first to produce valid probability distribution
// TensorConversion.h: for SYM_INT32 input, dequantize before log

// Integration with model:
lossFunction_t loss;
lossFunctionInit(&loss, CROSS_ENTROPY, REDUCTION_MEAN);
float L = loss.forward(&loss, softmaxOut, target);      // Forward: scalar
loss.backward(&loss, softmaxOut, target, grad);         // Backward: gradient
```

WARNING	crossEntropyForward() detects NaN/Inf softmax output and calls abort(). On MCU, abort() triggers a hard fault. Ensure the softmax output is valid before calling cross-entropy. A numerical stability check is applied: pi = fmaxf(p_data[i], 1e-7f) before each logf() call. This clamp prevents log(0) but does NOT prevent an invalid probability distribution (sum != 1.0). Always verify softmax is applied correctly.

15.4 Line-by-Line Explanation

crossEntropyForwardFloat() -- Compute Loss

`float crossEntropyForwardFloat(tensor_t *p, tensor_t *y, reduction_t r) {`	
`size_t n = calcNumberOfElementsByTensor(p);`	`// Total elements`
`size_t microbatch = getDimensionsByIndex(p, 0);`	`// Batch size`
`float *p_data = (float*)p->data;`	`// Prob distribution`
`float *y_data = (float*)y->data;`	`// One-hot targets`
`float loss = 0.0f;`	`// Accumulator`
`for (size_t i = 0; i < n; i++) {`	`// Over all batch*classes`
`float pi = fmaxf(p_data[i], 1e-7f);`	`// Clamp to avoid log(0)`
`loss += y_data[i] * -logf(pi);`	`// Cross-entropy term`
`}`	`// Only non-zero y_i contribute`
`if (r == REDUCTION_MEAN) loss /= (float)microbatch;`	`// Normalize by batch`
`return loss;`	`// Scalar loss value`
`}`	
`// For one-hot targets, only the true class index contributes to the sum.`	
`// loss = -log(p[true_class]) when y is one-hot.`	

crossEntropyBackwardFloat() -- Gradient p - y

`void crossEntropyBackwardFloat(tensor_t *p, tensor_t *y,`	
`tensor_t *grad, reduction_t r) {`	
`size_t n = calcNumberOfElementsByTensor(p);`	
`size_t microbatch = getDimensionsByIndex(p, 0);`	
`float scale = (r == REDUCTION_MEAN) ? 1.0f/(float)microbatch : 1.0f;`	
`float *p_data = (float*)p->data;`	
`float *y_data = (float*)y->data;`	
`float *g_data = (float*)grad->data;`	`// Output gradient`
`for (size_t i = 0; i < n; i++) {`	
`g_data[i] = (p_data[i] - y_data[i]) * scale;`	`// grad = (p-y)/batch`
`}`	
`}`	

```
// Example (batch=1, WALKING true,
p=[0.8,0.1,0.05,0.02,0.02,0.01]):
// grad = [-0.20, 0.10, 0.05, 0.02, 0.02, 0.01] (not scaled:
batch=1)
```

15.5 Numerical Stability and HAR Context

The HAR dataset has 6 balanced classes. With a well-initialized model, the softmax output starts near uniform (1/6 ≈ 0.167 per class). The initial cross-entropy is approximately -log(0.167) = 1.79 nats. During training, the loss decreases toward -log(1.0) = 0 as the model becomes confident.

```
// Typical training curves for HAR 6-class:
// epoch 0: loss ≈ 1.79 (uniform random, -log(1/6))
// epoch 10: loss ≈ 0.85 (model learning basic features)
// epoch 30: loss ≈ 0.35 (refining boundaries)
// epoch 50: loss ≈ 0.20 (near convergence)
// Test accuracy at convergence: ~93-95% on UCI HAR

// Common mistake: applying cross-entropy to raw logits
(before softmax)
// Result: gradient = p_from_logits - y (same formula but
applied to wrong layer)
// Effect: correct math but double-counts softmax Jacobian in
backward pass
```

15.6 Missing RigL Code

CrossEntropy.c does not need RigL changes. It operates on softmax output (activations), not on weights. The gradient it produces (p - y) flows backward through linear layers where the mask-aware matmul and SGD will apply the mask correctly. The loss computation itself is identical for dense and sparse networks.

TIP For binary classification (HAR: stationary vs. moving), use BCELoss instead of CrossEntropy. ODT may include BCELoss in a separate file. BCELoss: L = -y*log(p) - (1-y)*log(1-p). Gradient: grad = p - y. The same elegant gradient formula applies.

Chapter 16

DataLoader.h / DataLoader.c + NPYLoader.h / NPYLoader.c

Streaming Dataset Access for Memory-Constrained MCU Deployment

16.1 Usage of This File

DataLoader.h defines the streaming data loading infrastructure. On MCU, datasets cannot fit in RAM, so samples must be streamed from external storage one at a time. The DataLoader provides a getBatch() interface that triggers getSample() calls, applying optional transforms and shuffling via index permutation.

```
struct dataLoader {
getSampleFn_t getSample; // Gets sample by ID from storage
getDatasetSizeFn_t getDatasetSize;
uint16_t batchSize;
getBatchFn_t getBatch; // Orchestrates getSample calls
transformFn_t transform; // Applied once at load time
transformFn_t targetTransform; // Applied to each batch
bool shuffle; uint64_t shuffleSeed;
size_t *indices; // Shuffled index array
bool dropLast; // Drop incomplete last batch
};
```

NPYLoader -- Parsing .npy Files

.npy format: 6-byte magic (■NUMPY), 1-byte major version, 1-byte minor version, 2-byte header length, ASCII header dict, then raw data. NPYLoader.c parses this header to extract shape and dtype, then provides direct access to the raw data.

```
FILE *openNPYFile(char *path); // Opens file, checks magic
void checkMagic(FILE *f); // Verifies ■NUMPY prefix
uint32_t readHeaderSize(FILE *f); // Reads 2-byte header length
void readHeader(char *buf, uint32_t size, FILE *f); // ASCII header dict
dtype_t getDTypeFromHeader(char *header); // Parse dtype string
void getShapeFromHeader(shape_t*, size_t*, size_t*, char*, size_t); // Parse shape
```

16.2 Interactions and Hardware

DataLoader.c depends on: Dataset.h (sample_t, batch_t), Tensor.h. NPYLoader.c depends on standard I/O (FILE*, fread()). On STM32 with FatFS + SD card, FILE* is provided by the FatFS POSIX compatibility layer. On Linux simulation, it is the standard libc FILE*.

```
// HAR dataset streaming:
// train_x.npy: shape (6617, 9, 128), dtype float32, C order
// Each sample: sample 0 starts at byte offset = header_size + 0*9*128*4
// Sample i: starts at byte offset = header_size + i*9*128*4
size_t sampleBytes = 9 * 128 * sizeof(float); // 4608 bytes
fseek(f, headerSize + sampleId * sampleBytes, SEEK_SET);  // Seek to sample
fread(sampleBuffer, 1, sampleBytes, f);  // Read exactly one sample
```

16.3 Line-by-Line Explanation

initDataLoader()

```
void initDataLoader(dataLoader_t *dl, getSampleFn_t getSample,
getDatasetSizeFn_t getDatasetSize, getBatchFn_t getBatch,
uint16_t batchSize, transformFn_t transform,
transformFn_t targetTransform, bool shuffle,
uint64_t shuffleSeed, size_t *indices, bool dropLast) {
dl->getSample = getSample;  // Function pointer assignment
dl->getDatasetSize = getDatasetSize;
dl->batchSize = batchSize;
dl->getBatch = getBatch;
dl->transform = transform;
dl->targetTransform = targetTransform;
dl->shuffle = shuffle;
dl->shuffleSeed = shuffleSeed;
dl->indices = indices;  // Caller-owned index array
dl->dropLast = dropLast;
}
```

16.4 Missing RigL Code

DataLoader and NPYLoader do not need RigL changes. They provide training data; RigL operates on the model parameters. The streaming architecture is actually ideal for RigL training: since only one sample is in memory at a time, there is more SRAM headroom for the mask tensors.

```
// Memory budget with RigL on 320 KB MCU:
// One HAR sample: 9*128*4 = 4,608 bytes
// 3-layer model weights (sparse 90%): ~3,000 bytes
```

// 3-layer masks (BOOL): ~375 bytes	
// Activations (batch=1): ~2,048 bytes	
// SGD state (momentum): ~3,000 bytes	
// Total: ~13 KB << 320 KB (fits comfortably!)	

Chapter 17

Serialize.h / Serialize.c

Model Checkpoint Serialization: Locked Binary Format v2

17.1 Usage of This File

Serialize.c saves and loads trained model parameters to/from binary files. The format is locked (append-only) to ensure backward compatibility with previously saved models. It writes: 4-byte magic "ODTS", uint32 version (currently 2), uint32 layer count, then one record per layer containing a type tag byte followed by layer-specific payload.

```
void serializeModel(layer_t **model, size_t n, FILE *f)          // Save full model
void deserializeModel(layer_t **model, size_t n, FILE *f)        // Load full model
void serializeTensor(tensor_t *t, FILE *f)                       // Save one tensor
void deserializeTensor(tensor_t *t, FILE *f)                     // Load one tensor
void serializeParameter(parameter_t *p, FILE *f)                 // Save param + grad
void deserializeParameter(parameter_t *p, FILE *f)               // Load param + grad

// Format: "ODTS" | u32 version | u32 layerCount | [u8 tag + payload]*
// Tag byte = layerType_t enum position (append-only, never change!)
// Tag 0=LINEAR, 1=CONV1D, 2=SOFTMAX, 3=LAYERNORM, ...
```

ODTS v2 Binary Format Diagram

```
Offset Size Field
0 4 Magic "ODTS" (0x4F 0x44 0x54 0x53)          // File identifier
4 4 Version = 2 (u32 little-endian)             // Format version
8 4 LayerCount (u32 little-endian)              // Number of layers
12 1 Tag[0] (u8 = layerType_t)                  // Layer 0 type
13 var Payload[0] (layer-specific)              // Layer 0 data
... ... Tag[1] + Payload[1]                     // Layer 1
... ... (repeats for all layers)

// LINEAR payload: serializeParameter(weights) + serializeParameter(bias)
// Each parameter: serializeTensor(param) + serializeTensor(grad)
// Each tensor: serializeShape + serializeQuantization + data bytes + sparsity
```

17.2 serializeTensor() -- The Core Write

```
void serializeTensor(tensor_t *t, FILE *f) {
serializeShape(t->shape, f);                    // Write dims + orderOfDimensions
serializeQuantization(t->quantization, f);      // Write qtype + qconfig
size_t dataBytes = calcBytesPerTensor(t);
serialWriteBytes(t->data, dataBytes, f);        // Write raw data payload
serializeSparsity(t->mask, f);                  // Write mask (currently stub!)
}

void serializeShape(shape_t s, FILE *f) {
serialWriteU32LE(s.numberOfDimensions, f);
for (size_t i = 0; i < s.numberOfDimensions; i++)
serialWriteU32LE(s.dimensions[i], f);           // Dimension sizes
for (size_t i = 0; i < s.numberOfDimensions; i++)
serialWriteU32LE(s.orderOfDimensions[i], f);    // Stride order
}
```

WARNING serializeSparsity() is currently an EMPTY STUB. It takes no arguments and writes nothing. This means RigL mask state is NOT saved to checkpoints. After loading a saved model, all masks are reset to all-ones (dense), losing the sparse structure trained over many epochs. Fix provided in Section 17.5.

17.3 Interactions and Hardware

Serialize.c depends on: SerialWire.h (checked primitive writes), all layer headers (Linear.h, Conv1d.h, etc.), Tensor.h, DTypes.h. On STM32 with SD card + FatFS, the FILE* is a FatFS file handle. All writes use serialWriteU32LE() which ensures little-endian byte ordering regardless of host endianness.

The STM32F746ZG is little-endian (ARM Cortex-M7). Writing u32 values in little-endian order means the serialized file can be directly memory-mapped on the MCU (future optimization). On a host PC (x86, also little-endian), the same serialization code works identically, enabling train-on-host, deploy-on-MCU workflows.

```
// serialWriteU32LE: always little-endian, portable
void serialWriteU32LE(uint32_t v, FILE *f) {
uint8_t buf[4];
buf[0] = (uint8_t)(v & 0xFF);                   // LSB first
buf[1] = (uint8_t)((v >> 8) & 0xFF);
buf[2] = (uint8_t)((v >> 16) & 0xFF);
buf[3] = (uint8_t)((v >> 24) & 0xFF);           // MSB last
fwrite(buf, 1, 4, f);                           // 4 bytes to file
}
// Example: serialWriteU32LE(2, fp) writes: 02 00 00 00
// Never: fwrite(&v, 4, 1, fp) -- UB on big-endian hosts
```

17.4 Line-by-Line: serializeModel()

```
void serializeModel(layer_t **model, size_t n, FILE *f) {
serialWriteBytes("ODTS", 4, f);                    // Magic: identifies ODT format
serialWriteU32LE(SERIALIZE_FORMAT_VERSION, f);     // Version = 2
serialWriteSizeAsU32LE(n, f);                      // Layer count
for (size_t i = 0; i < n; i++) {                   // For each layer
uint8_t tag = (uint8_t)model[i]->type;             // Type enum value
serialWriteU8(tag, f);                             // 1-byte tag (never changes!)
serializeLayer(model[i], f);                       // Layer-specific payload
}
}

// serializeLayer dispatches on model[i]->type:
// LINEAR: serializeLinear() -> weights param + bias param
// CONV1D: serializeConv1d() -> kernel param + bias param
// SOFTMAX: no parameters, writes nothing
// LAYERNORM: serializeLayerNorm() -> gamma + beta params
```

deserializeModel() -- Load Checkpoint

```
void deserializeModel(layer_t **model, size_t n, FILE *f) {
char magic[4];
fread(magic, 1, 4, f);                             // Read 4 bytes
if (memcmp(magic, "ODTS", 4) != 0) {               // Verify magic
PRINT_ERROR("not an ODTS file"); abort();
}
uint32_t ver = serialReadU32LE(f);                 // Read version
if (ver != SERIALIZE_FORMAT_VERSION) {             // Check version
PRINT_ERROR("wrong version: %d", ver); abort();
}
uint32_t nLayers = serialReadU32LE(f);             // Read layer count
ODT_ASSERT(nLayers == n, "layer count mismatch");  // Must match model
for (size_t i = 0; i < n; i++) {
uint8_t tag = serialReadU8(f);                     // Read 1-byte tag
ODT_ASSERT(tag == model[i]->type, "layer type mismatch");
deserializeLayer(model[i], f);                     // Fill weights
}
}
```

CONCEPT	The deserialize checks are defensive: if the file was saved by a different model architecture (different layer count or layer types), the assertions fire immediately rather than silently loading wrong weights into wrong layers. This prevents subtle numerical errors from a mismatched checkpoint.

17.5 Missing RigL Code -- serializeSparsity() Fix

The current serializeSparsity() stub is the most critical gap for RigL deployment. Without it, the sparse mask cannot be saved or loaded. After 50 epochs of RigL training, the mask encodes which 10% of weights are active -- this structure must survive checkpointing.

RigL: serialize Sparsity Fix	Write 1-byte presence flag, then (n+7)/8 bytes of bit-packed mask data. On load, read presence flag and reconstruct mask tensor.

```
// FIXED serializeSparsity():
void serializeSparsity(tensor_t *mask, FILE *fp) {                  // Takes mask as argument
if (mask == NULL || mask->quantization->type != BOOL) {             // Check for valid mask
uint8_t noMask = 0;
fwrite(&noMask, 1, 1, fp);                                          // Write 0 = no mask present
return;
}
uint8_t hasMask = 1;
fwrite(&hasMask, 1, 1, fp);                                         // Write 1 = mask present
size_t n = calcNumberOfElementsByTensor(mask);                      // Number of weight elements
size_t bytes = (n + 7) / 8;                                         // Bit-packed (ceiling div)
fwrite(mask->data, 1, bytes, fp);                                   // Write packed bit array
}

// FIXED deserializeSparsity():
tensor_t* deserializeSparsity(size_t n, FILE *fp) {
uint8_t hasMask = 0;
fread(&hasMask, 1, 1, fp);                                          // Read presence flag
if (!hasMask) return NULL;                                          // Dense layer: no mask
tensor_t *mask = allocBoolTensor(n);                                // Allocate bit-packed mask
size_t bytes = (n + 7) / 8;
fread(mask->data, 1, bytes, fp);                                    // Read packed bit array
return mask;                                                        // Caller attaches to cfg->weightMask
}
```

For a LINEAR layer with 64 outputs and 1152 inputs (the HAR MFCC feature size), the weight tensor has 64*1152 = 73728 elements. At 90% sparsity, 7373 are active. The bit-packed mask takes (73728+7)/8 = 9216 bytes = 9 KB. This is tiny compared to the 294 KB weight tensor. Saving the mask costs only 9 KB extra per checkpoint.

```
// Integration: modify serializeTensor() to pass mask:
```

`void serializeTensor(tensor_t *t, tensor_t *mask, FILE *f) {`	`// Add mask parameter`
`serializeShape(t->shape, f);`	
`serializeQuantization(t->quantization, f);`	
`serialWriteBytes(t->data, calcBytesPerTensor(t), f);`	`// Weight data`
`serializeSparsity(mask, f);`	`// Now saves the mask!`
`}`	
`// Call as: serializeTensor(weights, cfg->weightMask, f);`	`// From serializeLinear()`

Chapter 18

PpcaReplay.h / PpcaReplay.c

Probabilistic PCA Replay for Continual Learning

18.1 Usage of This File

PpcaReplay.c implements memory-efficient continual learning for the STM32. Instead of storing raw training exemplars (hundreds of KB), it fits a Probabilistic PCA model to each class distribution and replays synthetic samples drawn from the learned Gaussian. This prevents catastrophic forgetting when the device retrains on new data without access to all historical data.

The PPCA generative model is: x = W*z + mu + epsilon, where W is the DxK principal components matrix, z ~ N(0,I_K) is a latent code, mu is the class mean, and epsilon ~ N(0, sigma^2 * I_D) is isotropic noise. Only W, mu, and sigma^2 are stored -- much smaller than raw samples.

```
typedef struct ppcaReplay {
tensor_t *W; // Principal components [D, K]
tensor_t *mu; // Class mean [D]
float sigma2; // Noise variance (scalar)
size_t D; // Data dimension
size_t K; // Number of components
rng32_t rng; // Per-model private RNG context
} ppcaReplay_t;
```

```
void ppcaReplayInit(ppcaReplay_t *r, size_t D, size_t K);  // Allocate W, mu
void ppcaFit(ppcaReplay_t *r, tensor_t *data);  // Fit to N data samples
void ppcaSample(ppcaReplay_t *r, tensor_t *out);  // Draw one synthetic sample
```

18.2 Interactions with Other Files and STM32

PpcaReplay.c depends on JacobiEig.c (eigenvalue decomposition), Tensor.h, RNG.h, and standard math (sqrtf). It is called by the training loop after completing a task to fit the replay model, and during replay to generate synthetic training samples.

The PPCA model for one class requires: W (D x K floats) + mu (D floats) + sigma2 (1 float). For HAR with D=1152, K=10: ~50 KB per class, 300 KB for 6 classes -- too large. With K=3: ~18 KB per class, 110 KB total. With K=2: ~12 KB per class, 72 KB total. Practical deployment uses K=2 or K=3 components.

CONCEPT	Memory budget on STM32F746ZG (320 KB SRAM): Sparse model weights ~13 KB, activations ~4 KB, PPCA 6 classes x 12 KB (K=2) = 72 KB, DataLoader buffer 5 KB = 94 KB total. Leaves 226 KB headroom. PPCA replay is feasible with low K.

WARNING	The full DxD covariance matrix for D=1152 requires 1152*1152*4 = 5.3 MB -- 16x larger than MCU SRAM. In practice, apply PPCA on Conv1d output features (dimension 32-64), not raw 1152-D sensor inputs. The feature extractor reduces dimensionality to a tractable size.

18.3 Line-by-Line Explanation

ppcaFit() -- Fitting the Model to Data

ppcaFit() computes the sample mean, centers the data, computes the low-rank covariance (using X^T X instead of the full DxD matrix when N < D), and extracts the top K eigenvectors via JacobiEig. The ML estimate of sigma^2 is the average of the discarded eigenvalues.

```
void ppcaFit(ppcaReplay_t *r, tensor_t *data) {                    // data shape: [N, D]
// 1. Compute mean: mu = (1/N) sum_i data[i]
computeMean(data, r->mu);
// 2. Center: data[i] -= mu
subtractMeanInplace(data, r->mu);
// 3. Low-rank cov: C = (1/N) * data^T * data [D x D]
// If N < D, use NxN Gram matrix instead
// 4. Top K eigenvectors via JacobiEig
jacobiEig(&C, eigenvalues, eigenvectors, K);
// 5. Store W = top K eigenvectors
// 6. sigma2 = mean of remaining eigenvalues
r->sigma2 = meanOfRemainingEigenvalues(eigenvalues, D, K);
}
```

ppcaSample() -- Generating Synthetic Samples

```
void ppcaSample(ppcaReplay_t *r, tensor_t *out) {
// Sample latent: z_k ~ N(0,1), k=0..K-1
float z[K];
for (size_t k = 0; k < r->K; k++)
z[k] = sampleStandardNormal(&r->rng);                              // Box-Muller or ICDF
// Project: out = W*z (DxK matmul with K-vector)
matmulVec(r->W, z, out->data, r->D, r->K);
// Add mean: out += mu
addVecInplace(out->data, r->mu->data, r->D);
// Add noise: out[d] += N(0, sigma2)
float sigma = sqrtf(r->sigma2);
for (size_t d = 0; d < r->D; d++)
out->data[d*4] += sigma * sampleStandardNormal(&r->rng);
}
```

Private RNG Context

ppcaReplay_t maintains its own rng32_t field (separate from the global RNG used for stochastic rounding). This ensures that drawing replay samples does not perturb the SR random stream, keeping training reproducible given the same seed.

```
// During replay sampling:
ppcaSample(&replay[c], &syntheticSample);  // Uses r->rng internally
// Global rngNextFloat() stream is UNAFFECTED
// SR in backward pass stays reproducible
```

18.4 Missing RigL Code

PpcaReplay.c does not interact with the RigL sparsity mask. The replay samples are dense input tensors fed to the forward pass, and the mask-aware matmul in Matmul.c handles them correctly -- no PPCA-specific changes are needed.

TIP Best practice: apply PpcaReplay to Conv1d output features (dimension 32-64) rather than raw 1152-D sensor inputs. This keeps the covariance matrix tractable (e.g., 64x64 = 16 KB) and the W matrix small (64 x K per class).

Chapter 19

Relu.h / Relu.c + Softmax.h / Softmax.c

Activation Functions: ReLU and Softmax

19.1 Usage of This File

Relu.c implements the Rectified Linear Unit activation function: out = max(threshold, x), defaulting to threshold=0. Softmax.c converts a vector of logits into a probability distribution: p_i = exp(x_i) / sum_j(exp(x_j)). Together these are the two activation functions used in standard HAR classifiers.

For a 3-layer HAR model: linear(1152,64) -> ReLU -> linear(64,32) -> ReLU -> linear(32,6) -> Softmax, the ReLU layers provide non-linearity and the Softmax converts the 6 logits to probabilities over the 6 activity classes (walking, running, sitting, standing, lying, going up/down stairs).

```
// Relu.h
void reluForward(layer_t *l, tensor_t *in, tensor_t *out);          // out = max(0, in)
void reluBackward(layer_t *l, tensor_t *fwdIn,
tensor_t *loss, tensor_t *propLoss);                                // propLoss = loss * (fwdIn > 0)
shape_t reluCalcOutputShape(layer_t *l, shape_t in);                // Same shape as input

// Softmax.h
void softmaxForward(layer_t *l, tensor_t *in, tensor_t *out);
shape_t softmaxCalcOutputShape(layer_t *l, shape_t in);
// softmaxBackward = NULL: gradient combined with
CrossEntropy
```

19.2 Interactions with Other Files and STM32

Relu.c depends on Tensor.h, DTypes.h, and ExecuteOp.h. Softmax.c additionally depends on Reduce.c (for the max-shift numerical stability trick). Both are called through the Layer.c vtable. On STM32, the Cortex-M7 FPU handles max() and expf() in hardware -- expf() takes ~20-30 cycles.

CONCEPT	Numerical stability for Softmax: compute exp(x_i - max(x)) instead of exp(x_i). This prevents float32 overflow (expf(88.7) = FLT_MAX). The shift does not change the result since softmax(x) = softmax(x - c) for any constant c: exp(x_i - c)/sum(exp(x_j - c)) = exp(x_i)/sum(exp(x_j)).

WARNING	softmaxBackward is NULL in the layerFunctions[] vtable. The backward pass is instead fused with CrossEntropy: grad = predictions - targets (a beautifully simple formula). If softmax is used without CrossEntropy loss, the backward pass would need to compute the full Jacobian J_ij = p_i*(delta_ij - p_j), which is more complex.

19.3 Line-by-Line Explanation

reluForward() -- Element-Wise max(threshold, x)

```
void reluForward(layer_t *l, tensor_t *in, tensor_t *out) {
size_t n = calcNumberOfElementsByTensor(in);                    // Total elements
reluConfig_t *cfg = l->config->relu;
float threshold = (cfg != NULL) ? cfg->threshold : 0.0f;        // Default: 0
for (size_t i = 0; i < n; i++) {
float v = readBytesAsFloat(&in->data[i * sizeof(float)]);
v = (v > threshold) ? v : threshold;                            // ReLU clamp
writeFloatToByteArray(v, &out->data[i * sizeof(float)]);
}
}
```

reluBackward() -- Gradient Masking

ReLU backward multiplies the incoming loss gradient by the indicator function I(fwdIn > threshold). Where the forward input was below the threshold (dead neuron), the gradient is zero. Where it was positive, the gradient passes through unchanged.

```
void reluBackward(layer_t *l, tensor_t *fwdIn,
tensor_t *loss, tensor_t *propLoss) {
size_t n = calcNumberOfElementsByTensor(fwdIn);
float threshold = getCfgThreshold(l);
for (size_t i = 0; i < n; i++) {
float x = readBytesAsFloat(&fwdIn->data[i*4]);                  // Forward input
float g = readBytesAsFloat(&loss->data[i*4]);                   // Incoming gradient
float pg = (x > threshold) ? g : 0.0f;                          // Mask: dead neurons get 0
writeFloatToByteArray(pg, &propLoss->data[i*4]);
}
}
```

softmaxForward() -- Numerically Stable

```
void softmaxForward(layer_t *l, tensor_t *in, tensor_t *out)
{
size_t n = calcNumberOfElementsByTensor(in);
// Step 1: find max for stability
float maxVal = findMaxFloat(in);
// Step 2: compute shifted exponentials and sum
float sum = 0.0f;
for (size_t i = 0; i < n; i++) {
float v = expf(readBytesAsFloat(&in->data[i*4]) - maxVal);
writeFloatToByteArray(v, &out->data[i*4]);                      // Store exp(x_i - max)
sum += v;
}
```

```
// Step 3: normalize by sum
for (size_t i = 0; i < n; i++) {
float p = readBytesAsFloat(&out->data[i*4]) / sum;
writeFloatToByteArray(p, &out->data[i*4]);  // p_i = softmax(x_i)
}
}
```

Fused CrossEntropy+Softmax Backward

The combined gradient of CrossEntropy(Softmax(x), y) with respect to x is simply: grad_i = p_i - y_i, where p = softmax(x) and y is the one-hot target. This elegant formula replaces the product of two Jacobians (softmax Jacobian and CE gradient) with a single subtraction.

```
// In crossEntropyBackwardFloat():
for (size_t i = 0; i < numClasses; i++) {
float p_i = predictions[i]; // softmax output
float y_i = targets[i]; // one-hot label
grad[i] = p_i - y_i; // combined gradient
}
// For HAR: if true class is 2 (sitting) and pred = [0.1,0.1,0.7,0.05,0.025,0.025]:
// grad = [0.1, 0.1, -0.3, 0.05, 0.025, 0.025] -- pulls class 2 up
```

19.4 Missing RigL Code

Relu.c and Softmax.c do not need RigL changes. They operate on dense activation tensors flowing between layers. Sparsity is applied only to LINEAR layer weight matrices. The mask-aware matmul in Matmul.c ensures sparse forward/backward passes without any changes to activation functions.

TIP Dead ReLU diagnosis: if more than 50% of neurons have threshold > max(fwdIn) consistently, the network has a dying ReLU problem. Solutions: lower learning rate, use Leaky ReLU (threshold with negative slope), or use batch normalization before ReLU. On MCU, monitor activation statistics via PRINT_DEBUG during first epochs.

Chapter 20

Conv1d.h / Conv1d.c + Conv1dKernel.h

1D Convolution: Temporal Feature Extraction for IMU Data

20.1 Usage of This File

Conv1d.c implements 1D convolution for processing temporal sequences such as HAR IMU sensor windows. A Conv1d layer with in_channels=9, out_channels=16, kernel_size=3 processes the 128-sample windows across all 9 IMU channels simultaneously, learning temporal features like step frequency, vibration patterns, and motion transitions.

1D convolution is the recommended front-end for HAR on MCU because it dramatically reduces the feature dimension fed to subsequent linear layers. A conv1d(9,16,3) followed by avgpool(4) reduces the 1152-D raw input to 16*31=496 features, saving 57% of linear layer weight memory and compute.

`typedef struct conv1dConfig {`	
`parameter_t *kernel; // Shape: [out_ch, in_ch, kernel_size]`	
`parameter_t *bias; // Shape: [out_ch]`	
`size_t inChannels, outChannels, kernelSize;`	
`size_t stride, padding;`	
`arithmetic_t arithType;`	`// FLOAT32 or SYM_INT32`
`roundingMode_t roundingMode;`	`// SR_HALF_AWAY for FQT`
`} conv1dConfig_t;`	
`void conv1dForward(layer_t*, tensor_t *in, tensor_t *out);`	`// Sliding window conv`
`void conv1dBackward(layer_t*, tensor_t *fwdIn, tensor_t *loss, tensor_t *propLoss);`	`// Backward`
`shape_t conv1dCalcOutputShape(layer_t*, shape_t in);`	`// L_out = (L_in+2P-K)/S + 1`

20.2 Interactions and Conv1d Math

Conv1d uses Conv1dKernel.c for the actual sliding window computation. For each output position t and output channel c: out[c,t] = sum over (ic, k): kernel[c,ic,k] * in[ic, t*stride + k - padding] + bias[c]. This is equivalent to a matmul if the input is unrolled via the im2col transformation.

Conv1d.c depends on Conv1dKernel.c, Matmul.c (if using im2col path), Tensor.h, DTypes.h, ExecuteOp.h, and Rounding.h. It calls into the same executeOp() infrastructure as Linear.c, supporting FLOAT32 and SYM_INT32 arithmetic.

CONCEPT Output length formula: L_out = floor((L_in + 2*padding - kernel_size) / stride) + 1. For HAR with L_in=128, kernel_size=3, stride=1, padding=0: L_out = (128+0-3)/1 + 1 = 126. With padding=1: L_out = (128+2-3)/1 + 1 = 128 (same-size output).

20.3 Line-by-Line Explanation

conv1dForward() -- Sliding Window

```
void conv1dForward(layer_t *l, tensor_t *in, tensor_t *out) {
conv1dConfig_t *cfg = l->config->conv1d;
// out shape: [batch, out_channels, L_out]
for (size_t b = 0; b < batch; b++) {                    // Batch dimension
for (size_t oc = 0; oc < outCh; oc++) {                 // Output channel
for (size_t t = 0; t < L_out; t++) {                    // Time position
float acc = bias[oc];                                   // Bias initialization
for (size_t ic = 0; ic < inCh; ic++) {                  // Input channel
for (size_t k = 0; k < kernelSize; k++) {               // Kernel position
size_t tIn = t * stride + k - padding;
if (tIn >= L_in) continue;                              // Padding boundary check
acc += kernel[oc,ic,k] * in[b,ic,tIn];                  // MAC
}
}
out[b,oc,t] = acc;
}
}
}
}
```

Backward Pass -- Gradient Computation

conv1dBackward() computes three gradients: dKernel[oc,ic,k] = sum over (b,t): loss[b,oc,t] * in[b,ic,t*stride+k], dBias[oc] = sum over (b,t): loss[b,oc,t], and propLoss[b,ic,tIn] += sum over (oc,k): loss[b,oc,t] * kernel[oc,ic,k]. The propagated loss is the gradient w.r.t. the conv1d input, flowing to the previous layer.

```
// Kernel gradient: dK[oc,ic,k] += loss[t] * in[t*s+k]
// Bias gradient: dB[oc] += sum_t loss[b,oc,t]
// Propagated loss (to previous layer):
// propLoss[ic,tIn] += sum_{oc,k} loss[oc,t] *
kernel[oc,ic,k]
```

20.4 Missing RigL Code -- Conv1d Kernel Sparsity

Conv1d can benefit from RigL sparsity on the kernel weights. The conv1dConfig_t would need a kernelMask field analogous to linearConfig_t.weightMask. The mask shape would be [out_ch, in_ch, kernel_size] (same as kernel). The inner loop would check tensorBoolGet(kernelMask, flatIdx) before each multiply.

RigL: Conv1d RigL Extension	Add: tensor_t *kernelMask field to conv1dConfig_t. Modify: inner loop to check tensorBoolGet(kernelMask, oc*inCh*K + ic*K + k). For a 16x9x3 kernel: 432 weights, 1-bit mask = 54 bytes -- negligible overhead.

```
// Modified kernel accumulation:
```

```
size_t flatIdx = oc * inCh * kernelSize + ic * kernelSize + k;
if (cfg->kernelMask != NULL &&
!tensorBoolGet(cfg->kernelMask, flatIdx)) continue;  // Skip inactive kernel element
acc += kernel[oc,ic,k] * in[b,ic,tIn];  // Only active elements contribute
```

TIP A Conv1d layer with 90% sparsity on its kernels reduces compute by 90% while maintaining the same output shape. For a 9x16x3 kernel, only ~43 of 432 kernel weights are non-zero, computed via 43 MACs instead of 432 per output timestep.

Chapter 21

ArithmeticType.h / ArithmeticType.c

Arithmetic Type Enumeration and Quantization-to-Arithmetic Mapping

21.1 Usage of This File

ArithmeticType.h defines the arithmetic_t struct which specifies how a computation should be performed -- float32 or integer (SYM_INT32) -- along with the rounding mode. The function arithmeticFromQuantizationOrDefault() maps a quantization config to the appropriate arithmetic type automatically, removing the need for callers to manually select the compute path.

This separation between storage type (quantization_t) and compute type (arithmetic_t) is a key design decision in ODT. A tensor stored as SYM_INT32 is always computed with ARITH_SYM_INT32 and SR_HALF_AWAY rounding. A tensor stored as FLOAT32 uses ARITH_FLOAT32. The arithmetic type determines which kernel (matmulFloat32Core vs matmulIntCore) is called.

```
typedef enum arithmeticType {
ARITH_FLOAT32,                  // Standard single-precision float arithmetic
ARITH_SYM_INT32,                // Integer arithmetic with scale factor
} arithmeticType_t;

typedef struct arithmetic {
arithmeticType_t type;          // Which compute kernel to use
roundingMode_t roundingMode;    // HALF_AWAY or SR_HALF_AWAY
} arithmetic_t;

arithmetic_t arithmeticFromQuantizationOrDefault(
quantization_t *q);             // Maps qtype -> arithmetic type
```

21.2 Interactions with Other Files and STM32

ArithmeticType.h is depended upon by Linear.h, Sgd.h, AdamW.h, ExecuteOp.h, Matmul.h, and all layer config structs. It is the bridge between quantization (storage format) and computation (arithmetic path). Every function that selects between float and integer computation uses an arithmetic_t parameter.

On the STM32F746ZG, ARITH_FLOAT32 uses the single-precision FPU (VFMA instruction, 1-cycle throughput). ARITH_SYM_INT32 uses the 32-bit integer multiply-accumulate (SMLAL for 64-bit product, or MUL+ADD). The FQT integer path avoids the FPU entirely, which reduces power consumption and allows running the backward pass in integer arithmetic.

CONCEPT	The arithmetic_t abstraction allows ODT to implement mixed-precision training cleanly. Forward pass: ARITH_SYM_INT32 (integer matmul + scale propagation). Backward pass: ARITH_SYM_INT32 with SR_HALF_AWAY rounding. Weight update (SGD): read integer params, convert to float for lr multiply, write back as integer with SR. The arithmetic_t tells each function which path to take.

21.3 Line-by-Line Explanation

arithmeticFromQuantizationOrDefault()

This function inspects the quantization config and returns the matching arithmetic type. It is called in linearInitConfig(), conv1dInitConfig(), and other layer initializers to set the arithmetic type from the weight quantization. If the quantization is NULL (unspecified), it defaults to ARITH_FLOAT32 for safety.

```
arithmetic_t arithmeticFromQuantizationOrDefault(
quantization_t *q) {
if (q == NULL)                                  // No quantization specified
return (arithmetic_t){ARITH_FLOAT32, HALF_AWAY};  // Float default
if (q->type == SYM_INT32)                       // Integer quantization
return (arithmetic_t){ARITH_SYM_INT32, SR_HALF_AWAY};  // SR for FQT
if (q->type == FLOAT32)                         // Float quantization
return (arithmetic_t){ARITH_FLOAT32, HALF_AWAY};
// Unknown type: safe default
return (arithmetic_t){ARITH_FLOAT32, HALF_AWAY};
}
```

Usage in linearInitConfig()

When a linear layer is initialized, it calls arithmeticFromQuantizationOrDefault() on its weight quantization to set the arithmetic type. This is stored in linearConfig_t.arithType and used in every forward and backward pass to select the correct computation kernel.

```
void linearInitConfig(linearConfig_t *cfg) {
if (cfg == NULL) { PRINT_ERROR('null cfg'); exit(1); }
// Set arithmetic type from weight quantization:
cfg->arithType = arithmeticFromQuantizationOrDefault(
cfg->weights->param->quantization);              // Auto-select FLOAT32 or SYM_INT32
// Set defaults for optional fields:
cfg->weightMask = NULL; // Dense by default
cfg->hasBias = (cfg->bias != NULL);
// Validate dimensions:
validateLinearDimensions(cfg);
}
```

Using arithmetic_t in Forward Pass

```
// In linearForward():
switch (cfg->arithType.type) {
case ARITH_FLOAT32:
matmulFloat32Tensors(input, weights, output);   // Float path
break;
```

`case ARITH_SYM_INT32:`	
`matmulSymInt32Tensors(input, weights, output);`	`// Integer path + scale`
`break;`	
`default:`	
`PRINT_ERROR('unsupported arith'); exit(1);`	
`}`	

21.4 Missing RigL Code

ArithmeticType.h does not need RigL changes. The mask-aware matmul operates below the arithmetic dispatch: whether the arithmetic type is ARITH_FLOAT32 or ARITH_SYM_INT32, the mask check (tensorBoolGet) is identical. The same flatIdx formula applies to both compute paths.

`// RigL mask check is arithmetic-type agnostic:`	
`size_t flatIdx = row * aColumns + col;`	`// Same for both paths`
`if (weightMask != NULL && !tensorBoolGet(weightMask, flatIdx))`	
`continue;`	`// Skip regardless of FLOAT32 or SYM_INT32`

TIP For HAR on STM32 with RigL: use ARITH_FLOAT32 for initial development (easier debugging), then switch to ARITH_SYM_INT32 for final deployment. The FQT integer path reduces memory by 34.8% and latency by 49% compared to float, while the 90% sparse RigL mask reduces active weight count by 10x.

Chapter 22

Add.h / Add.c + Sub.h / Sub.c + Mul.h / Mul.c

Element-Wise Arithmetic Operations

22.1 Usage of This File

Add.c, Sub.c, and Mul.c implement elementwise arithmetic operations on tensors. These primitives underpin the optimizer kernels, normalization layers, and residual connections. The most important function is accumulateTensorIntoFloat32Inplace() which accumulates gradients across a mini-batch.

In the ODT training loop, these operations appear in: SGD update (param -= lr * grad), bias gradient accumulation (dB += sum over batch), LayerNorm mean subtraction (x -= mean), and residual addition in skip connections.

```
// Add.h
void addTensors(tensor_t *a, tensor_t *b, tensor_t *out);          // out = a + b
void addTensorsInplace(tensor_t *a, tensor_t *b);                  // a += b
void accumulateTensorIntoFloat32Inplace(
tensor_t *acc, tensor_t *increment);                               // acc += dequant(increment)

// Sub.h
void subTensors(tensor_t *a, tensor_t *b, tensor_t *out);          // out = a - b

// Mul.h
int32_t mulInt32s(int32_t a, int32_t b);                           // Scalar int multiply
float mulFloat32s(float a, float b);                               // Scalar float multiply
void mulTensorByScalar(tensor_t *t, float s, tensor_t *out);       // out = s * t
```

22.2 Interactions with Other Files and STM32

Add.c/Sub.c/Mul.c depend on DTypes.h, Tensor.h, and TensorConversion.h. They are called by: ExecuteOp.c (accumulate modes), LayerNorm.c (subtract mean, scale by gamma), Sgd.c (param update), AdamW.c (moment updates via axpby), and Conv1d.c (bias gradient accumulation).

On STM32F746ZG, element-wise float operations use VFADD/VFSUB/VFMUL. For int32 operations, QADD/QSUB handle saturating arithmetic. GCC -O2 can auto-vectorize some loops using the Cortex-M7 SIMD extensions if arrays are aligned and loop bounds are compile-time constants.

CONCEPT accumulateTensorIntoFloat32Inplace() is the key function for mini-batch gradient accumulation. Strategy A (used by ODT): maintain gradient accumulator as float32, add each sample's gradient after dequantization. This avoids integer overflow when summing N integer gradients with large values.

22.3 Line-by-Line Explanation

addTensors() -- Element-Wise Addition

```
void addTensors(tensor_t *a, tensor_t *b, tensor_t *out) {
size_t n = calcNumberOfElementsByTensor(a);                    // Total elements
for (size_t i = 0; i < n; i++) {
float av = readBytesAsFloat(&a->data[i*4]);
float bv = readBytesAsFloat(&b->data[i*4]);
writeFloatToByteArray(av + bv, &out->data[i*4]);
}
}
// Uses readBytesAsFloat/writeFloatToByteArray for type
safety
// Avoids pointer aliasing UB (see DTypes.h chapter)
```

accumulateTensorIntoFloat32Inplace() -- Gradient Accumulation

This function dequantizes the increment tensor (handling SYM_INT32 -> float conversion via the scale factor) and adds it to the float32 accumulator. It is called once per sample in the training loop to build up the batch gradient.

```
void accumulateTensorIntoFloat32Inplace(
tensor_t *acc, tensor_t *increment) {
size_t n = calcNumberOfElementsByTensor(acc);
for (size_t i = 0; i < n; i++) {
float accVal = readBytesAsFloat(&acc->data[i*4]);              // Read float acc
float incVal;
if (increment->quantization->type == SYM_INT32) {
int32_t m = readBytesAsInt32(&increment->data[i*4]);           // Read integer
float scale = getSymInt32Scale(increment);
incVal = (float)m * scale;                                     // Dequantize
} else {
incVal = readBytesAsFloat(&increment->data[i*4]);
}
writeFloatToByteArray(accVal + incVal, &acc->data[i*4]);       // Accumulate
}
}
```

mulTensorByScalar() -- Learning Rate Scaling

```
void mulTensorByScalar(tensor_t *t, float s, tensor_t *out) {
size_t n = calcNumberOfElementsByTensor(t);
for (size_t i = 0; i < n; i++) {
float v = readBytesAsFloat(&t->data[i*4]);
writeFloatToByteArray(v * s, &out->data[i*4]);
}
```

```
}
// Called as: mulTensorByScalar(gradAccum, lr, scaledGrad)
// Then: subTensors(weights, scaledGrad, newWeights)
// Equivalent to: weights -= lr * gradAccum
```

22.4 Missing RigL Code

Element-wise operations do not need direct RigL changes. However, during the DROP step in rigLStep(), newly inactive weights must be zeroed. This is done with a simple loop over the mask, not these general tensor functions -- the mask check is needed anyway, so a dedicated loop is clearer.

```
// Zero weights at newly inactive positions after DROP:
float *w = (float *)cfg->weights->param->data;
float *g = (float *)cfg->weights->grad->data;
size_t n = numWeights;
for (size_t i = 0; i < n; i++) {
if (!tensorBoolGet(cfg->weightMask, i)) {
w[i] = 0.0f;                    // Zero weight
g[i] = 0.0f;                    // Zero gradient too
}
}
```

Chapter 23

Axpby.h + Reduce.h + Sum.h

Linear Algebra Utilities: Scaled Add, Reduction, Summation

23.1 Usage of This File

Axpby.c implements y = a*x + b*y (scaled vector addition) -- the fundamental BLAS-1 operation for optimizer momentum updates. Reduce.c implements reduction operations (max, mean) over tensor axes, used by Softmax and LayerNorm. Sum.c implements axis-wise and total summation, used for bias gradient computation.

The name Axpby comes from the BLAS Level-1 routine: alpha * x + beta * y. In AdamW, the moment update m = beta1*m + (1-beta1)*g is exactly axpby(1-beta1, g, beta1, m, m). PyTorch calls this operation lerp_ (linear interpolation).

```
// Axpby.h: y = a*x + b*y
void axpby(float a, tensor_t *x, float b, tensor_t *y,
tensor_t *out);
void axpbyInplace(float a, tensor_t *x, float b, tensor_t *y); // y = a*x + b*y

// Reduce.h: reductions over axes
float reduceMean(tensor_t *in, size_t axis); // Mean over one axis
float reduceMax(tensor_t *in, size_t axis); // Max over one axis
void reduceMeanToTensor(tensor_t *in, tensor_t *out, size_t
axis);

// Sum.h: summation
float sumTensor(tensor_t *in); // Sum all elements
void sumTensorAlongAxis(tensor_t *in, tensor_t *out, size_t
axis); // Sum over axis
```

23.2 Interactions with Other Files and STM32

Axpby.c is used by AdamW.c for moment updates (m = lerp(g, m, beta1)) and velocity updates (v = lerp(g^2, v, beta2)). Reduce.c is called by Softmax.c (findMaxFloat for stability) and LayerNorm.c (compute mean). Sum.c is called in linearCalcBiasGrads() to sum the batch-dimension of the bias gradient.

On STM32, the Cortex-M7 FPU can execute VFMA (fused multiply-add) in 1 cycle after a 2-cycle latency. The axpby inner loop maps directly to VFMA: acc = a * x_i + b * y_i. GCC -O2 with -mfpu=fpv5-d16 generates VLDR+VFMA sequences for float loops.

CONCEPT	The axpby operation is fundamental to all first-order optimizers: SGD with momentum (m = momentum*m + (1-momentum)*g), RMSprop (v = beta*v + (1-beta)*g^2), Adam (m = beta1*m + (1-beta1)*g). All are special cases of y = a*x + b*y with different values of a and b.

23.3 Line-by-Line Explanation

axpby() -- Scaled Vector Addition

```
void axpby(float a, tensor_t *x,
float b, tensor_t *y, tensor_t *out) {
size_t n = calcNumberOfElementsByTensor(x);
for (size_t i = 0; i < n; i++) {
float xi = readBytesAsFloat(&x->data[i*4]);
float yi = readBytesAsFloat(&y->data[i*4]);
float res = a * xi + b * yi;                    // a*x + b*y -- maps to VFMA
writeFloatToByteArray(res, &out->data[i*4]);
}
}
// AdamW moment update: axpby(1-beta1, grad, beta1, m, m)
// Result: m = (1-beta1)*grad + beta1*m
```

reduceMean() -- Mean Reduction

reduceMean() computes the arithmetic mean over one axis of a tensor. For a [batch, features] tensor, reduceMean(t, 0) computes the mean over the batch dimension, producing a [features] tensor. This is used in LayerNorm to compute the mean feature value across the feature dimension.

```
float reduceMeanScalar(tensor_t *in) {          // Mean of all elements
size_t n = calcNumberOfElementsByTensor(in);
float sum = 0.0f;
for (size_t i = 0; i < n; i++)
sum += readBytesAsFloat(&in->data[i*4]);
return sum / (float)n;
}
```

sumTensorAlongAxis() -- Bias Gradient

sumTensorAlongAxis() is called to sum the bias gradient over the batch dimension. For a linear layer with batch_size=B and out_features=F, the bias gradient is a [B, F] tensor. Summing over axis 0 gives the [F] bias gradient for the SGD update.

```
// Bias gradient accumulation:
// dBias has shape [batch_size, out_features]
// We need [out_features] for the update
sumTensorAlongAxis(dBias_batch, dBias, 0);      // Sum over batch axis
sgdUpdate(bias->param, bias->grad, optimizer);  // Update scalar biases
```

23.4 Missing RigL Code

Axpby, Reduce, and Sum do not need RigL changes. They operate on activations and optimizer moment tensors, not on the weight mask. The mask check is only needed in: (1) the matmul inner loop (Matmul.c), (2)

the optimizer weight update (Sgd.c/AdamW.c), and (3) the rigLStep DROP/GROW logic (Layer.c).

TIP	Activation tensors are always dense -- sparsity applies only to weight matrices. Therefore, Axpby, Reduce, and Sum operations on activations are always full-rank and do not need mask-aware variants. Only Matmul.c and the optimizer kernels need the mask check.

Chapter 24

Distributions.h / Distributions.c + Log.h / Log.c

Statistical Distributions and Log for Loss Functions

24.1 Usage of This File

Distributions.h provides sampling functions for standard probability distributions: Gaussian (normal), Bernoulli, and Uniform. These are used by PPCA replay (Gaussian samples for synthetic data), RigL mask initialization (Bernoulli draws), and dropout (Bernoulli masks). Log.h provides safe logarithm functions that guard against log(0) = -inf, which would cause NaN propagation in the loss computation.

The distinction between Distributions.h and RNG.h/Bernoulli.h is that Distributions.h provides higher-level sampling (e.g., Gaussian via Box-Muller), while RNG.h provides the low-level uniform random bit generation and Bernoulli.h provides binary mask filling. Log.h is a separate utility for numerically safe log operations.

```
// Distributions.h
float sampleGaussian(float mu, float sigma, rng32_t *rng);   // N(mu, sigma^2)
float sampleStandardNormal(rng32_t *rng);                    // N(0, 1) via Box-Muller
float sampleUniform(float lo, float hi, rng32_t *rng);       // U[lo, hi)
bool sampleBernoulli(float p, rng32_t *rng);                 // Bernoulli(p)

// Log.h
float safeLogFloat(float x);                                 // log(max(x, 1e-7)) -- prevents -inf
float safeLog2Float(float x);                                // log2(max(x, 1e-7))
```

24.2 Interactions with Other Files and STM32

Distributions.h depends on RNG.h (for rngNextFloatCtx). Log.h has no dependencies beyond math.h. Distributions.c is called by PpcaReplay.c (Gaussian samples), Dropout.c (Bernoulli masks), and any test code generating synthetic data. Log.h is called by CrossEntropy.c (safeLogFloat to prevent NaN loss).

On STM32F746ZG, logf() is computed using the CMSIS-DSP library or the newlib-nano math library. The STM32F7 does not have a hardware log instruction -- logf() takes ~50-100 cycles. For the CrossEntropy loss (6 class probabilities), that is 6 logf() calls = ~300-600 cycles total, which is negligible.

CONCEPT	Box-Muller transform for N(0,1): given U1, U2 ~ Uniform(0,1), compute Z = sqrt(-2*ln(U1)) * cos(2*pi*U2). This produces a standard normal random variable Z using only two uniform random draws and one sqrtf/cosf/logf call. Used in ppcaSample() for latent code generation.

24.3 Line-by-Line Explanation

sampleStandardNormal() -- Box-Muller

```
float sampleStandardNormal(rng32_t *rng) {
// Box-Muller: needs two independent uniform samples
float u1 = rngNextFloatCtx(rng);                    // U1 ~ Uniform(0,1)
if (u1 < 1e-10f) u1 = 1e-10f;                       // Guard against log(0)
float u2 = rngNextFloatCtx(rng);                    // U2 ~ Uniform(0,1)
// Z = sqrt(-2*ln(u1)) * cos(2*pi*u2)
float r = sqrtf(-2.0f * logf(u1));                  // Rayleigh magnitude
float theta = 2.0f * M_PI * u2;                     // Angle
return r * cosf(theta);                             // N(0,1) sample
// Note: r*sinf(theta) is a second independent N(0,1)
// but ODT discards it for simplicity
}
```

safeLogFloat() -- NaN Prevention

safeLogFloat() guards against log(0) which would produce -infinity and cause NaN to propagate through the backward pass. The clamp value 1e-7 ensures log(1e-7) = -16.1 is a large but finite negative number, bounding the loss at a maximum finite value.

```
float safeLogFloat(float x) {
// Same threshold as PyTorch: torch.clamp(x, min=1e-7)
if (x < 1e-7f) x = 1e-7f;                           // Clamp to minimum
return logf(x);
}

// In CrossEntropyForward:
// loss += y[c] * -safeLogFloat(p[c])
// Without safeLog: if p[c]=0 and y[c]=1:
// -log(0) = +inf -> NaN in backward
// With safeLog: -log(1e-7) = 16.1 (large but finite)
```

sampleBernoulli() -- Binary Draw

```
bool sampleBernoulli(float p, rng32_t *rng) {
return rngNextFloatCtx(rng) < p;                    // True with prob p
}

// Dropout usage: for each activation x_i:
// if sampleBernoulli(dropoutRate, rng): x_i = 0
// else: x_i /= (1 - dropoutRate) // inverted dropout
```

24.4 Missing RigL Code

Distributions.h and Log.h do not need RigL changes. The Bernoulli draws for mask initialization are already handled by Bernoulli.h (bernoulliFillMask). RigL weight mask updates (DROP/GROW) use deterministic

threshold comparisons, not random draws. Only the initial mask is stochastic.

TIP	The HAR CrossEntropy loss with 6 classes calls safeLogFloat() 6 times per sample. On MCU, the 6 logf() calls (~300 cycles) are dominated by the linear layer forward pass (~260K cycles for a 1152->64 matmul). Loss computation is not a bottleneck.

Chapter 25

Kernel.h / Kernel.c + PointwiseFused.h

Kernel Infrastructure and Fused Pointwise Operations

25.1 Usage of This File

Kernel.h provides the base infrastructure for all compute kernels in ODT. A kernel is the innermost computation function that operates on raw data pointers rather than tensor_t structs. Separating kernel logic from tensor bookkeeping allows the same kernel to be reused by executeOp() with different input/output storage dtypes.

PointwiseFused.h provides fused kernels that combine multiple pointwise operations in a single pass over data: for example, bias addition fused with ReLU activation. Fusion eliminates intermediate buffer reads and writes, reducing memory bandwidth pressure on the STM32 AHB bus and reducing stack usage.

```
// Kernel.h: raw kernel function signatures
typedef void (*floatKernelFn_t)(float *a, float *b, float *out, size_t n);
typedef void (*int32KernelFn_t)(int32_t *a, int32_t *b, int32_t *out, size_t n);
typedef struct kernelOp {
floatKernelFn_t floatKernel; // Float32 compute path
int32KernelFn_t int32Kernel; // Int32 compute path
const char *name; // Debug label
} kernelOp_t;

// PointwiseFused.h:
void fusedBiasRelu(float *x, float *bias, float *out, size_t features, size_t batch);  // out=max(0,x+bias)
void fusedBiasAdd(float *x, float *bias, float *out, size_t features, size_t batch);  // out=x+bias
void fusedScaleShift(float *x, float *gamma, float *beta, float *out, size_t n);  // out=gamma*x+beta
void fusedAddRelu(float *x, float *y, float *out, size_t n);  // out=max(0,x+y)
```

25.2 Memory Bandwidth Analysis

Fusion is important on STM32 because SRAM bandwidth is the bottleneck for pointwise operations. The AHB bus to SRAM D-TCM runs at up to 2 GB/s at 216 MHz. For a 64-neuron activation layer (256 bytes), unfused operations require reading and writing the full buffer for each operation.

```
// UNFUSED: bias add then ReLU (two separate passes)
// Pass 1 (bias add): read 256B in + read 256B bias + write 256B temp = 768B
```

```
// Pass 2 (ReLU): read 256B temp + write 256B out = 512B
// Total: 1280B memory traffic per sample

// FUSED: fusedBiasRelu (single pass)
// Pass 1: read 256B in + read 256B bias + write 256B out = 768B
// Total: 768B memory traffic per sample
// Saving: 512B = 40% reduction in memory traffic

// For batch=32: unfused=40960B, fused=24576B (saves 16384B of bandwidth)
```

CONCEPT Memory bandwidth on STM32F746ZG AHB bus: ~2 GB/s at 216 MHz. A 64-neuron activation buffer is 256 bytes. Fused bias+ReLU saves 40% memory bandwidth vs. two separate passes. Over thousands of training iterations, this bandwidth saving translates to measurably faster training on the constrained MCU.

25.3 Line-by-Line Explanation

fusedBiasRelu() -- Single-Pass Bias + Activation

fusedBiasRelu() is the most commonly called fused kernel in ODT. It is used by every LINEAR layer that has a ReLU activation. The fused version avoids materializing the intermediate (x + bias) buffer in SRAM.

```
void fusedBiasRelu(float *x, float *bias, float *out,
size_t features, size_t batch) {
for (size_t b = 0; b < batch; b++) {          // Outer: batch samples
for (size_t f = 0; f < features; f++) {       // Inner: features
size_t idx = b * features + f;                // Flat index
float v = x[idx] + bias[f];                   // Bias add (same bias per sample)
out[idx] = (v > 0.0f) ? v : 0.0f;             // ReLU in same pass
}
}
}
// No intermediate buffer needed
// Cortex-M7 FPU: VADD + VCMP + VMOVLT in ~5 cycles per element
// For features=64, batch=32: ~64*32*5 = 10240 cycles ≈ 47 microseconds
```

fusedScaleShift() -- LayerNorm Final Step

fusedScaleShift() applies the learnable scale (gamma) and shift (beta) parameters that are the final step of LayerNorm: out = gamma * normalized + beta. This is the affine transform that allows LayerNorm to learn to undo the normalization if beneficial.

```
void fusedScaleShift(float *x, float *gamma, float *beta,
float *out, size_t n) {
```

```
for (size_t i = 0; i < n; i++)                                  // Single pass
out[i] = gamma[i] * x[i] + beta[i];                             // FMA: gamma*x + beta
}
// Cortex-M7 VFMA instruction: gamma*x+beta in 1 FPU cycle (pipelined)
// For n=64: 64 VFMA + 64 VLDR*2 + 64 VSTR ≈ 192 FPU ops ≈ 100 cycles
// Maps to SIMD with NEON on Cortex-A; on M7 it is scalar FPU
```

fusedAddRelu() -- Residual Connection + Activation

fusedAddRelu() adds two tensors element-wise and applies ReLU. This is used in residual connections (skip connections) where the shortcut and main path are summed and then activated. Without fusion, this would require a separate Add pass followed by a ReLU pass.

```
void fusedAddRelu(float *x, float *y, float *out, size_t n) {
for (size_t i = 0; i < n; i++) {
float v = x[i] + y[i];                                          // Residual addition
out[i] = (v > 0.0f) ? v : 0.0f;                                 // ReLU
}
}
// Used when main path and residual branch are added before activation.
// Example: out = ReLU(conv_output + residual_input)
```

25.4 Kernel.c: The executeOp Bridge

Kernel.c provides the routing logic that executeOp() uses to select the right kernel based on arithmetic type. When executeOp() is called with ARITH_FLOAT32, it selects floatKernel; for ARITH_INT32 it selects int32Kernel. This abstraction allows the same training loop code to work with different precision settings.

```
// executeOp() kernel selection (from ExecuteOp.c):
if (op->arithmeticType == ARITH_FLOAT32) {
op->floatKernel(float_a, float_b, float_out, n);                // Float32 path
} else if (op->arithmeticType == ARITH_INT32) {
op->int32Kernel(int32_a, int32_b, int32_out, n);                // Int32 path
} else {
PRINT_ERROR('unsupported arithmetic type %d', op->arithmeticType);
}
```

25.5 Missing RigL Code

Kernel.h and PointwiseFused.h do not need RigL changes. The fused kernels operate on activations (dense outputs from the sparse matmul). Sparsity is in the weight kernel (matmulFloatCore), not in bias or activation operations. After the sparse matmul produces a dense output vector, fusedBiasRelu adds the bias and

activates it without any mask checks.

TIP	RigL + fused kernels: the combined operation is out = ReLU(sparse_matmul(x, W_sparse) + bias). The sparse matmul output is always dense (every output neuron gets a value, just computed from fewer inputs). fusedBiasRelu sees a full-rank dense input and operates normally. No changes needed.

Chapter 26

Conv1dTransposed.h / Conv1dTransposed.c

Transposed 1D Convolution for Signal Upsampling

26.1 Usage of This File

Conv1dTransposed.c implements the transposed 1D convolution (also called fractionally strided convolution or deconvolution). It is the inverse operation of Conv1d in terms of spatial dimensions, mapping from a compressed feature sequence back to a longer sequence. For HAR, it is useful in encoder-decoder architectures where the model compresses a 128-sample window and then reconstructs it.

The output length formula is: L_out = (L_in - 1) * stride - 2*padding + kernel_size. For L_in=32, stride=4, kernel_size=4, padding=0: L_out = (32-1)*4 + 4 = 128. This recovers the original sequence length from a 4x downsampled representation.

```
typedef struct conv1dTransposedConfig {
parameter_t *kernel; // Shape: [in_ch, out_ch, kernel_size]  // Note: transposed dims
parameter_t *bias; // Shape: [out_ch]
size_t inChannels; // Input channels (compressed)
size_t outChannels; // Output channels (expanded)
size_t kernelSize; // Filter length
size_t stride; // Upsampling factor
size_t padding; // Input padding (removes output samples)
size_t outputPadding; // Extra output samples for odd output sizes
} conv1dTransposedConfig_t;

void conv1dTransposedForward(layer_t *l, tensor_t *in, tensor_t *out);
void conv1dTransposedBackward(layer_t *l, tensor_t *fwdIn,
tensor_t *loss, tensor_t *propLoss);
shape_t conv1dTransposedCalcOutputShape(layer_t *l, shape_t in);
```

C Usage Example: Encoder-Decoder for HAR

```
// Encoder: compress 128 -> 32 samples, 9 -> 16 channels
layer_t *enc = makeConv1d(9, 16, kernelSize=4, stride=4, padding=0);
// Decoder: reconstruct 32 -> 128 samples, 16 -> 9 channels
layer_t *dec = makeConv1dTransposed(16, 9, kernelSize=4, stride=4, padding=0);
```

```
// dec output shape: L_out = (32-1)*4 + 4 = 128. Shape: [batch, 9, 128]
// This recovers the original sensor signal shape!

// Autoencoder training: minimize MSE(dec_out, original_input)
// Useful for: anomaly detection, feature visualization, pre-training
```

26.2 Interactions with Other Files and STM32

Conv1dTransposed.c depends on Conv1dKernel.c (shared kernel implementations), Tensor.h, DTypes.h, and ExecuteOp.h. The transposed convolution is implemented as a regular convolution with the dilated (stride-upsampled) input and flipped kernel. On STM32, the output tensor is up to stride times larger than the input, so careful memory planning is required.

CONCEPT	Transposed conv is NOT the inverse of conv in terms of values -- it is the transpose of the conv operation as a linear map. If conv is matrix C, then conv-transposed computes C^T * x. This is exactly the backward-pass operation for a regular conv: the backward through a Conv1d(in_ch, out_ch) with respect to the input IS a Conv1dTransposed(out_ch, in_ch).

On STM32F746ZG, a Conv1dTransposed(16, 9, kernel=4, stride=4) with L_in=32 produces L_out=128. The kernel has 16*9*4=576 weights. For each of 128 output time steps, the kernel checks which input time step contributes (stride-alignment check). At 90% sparsity (if masked), only 58 kernel weights are active -- dramatically reducing operations.

26.3 Line-by-Line Explanation

conv1dTransposedCalcOutputShape() -- Output Dimensions

```
shape_t conv1dTransposedCalcOutputShape(layer_t *l, shape_t in) {
conv1dTransposedConfig_t *cfg = l->config->conv1dTransposed;
size_t L_in = in.dimensions[2];                    // Input time length
size_t L_out = (L_in - 1) * cfg->stride
- 2 * cfg->padding
+ cfg->kernelSize
+ cfg->outputPadding;
shape_t out;
out.numberOfDimensions = 3;
out.dimensions[0] = in.dimensions[0];              // Batch unchanged
out.dimensions[1] = cfg->outChannels;              // Expanded channels
out.dimensions[2] = L_out;                         // Upsampled time
return out;
}
```

```
// Example: L_in=32, stride=4, k=4, pad=0, outPad=0 -> L_out=128
```

conv1dTransposedForward() -- Scatter-Add Implementation

The transposed convolution forward pass uses the scatter-add interpretation: for each input position t_in and kernel position k, the input contributes to output position t_out = t_in * stride + k - padding. This requires a bounds check to ensure t_out is in [0, L_out). The stride check from the regular conv becomes a scatter index instead of a gather index.

```
void conv1dTransposedForward(layer_t *l, tensor_t *in, tensor_t *out) {
// Initialize output to bias:
for (size_t oc = 0; oc < outCh; oc++)
for (size_t t = 0; t < L_out; t++)
out[oc, t] = bias[oc];                          // Start with bias
// Scatter: for each input (ic, tIn), scatter to output:
for (size_t ic = 0; ic < inCh; ic++) {          // Input channel
for (size_t tIn = 0; tIn < L_in; tIn++) {       // Input time
for (size_t oc = 0; oc < outCh; oc++) {         // Output channel
for (size_t k = 0; k < kernelSize; k++) {       // Kernel position
int tOut = (int)(tIn * stride) + (int)k - (int)padding;
if (tOut < 0 || tOut >= (int)L_out) continue;   // Boundary check
out[oc, tOut] += kernel[ic, oc, k] * in[ic, tIn];  // Scatter-add
}
}
}
}
}
```

conv1dTransposedBackward() -- Backward = Regular Convolution

The backward pass with respect to the input is a regular Conv1d (with the kernel flipped). This is the mathematical transpose of the transposed convolution. The weight gradient is a regular cross-correlation of input and propagated loss.

```
// Backward through Conv1dTransposed:
// dL/d(in[ic, tIn]) = sum_oc sum_k dL/d(out[oc, tOut]) * kernel[ic, oc, k]
// where tOut = tIn * stride + k - padding

// This is equivalent to:
// propLoss = Conv1d(loss, kernel_flipped, stride=stride)
// i.e., the backward through transpose conv is a FORWARD regular conv

// Weight gradient:
```

```
// dL/d(kernel[ic, oc, k]) = sum_tIn in[ic, tIn] *
dL/d(out[oc, tIn*stride+k])
```

26.4 Missing RigL Code

Conv1dTransposed.c could support RigL sparsity on its kernel weights using the same pattern as Conv1d: add a kernelMask tensor_t* to conv1dTransposedConfig_t and check tensorBoolGet(kernelMask, flatIdx) in the scatter-add inner loop. However, transposed convolutions are typically used only in decoder networks, which are often small.

RigL: Low priority for RigL	Conv1dTransposed kernels are typically small (e.g., 16*9*4 = 576 weights). The memory saving from 90% sparsity (saves 52 bytes of compute, 72 bytes of mask storage) is minimal. Focus RigL on the large LINEAR layers (e.g., 64*1152 = 294K parameter weights). Apply RigL to Conv1dTransposed only if the model has very large decoder kernels.

Chapter 27

MaxPool1d.h + AvgPool1d.h + AdaptiveAvgPool1d.h

1D Pooling Layers for Temporal Downsampling

27.1 Usage of This File

The pooling layers implement temporal downsampling of 1D feature sequences. MaxPool1d selects the maximum value in each window, preserving strong local features. AvgPool1d computes the average, providing smoother downsampling. AdaptiveAvgPool1d takes a target output size rather than kernel_size, computing the required kernel and stride automatically.

For HAR with Conv1d output of shape [16, 126] (16 channels, 126 time steps), MaxPool1d(kernel_size=4, stride=4) produces [16, 31], reducing the feature dimension from 2016 to 496. This significantly reduces the weight count in subsequent linear layers.

```
// MaxPool1d.h
typedef struct maxPool1dConfig {
size_t kernelSize, stride, padding;
} maxPool1dConfig_t;
void maxPool1dForward(layer_t*, tensor_t *in, tensor_t *out);
void maxPool1dBackward(layer_t*, tensor_t *fwdIn,
tensor_t *loss, tensor_t *propLoss);

// AdaptiveAvgPool1d.h
typedef struct adaptiveAvgPool1dConfig {
size_t outputSize;                          // Target output length
} adaptiveAvgPool1dConfig_t;
void adaptiveAvgPool1dForward(layer_t*, tensor_t *in,
tensor_t *out);
```

27.2 Interactions with Other Files and STM32

Pooling layers depend on Tensor.h and DTypes.h. MaxPool1d backward requires storing the argmax indices from the forward pass (which positions won the max competition) to route gradients correctly. This requires an auxiliary index buffer -- on STM32, this must be pre-allocated as part of model setup.

CONCEPT	MaxPool1d backward: during forward, record index[b,c,t] = argmax position within window. During backward, propLoss[b,c,argmax] += loss[b,c,t], all other positions in the window get 0. Only the maximum-contributing input receives the gradient. AvgPool1d backward distributes loss[b,c,t] / kernelSize evenly to all positions in the window.

WARNING	AdaptiveAvgPool1d computes kernel_size = floor(L_in / output_size) and stride = floor(L_in / output_size). This is an approximation that may not exactly divide L_in. The PyTorch implementation handles uneven splits by using ceil/floor, but the ODT implementation may differ slightly for non-divisible sizes.

27.3 Line-by-Line Explanation

maxPool1dForward() -- Window Maximum

`void maxPool1dForward(layer_t *l, tensor_t *in, tensor_t *out) {`	
`maxPool1dConfig_t *cfg = l->config->maxPool1d;`	
`for (size_t b = 0; b < batch; b++) {`	
`for (size_t c = 0; c < channels; c++) {`	
`for (size_t t = 0; t < L_out; t++) {`	
`size_t tStart = t * cfg->stride - cfg->padding;`	
`float maxVal = -FLT_MAX;`	
`for (size_t k = 0; k < cfg->kernelSize; k++) {`	
`size_t tIn = tStart + k;`	
`if (tIn >= L_in) continue;`	`// Boundary check`
`float v = readBytesAsFloat(&in->data[...]);`	
`if (v > maxVal) maxVal = v;`	`// Track maximum`
`}`	
`writeFloatToByteArray(maxVal, &out->data[...]);`	
`}`	
`}`	
`}`	
`}`	

adaptiveAvgPool1dForward() -- Target Output Size

`void adaptiveAvgPool1dForward(layer_t *l, tensor_t *in, tensor_t *out) {`	
`size_t outputSize = l->config->adaptiveAvgPool1d->outputSize;`	
`size_t L_in = getDimensionsByIndex(in, 1);`	
`// Compute effective kernel and stride:`	
`size_t stride = L_in / outputSize;`	`// Integer division`
`size_t kernel = L_in - (outputSize-1) * stride;`	`// Last bin wider`
`for (size_t t = 0; t < outputSize; t++) {`	
`size_t tStart = t * stride;`	
`float sum = 0.0f;`	
`for (size_t k = 0; k < kernel; k++)`	
`sum += readBytesAsFloat(&in->data[(tStart+k)*4]);`	

```
writeFloatToByteArray(sum/kernel, &out->data[t*4]);
}
}
```

27.4 Missing RigL Code

Pooling layers do not need RigL changes. They operate on activation tensors (outputs of conv layers), which are always dense. Sparsity in the preceding Conv1d layer affects which values are computed, but the pooling layer receives the full output tensor and pools over it normally.

TIP	For HAR deployment: AdaptiveAvgPool1d(1) computes the global average over the time dimension, collapsing [channels, T] to [channels]. This is a powerful way to make the model invariant to sequence length -- useful if the HAR system processes variable-length windows.

Chapter 28

Dropout.h / Dropout.c

Stochastic Regularization via Random Neuron Silencing

28.1 Usage of This File

Dropout.c implements stochastic regularization: during training, each activation is independently zeroed with probability p (the dropout rate) and scaled by 1/(1-p) (inverted dropout scaling). During inference, dropout is bypassed entirely and all activations are passed through unchanged.

For HAR training with 320 KB SRAM, dropout is an effective regularizer because it requires only a Bernoulli mask tensor of the same shape as the activation. For a 64-neuron hidden layer, the mask is 64 bits = 8 bytes. Dropout prevents co-adaptation of neurons and improves generalization from the small HAR training set.

```
typedef struct dropoutConfig {
float dropoutRate; // Probability of zeroing (0.0 to 1.0)
bool training; // false = bypass during inference
tensor_t *mask; // BOOL tensor, pre-allocated
} dropoutConfig_t;

void dropoutForward(layer_t*, tensor_t *in, tensor_t *out);
void dropoutBackward(layer_t*, tensor_t *fwdIn,
tensor_t *loss, tensor_t *propLoss);
shape_t dropoutCalcOutputShape(layer_t*, shape_t in);  // Same as input
```

28.2 Interactions with Other Files and STM32

Dropout.c depends on Bernoulli.h (bernoulliFillMask for mask generation), Tensor.h, DTypes.h, and RNG.h. The dropout mask is regenerated each forward pass during training. During inference (cfg->training = false), the function is a no-op pass-through. The BOOL mask tensor is pre-allocated to avoid runtime malloc.

WARNING	IMPORTANT: always set cfg->training = false before inference. Leaving training=true during inference causes stochastic outputs (different predictions for the same input each run). On STM32, where there is no automatic training/eval mode switch, this must be done explicitly before deploying.

CONCEPT	Inverted dropout scales by 1/(1-p) during training so that the expected value of the output is unchanged: E[scale * Bernoulli(1-p) * x] = scale * (1-p) * x. With scale = 1/(1-p): E[output] = x. This means inference does NOT need to scale -- the model weights already have the correct magnitude.

28.3 Line-by-Line Explanation

dropoutForward() -- Inverted Dropout

```
void dropoutForward(layer_t *l, tensor_t *in, tensor_t *out)
{
dropoutConfig_t *cfg = l->config->dropout;
size_t n = calcNumberOfElementsByTensor(in);
if (!cfg->training) {                          // Inference: bypass
copyTensor(in, out);                           // Direct copy
return;
}
// Generate fresh Bernoulli mask each forward pass:
bernoulliFillMask(cfg->mask, 1.0f - cfg->dropoutRate);  // Keep prob = 1-rate
float scale = 1.0f / (1.0f - cfg->dropoutRate);         // Inverted scaling
for (size_t i = 0; i < n; i++) {
bool keep = tensorBoolGet(cfg->mask, i);
float v = readBytesAsFloat(&in->data[i*4]);
float out_v = keep ? (v * scale) : 0.0f;       // Zero or scale
writeFloatToByteArray(out_v, &out->data[i*4]);
}
}
```

dropoutBackward() -- Gradient Gating

Dropout backward uses the same mask generated during forward to gate the incoming gradient. Positions that were zeroed in the forward pass get zero gradient (no update for those weights); active positions get the scaled gradient.

```
void dropoutBackward(layer_t *l, tensor_t *fwdIn,
tensor_t *loss, tensor_t *propLoss) {
dropoutConfig_t *cfg = l->config->dropout;
float scale = 1.0f / (1.0f - cfg->dropoutRate);
size_t n = calcNumberOfElementsByTensor(loss);
for (size_t i = 0; i < n; i++) {
bool kept = tensorBoolGet(cfg->mask, i);       // Same mask as fwd
float g = readBytesAsFloat(&loss->data[i*4]);
float pg = kept ? (g * scale) : 0.0f;          // Gate gradient
writeFloatToByteArray(pg, &propLoss->data[i*4]);
}
}
```

28.4 Missing RigL Code and Interaction

Dropout.c does not need RigL changes. However, there is an important interaction: Dropout uses BOOL tensors (via tensorBoolSet/Get) for its activation mask, the same infrastructure used by RigL for the weight

mask. The bit-packing format is identical. Both use bernoulliFillMask() for initialization -- dropout calls it every forward pass, RigL calls it once at initialization.

```
// Dropout mask vs RigL weight mask: same infrastructure
// Dropout: mask shape = activation shape [batch, features]
// bit[i] = 1 (keep activation i), 0 (zero activation i)
// RigL: mask shape = weight shape [out_features, in_features]
// bit[i] = 1 (weight is active), 0 (weight is zero)
// Both use tensorBoolGet/Set for O(1) bit access
```

TIP Typical dropout rates for HAR: 0.3-0.5 for linear hidden layers, 0.1-0.2 for convolutional layers. Start with 0.3 and adjust based on validation accuracy. If validation accuracy keeps increasing with more training but training accuracy is much higher, dropout rate is too low. If both plateau early, rate is too high.

Chapter 29

LayerNorm.h / LayerNorm.c + GroupNorm.h / GroupNorm.c

Normalization Layers: LayerNorm and GroupNorm

29.1 Usage of This File

LayerNorm.c normalizes activations over the feature dimension of each sample independently. For a [batch, features] input, it computes mean and variance over the features axis, normalizes, then applies learnable scale (gamma) and shift (beta). GroupNorm.c generalizes LayerNorm by dividing channels into groups and normalizing within each group.

For HAR, LayerNorm is preferred over BatchNorm because it operates on a single sample, making it compatible with the streaming single-sample inference required on MCU (no batch statistics to track at inference time). GroupNorm with G=1 is equivalent to LayerNorm; GroupNorm with G=C is equivalent to InstanceNorm.

```
typedef struct layerNormConfig {
parameter_t *gamma; // Learnable scale [features]
parameter_t *beta; // Learnable shift [features]
float eps; // Numerical stability (default 1e-5)
size_t normalizedShape; // Number of features to normalize over
tensor_t *savedMean; // Cached for backward (one per batch sample)
tensor_t *savedInvStd; // Cached for backward
} layerNormConfig_t;

typedef struct groupNormConfig {
parameter_t *gamma, *beta;
size_t numGroups; // G: number of channel groups
float eps;
} groupNormConfig_t;
```

29.2 Math: Forward Pass Step by Step

LayerNorm applies four operations in sequence. Given a feature vector x of length N for one sample in the batch:

```
// Step 1: compute mean over features
// mu = (1/N) * sum_i(x_i)
// Example: x=[2.0, -1.0, 3.0, 0.5], N=4
```

```
// mu = (2.0 + (-1.0) + 3.0 + 0.5) / 4 = 1.125
// Step 2: compute variance over features
// sigma^2 = (1/N) * sum_i((x_i - mu)^2)
// (2.0-1.125)^2=0.766 (-1.0-1.125)^2=4.516
(3.0-1.125)^2=3.516 (0.5-1.125)^2=0.391
// sigma^2 = (0.766 + 4.516 + 3.516 + 0.391) / 4 = 2.297
// Step 3: normalize
// x_hat_i = (x_i - mu) / sqrt(sigma^2 + eps)
// invStd = 1 / sqrt(2.297 + 1e-5) = 1 / 1.516 = 0.660
// x_hat = [(2.0-1.125)*0.660, (-1.0-1.125)*0.660, ...]
// = [0.577, -1.401, 1.240, -0.413]                    // Normalized values
// Step 4: affine transform (learnable)
// out_i = gamma_i * x_hat_i + beta_i
// If gamma=all-ones, beta=all-zeros: out = x_hat (identity)    // Default init
```

CONCEPT LayerNorm math: (1) mu = mean(x) over features. (2) sigma^2 = mean((x-mu)^2) over features. (3) x_hat = (x - mu) / sqrt(sigma^2 + eps). (4) out = gamma * x_hat + beta. Steps 1-3 perform normalization; step 4 is the learnable affine transform. eps=1e-5 prevents division by zero when the entire feature vector is constant.

29.3 Interactions with Other Files and STM32

LayerNorm.c depends on Reduce.c (mean computation), Sum.c (variance computation), math.h (sqrtf()), and PointwiseFused.h (scale+shift as final step). GroupNorm.c has the same dependencies plus additional indexing for groups. Both layers have learnable parameters (gamma, beta) that are updated by the optimizer.

On STM32F746ZG, LayerNorm for a 64-feature vector requires: 2 passes over 64 floats (mean then variance), 1 sqrtf() call, and 1 final pass for affine transform. Total: ~3*64 = 192 float operations. At 216 MHz with Cortex-M7 FPU, this takes approximately 200-400 cycles (< 2 microseconds) -- negligible overhead.

TIP Memory for backward: savedMean and savedInvStd must be stored from the forward pass for backward computation. For batch=32, features=64: 32*64*4 = 8 KB for savedMean and another 8 KB for savedInvStd. Total 16 KB on top of the 8 KB activation buffer. Plan for ~24 KB SRAM per LayerNorm layer.

29.4 Line-by-Line Explanation

layerNormForward() -- Normalize + Scale + Shift

```
void layerNormForward(layer_t *l, tensor_t *in, tensor_t
*out) {
layerNormConfig_t *cfg = l->config->layerNorm;
size_t N = cfg->normalizedShape; // features per sample
size_t batch = getDimensionsByIndex(in, 0);
float eps = cfg->eps; // default 1e-5
```

```
float *gamma = (float*)cfg->gamma->param->data;                  // Learnable scale
float *beta = (float*)cfg->beta->param->data;                    // Learnable shift
for (size_t b = 0; b < batch; b++) {
float *x = (float*)in->data + b * N;                             // Sample b features
float *y = (float*)out->data + b * N;                            // Output buffer
float mu = 0.0f;
for (size_t i = 0; i < N; i++) mu += x[i];                       // Sum features
mu /= (float)N;                                                  // Mean
float var = 0.0f;
for (size_t i = 0; i < N; i++) {
float d = x[i] - mu;                                             // Deviation
var += d * d;                                                    // Squared deviation
}
var /= (float)N;                                                 // Variance
float invStd = 1.0f / sqrtf(var + eps);                          // 1/sigma, with eps guard
if (cfg->savedMean) cfg->savedMean->data[b] = mu;                // Cache for backward
if (cfg->savedInvStd) cfg->savedInvStd->data[b] = invStd;        // Cache for backward
for (size_t i = 0; i < N; i++) {
float xhat = (x[i] - mu) * invStd;                               // Normalized value
y[i] = gamma[i] * xhat + beta[i];                                // Affine transform
}
}
}
```

layerNormBackward() -- The Complex Gradient

WARNING LayerNorm backward is significantly more complex than forward. The gradient with respect to x involves the chain rule through the normalization: dL/dx_i = (1/N) * invStd * (N*dL/dy_i - sum_j(dL/dy_j) - x_hat_i * sum_j(dL/dy_j * x_hat_j)). This requires storing x_hat (or equivalently mu and invStd) from the forward pass.

```
// LayerNorm backward (simplified, FLOAT32):
// dL/dgamma_i = sum_b(dL/dy_i * x_hat_i) (sum over batch)
// dL/dbeta_i = sum_b(dL/dy_i) (sum over batch)
// dL/dx_i = (1/N) * invStd * (N*dy_i - sum_j(dy_j) -
x_hat_i*sum_j(dy_j*x_hat_j))
// In code: for each sample b in batch:
float dy_sum = 0.0f, dy_xhat_sum = 0.0f;
for (size_t i = 0; i < N; i++) {
dy_sum += dy[i];                                                 // sum_j(dy_j)
float xhat = (x[i] - mu) * invStd;                               // Recompute x_hat
dy_xhat_sum += dy[i] * xhat;                                     // sum_j(dy_j * x_hat_j)
```

```
}
for (size_t i = 0; i < N; i++) {
float xhat = (x[i] - mu) * invStd;
dx[i] = (1.0f/N) * invStd * (N*dy[i] - dy_sum - xhat*dy_xhat_sum);
}
```

```
// Full gradient
```

29.5 GroupNorm: Generalizing LayerNorm

GroupNorm divides C channels into G groups of C/G channels each, normalizing independently within each group. For Conv1d output of shape [batch, C, T], GroupNorm computes mean and variance over (C/G, T) for each group. When G=1: normalization over all C*T values (InstanceNorm for sequences). When G=C: normalization over each channel independently.

```
// GroupNorm example: C=16 channels, G=4 groups, T=31 time steps
// Group 0: channels 0-3, all time steps: normalize over 4*31=124 values
// Group 1: channels 4-7: normalize over 124 values
// Group 2: channels 8-11, Group 3: channels 12-15

// Memory layout for GroupNorm (channels-first): [batch, C, T]
// Group g spans channels [g*(C/G), (g+1)*(C/G)-1]
// Flat index in group g: g*(C/G)*T + channel_offset*T + t

// GroupNorm is preferred over LayerNorm for Conv layers because
// it respects channel structure (each group = one semantic feature group)
```

29.6 Missing RigL Code

LayerNorm and GroupNorm do not need RigL changes. Their learnable parameters (gamma, beta) are small [features] vectors that are always kept dense -- sparsifying them would prevent the norm from learning independent per-feature scaling. Only LINEAR weight matrices and Conv1d kernels are candidates for RigL sparsification.

RigL: No mask needed	LayerNorm/GroupNorm gamma and beta are always dense. They are tiny (64 floats = 256 bytes) compared to linear weights (74K+ floats). The overhead of masking these vectors is not justified, and sparsifying them would degrade normalization quality.

Chapter 30

Flatten.h / Flatten.c

Tensor Flattening for Conv-to-Linear Transition

30.1 Usage of This File

Flatten.c implements the tensor reshape operation that converts multi-dimensional feature maps from convolutional layers into a 1D vector suitable for linear layers. For a Conv1d output of shape [batch, 16, 31] (16 channels, 31 time steps), Flatten produces [batch, 496] -- a 496-dimensional feature vector per sample.

Flatten is an O(1) metadata operation: it does not copy data, only updates the shape descriptor. The underlying data buffer is unchanged. This makes Flatten effectively free in terms of computation and memory bandwidth -- a critical property on a resource-constrained MCU.

```
typedef struct flattenConfig {
size_t startDim; // First dim to flatten (default 1 = all non-batch dims)
size_t endDim; // Last dim to flatten (default -1 = last dim)
} flattenConfig_t;

void flattenForward(layer_t*, tensor_t *in, tensor_t *out); // O(1) reshape
void flattenBackward(layer_t*, tensor_t *fwdIn,
tensor_t *loss, tensor_t *propLoss); // O(1) restore shape
shape_t flattenCalcOutputShape(layer_t*, shape_t in); // Compute flattened shape
bool flattenIsInplace(void); // Returns true: no copy needed
```

C Usage Example: Conv1d + Flatten + Linear

```
// HAR model: Conv1d(9, 16, kernel=5) + Flatten + Linear(496, 6)
// Conv1d output shape: [batch=32, channels=16, time=31]
// Flatten output shape: [batch=32, features=496]
// Linear weight shape: [6, 496] (6 classes, 496 inputs)

// In model definition:
layer_t *conv = makeConv1d(9, 16, 5, 1, 2); // in=9,out=16,k=5,s=1,p=2
layer_t *flat = makeFlatten(1); // startDim=1
layer_t *lin = makeLinear(496, 6); // 496 inputs, 6 outputs
layer_t *model[] = {conv, flat, lin};

// At runtime: no data copy for Flatten
// conv output and flat output share the same data buffer
```

30.2 Interactions with Other Files and STM32

Flatten.c depends only on Tensor.h (for shape manipulation) and Common.h. It calls either transposeTensor() or directly modifies the shape struct -- no data copies. On STM32, the O(1) nature of Flatten is critical: with 320 KB SRAM, copying a [16, 128] = 8 KB activation tensor would consume 8 KB of stack or require a temporary SRAM allocation.

CONCEPT	O(1) Flatten via shape mutation: the shape_t struct stores numberOfDimensions and dimensions[]. Flatten rewrites these to collapse dimensions [startDim..endDim] into a single dimension of size product(dims[startDim..endDim]). The data pointer and byte layout are unchanged. The Tensor.h design specifically chose row-major storage to make Flatten zero-cost.

30.3 Line-by-Line Explanation

flattenForward() -- Shape Mutation

The forward pass shares the data pointer from input to output and rewrites only the shape metadata. No bytes are moved. The output tensor points to exactly the same memory as the input tensor.

```
void flattenForward(layer_t *l, tensor_t *in, tensor_t *out)
{
flattenConfig_t *cfg = l->config->flatten;
size_t startDim = cfg ? cfg->startDim : 1;                    // Default: flatten all non-batch
// Step 1: copy tensor metadata (NOT data):
*out = *in;                                                   // Copy shape, dtype, quantization
out->data = in->data;                                         // SAME buffer: no copy!
// Step 2: compute product of flattened dimensions:
size_t flatSize = 1;
for (size_t d = startDim; d < in->shape.numberOfDimensions;
d++)
flatSize *= in->shape.dimensions[d];                          // 16*31=496 for [16,31]
// Step 3: update shape to collapsed form:
out->shape.numberOfDimensions = startDim + 1;                 // e.g., 2D becomes 2D
out->shape.dimensions[startDim] = flatSize;                   // Last dim = flat product
// Note: dimensions[0] = batch size is unchanged
}
// Example: in=[2,16,31] -> out=[2,496] for startDim=1        // Batch preserved
```

flattenBackward() -- Restore Shape

The backward pass restores the original shape metadata and passes the gradient through unchanged. Since Flatten does not transform data values, the gradient is the propagated loss tensor with the original (un-flattened) shape restored.

```
void flattenBackward(layer_t *l, tensor_t *fwdIn,
tensor_t *loss, tensor_t *propLoss) {
// Gradient of flatten: just restore the original shape.
```

```
// loss has shape [batch, 496], propLoss should have [batch, 16, 31].
*propLoss = *loss;                                  // Copy metadata
propLoss->data = loss->data;                        // Same data pointer
// Restore original shape from fwdIn shape:
propLoss->shape = fwdIn->shape;                     // Undo the flatten
}
// O(1): no data movement, just shape metadata update
// The conv backward receives a [batch, 16, 31] gradient
// This is stored in the same memory as the [batch, 496] grad
```

flattenCalcOutputShape()

```
shape_t flattenCalcOutputShape(layer_t *l, shape_t in) {
size_t startDim = getStartDim(l);
size_t flatSize = 1;
for (size_t d = startDim; d < in.numberOfDimensions; d++)
flatSize *= in.dimensions[d];                       // Product of collapsed dims
shape_t out;
out.numberOfDimensions = startDim + 1;
for (size_t d = 0; d < startDim; d++)
out.dimensions[d] = in.dimensions[d];               // Preserve batch dims
out.dimensions[startDim] = flatSize;                // Flattened dim
return out;
}
// Called during model building to determine Linear layer input size
// buildModel() calls this for each layer to chain shape inference
```

30.4 RigL and Flatten

Flatten.c does not need RigL changes. Flatten is a pure shape operation with no weight parameters and no data movement. It appears in a model between convolutional and linear layers, and the RigL mask is attached to the linear layer weights, not to the reshape operation.

TIP When using RigL with a Conv1d + Flatten + Linear architecture: the Linear layer after Flatten has in_features = channels * time_steps (e.g., 16*31=496). At 90% sparsity, only ~50 of 496 inputs connect to each output neuron on average. This is a very local connectivity pattern -- each output sees only 50 of 496 possible input features. The mask structure may naturally group by frequency band or time range.

Memory Layout: Why Flatten is Free

```
// Conv1d output stored row-major in memory:
```

```
// buffer[ch * T + t] for ch in [0,16), t in [0,31)
// = buffer[0..495] (channel 0: indices 0-30, channel 1: 31-61, ...)

// Flatten[1] output: same buffer, read as linear[0..495]
// linear[i] = buffer[i] for i in [0, 496)
// The LINEAR matmul computes: sum_i(W[out, i] * flat[i])
// = sum_ch(sum_t(W[out, ch*31+t] * conv_out[ch, t]))
// Equivalent to treating each (channel, time) pair as a feature
```

Chapter 31

TensorConversion.h / TensorConversion.c

Dtype Conversion and Dequantization Between Tensor Types

31.1 Usage of This File

TensorConversion.c implements conversions between the quantized integer tensors used in FQT and the float tensors used in some optimizer and evaluation code. The two primary conversions are: (1) SYM_INT32 to FLOAT32: multiply mantissa by scale. (2) FLOAT32 to SYM_INT32: divide by scale, apply rounding mode.

This module is the interface between the integer arithmetic world (FQT) and the float world (loss computation, evaluation metrics, checkpoint loading). The conversion direction matters: quantization (float -> int) uses rounding, while dequantization (int -> float) is exact.

```
// Convert entire tensor between types:
void convertTensorToFloat32(tensor_t *in, tensor_t *out);   // Dequantize: int*scale -> float
void convertTensorToSYMInt32(tensor_t *in, tensor_t *out,
symInt32QConfig_t *qConfig);                                // Quantize: float/scale -> int

// Single-element conversion:
float dequantizeSymInt32(int32_t m, float scale);           // m * scale
int32_t quantizeSymInt32(float v, float scale,
roundingMode_t mode);                                       // round(v / scale)
```

31.2 Interactions with Other Files and STM32

TensorConversion.c depends on DTypes.h, Tensor.h, Rounding.h, and Quantization.h. It is called by: accumulateTensorIntoFloat32Inplace() in Add.c (during gradient accumulation), Serialize.c (when saving SYM_INT32 models to flash), and any evaluation code that needs float predictions from an integer model.

CONCEPT	SYM_INT32 dequantization: real_value = mantissa * scale. Example: mantissa=1500, scale=0.001 -> real_value=1.5. Quantization: mantissa = round(real_value / scale) = round(1.5 / 0.001) = round(1500.0) = 1500. With SR_HALF_AWAY rounding and scale=0.001, values between 0.0005 and 0.0015 all map to mantissa=1.

31.3 Line-by-Line Explanation

dequantizeSymInt32() -- Integer to Float

```
float dequantizeSymInt32(int32_t m, float scale) {
return (float)m * scale;                                    // Exact: no rounding needed
}

// Example: m=-750, scale=0.004 -> -750 * 0.004 = -3.0
```

```
// Used in: output dequantization, gradient accumulation
```

quantizeSymInt32() -- Float to Integer

```
int32_t quantizeSymInt32(float v, float scale,
roundingMode_t mode) {
float scaled = v / scale;                    // Scale to integer range
return (int32_t)roundByMode(scaled, mode);   // Apply rounding
}

// HALF_AWAY: round(1.5 / 0.001) = 1500 (deterministic)
// SR_HALF_AWAY: round(1.5 / 0.001 + U(-0.5,0.5))
(stochastic)
// SR prevents systematic bias from repeated small gradients
```

convertTensorToFloat32() -- Batch Conversion

```
void convertTensorToFloat32(tensor_t *in, tensor_t *out) {
size_t n = calcNumberOfElementsByTensor(in);
float scale = getSymInt32Scale(in);          // Read from quantization
for (size_t i = 0; i < n; i++) {
int32_t m = readBytesAsInt32(&in->data[i*4]);
float v = (float)m * scale;                  // Dequantize
writeFloatToByteArray(v, &out->data[i*4]);
}
out->quantization->type = FLOAT32;           // Update type metadata
}
```

31.4 Missing RigL Code

TensorConversion.c does not need direct RigL changes. However, when loading a RigL-trained model checkpoint: (1) load the integer weights and float scale, (2) load the binary mask, (3) zero any weights at positions where mask=0 (in case the checkpoint was saved before zeroing). This is a correctness requirement for checkpoint loading.

```
// After loading a RigL checkpoint:
for (size_t i = 0; i < numWeights; i++) {
if (!tensorBoolGet(cfg->weightMask, i)) {
// Zero-out inactive weight positions:
int32_t zero = 0;
writeInt32ToByteArray(zero, &cfg->weights->param->data[i*4]);
}
}
```

Chapter 32

SlidingWindow.h / SlidingWindow.c

Streaming Window Extraction for HAR Inference

32.1 Usage of This File

SlidingWindow.c implements a streaming window buffer for real-time sensor data processing. Instead of waiting for a full 128-sample window before processing, the sliding window maintains a ring buffer and processes overlapping windows as new samples arrive. For HAR at 50 Hz with 128-sample windows, a new window can be processed every 64 samples (50% overlap) = 1.28 seconds between predictions.

The module is essential for real-time HAR inference on STM32: the accelerometer delivers samples at 50 Hz via DMA. SlidingWindow buffers incoming samples and signals when a complete window of 128 samples is ready, triggering the neural network forward pass.

```
typedef struct slidingWindow {
float *buffer; // Ring buffer [channels * windowSize]
size_t channels; // 9 for UCI HAR (3 acc + 3 gyro + 3 total_acc)
size_t windowSize; // 128 for UCI HAR (2.56 seconds at 50 Hz)
size_t stride; // Step size: 64 = 50% overlap
size_t writePos; // Ring buffer write position
size_t sampleCount; // Total samples received since init
} slidingWindow_t;

void slidingWindowInit(slidingWindow_t *sw, size_t channels,
size_t windowSize, size_t stride);                    // Allocate ring buffer
bool slidingWindowPush(slidingWindow_t *sw, float *sample);   // True when window ready
void slidingWindowGet(slidingWindow_t *sw, tensor_t *out);    // Copy to tensor
void slidingWindowReset(slidingWindow_t *sw);                 // Clear buffer
```

32.2 HAR Dataset Context

The UCI HAR dataset was recorded with a Samsung Galaxy S2 smartphone worn on the waist. Sensor signals were sampled at 50 Hz: 3-axis accelerometer (body + gravity separated) and 3-axis gyroscope, giving 9 signals total. The dataset uses 128-sample windows with 50% overlap (stride=64).

```
// UCI HAR sensor channels (9 total):
// [0]: total_acc_x [1]: total_acc_y [2]: total_acc_z    // Raw accelerometer
// [3]: body_acc_x [4]: body_acc_y [5]: body_acc_z       // Body component
// [6]: body_gyro_x [7]: body_gyro_y [8]: body_gyro_z    // Gyroscope

// Timing:
```

```
// 50 Hz sampling -> 1 sample every 20ms
// 128 samples -> 2.56 seconds of signal per window
// Stride = 64 -> new window every 1.28 seconds at 50Hz
// Memory: ring buffer = 9 channels * 128 samples * 4 bytes = 4608 bytes
// Fits in STM32F746ZG DTCM SRAM with room to spare
```

CONCEPT Ring buffer memory layout: [channels * windowSize] floats stored as buffer[channel * windowSize + time]. A new sample writes to buffer[c * windowSize + writePos] for each channel c. writePos advances modulo windowSize. After windowSize samples, the ring wraps and new samples overwrite the oldest. slidingWindowGet() copies samples in chronological order by starting at startPos = writePos (oldest).

32.3 Interactions with Other Files and STM32

SlidingWindow.c depends on Tensor.h (for output tensor writing), DTypes.h (writeFloatToByteArray), and Common.h. On STM32, slidingWindowPush() is called from the DMA interrupt service routine (ISR) for the IMU. The ISR runs every 20ms (50 Hz). The main loop checks a ready flag set by the ISR and calls slidingWindowGet() followed by the forward pass.

```
// STM32 integration pattern:
volatile bool windowReady = false;          // Shared ISR <-> main loop
slidingWindow_t sw;
slidingWindowInit(&sw, 9, 128, 64);

// In DMA ISR (50 Hz):
void DMA_IRQHandler(void) {
float sample[9];
readImuSample(sample);                      // Read from DMA buffer
if (slidingWindowPush(&sw, sample))
windowReady = true;                         // Signal main loop
}

// In main loop:
if (windowReady) {
windowReady = false;                        // Clear flag
slidingWindowGet(&sw, &inputTensor);        // Copy window
forwardPass(model, &inputTensor, &outputTensor);  // Run NN
uint8_t label = argmax(&outputTensor);      // Get prediction
}
```

32.4 Line-by-Line Explanation

slidingWindowInit() -- Allocate Ring Buffer

```
void slidingWindowInit(slidingWindow_t *sw, size_t channels,
size_t windowSize, size_t stride) {
sw->channels = channels;                    // 9 for HAR
sw->windowSize = windowSize;                // 128
sw->stride = stride;                        // 64 (50% overlap)
sw->writePos = 0;                           // Ring starts empty
sw->sampleCount = 0;
size_t bufSize = channels * windowSize * sizeof(float);  // 4608 bytes
sw->buffer = malloc(bufSize);               // Allocate ring buffer
if (!sw->buffer) { PRINT_ERROR('OOM'); abort(); }
memset(sw->buffer, 0, bufSize);             // Zero-initialize
}
```

slidingWindowPush() -- Ring Buffer Write

Push adds a new 9-channel sample to the ring buffer and checks if a complete window is ready. A window is ready when: (1) at least windowSize samples have been received, and (2) the count since the last window is a multiple of stride.

```
bool slidingWindowPush(slidingWindow_t *sw, float *sample) {
// Write new sample to ring buffer:
size_t pos = sw->writePos % sw->windowSize;  // Ring position 0..127
for (size_t c = 0; c < sw->channels; c++)
sw->buffer[c * sw->windowSize + pos] = sample[c];  // All 9 channels
sw->writePos++;                              // Advance ring position
sw->sampleCount++;                           // Track total samples
// Window ready when: have enough samples AND aligned to
stride:
if (sw->sampleCount < sw->windowSize) return false;  // Not enough yet
return (sw->sampleCount - sw->windowSize) % sw->stride == 0;  // Stride-aligned
}
// Example: windowSize=128, stride=64
// sampleCount=128: (128-128)%64=0 -> READY (first window)
// sampleCount=129..191: (n-128)%64 != 0 -> not ready
// sampleCount=192: (192-128)%64=0 -> READY (second window)
```

slidingWindowGet() -- Copy Window to Tensor

```
void slidingWindowGet(slidingWindow_t *sw, tensor_t *out) {
// Copy ring buffer to output tensor in chronological order:
size_t startPos = sw->writePos % sw->windowSize;  // Oldest sample position
for (size_t c = 0; c < sw->channels; c++) {       // For each sensor channel
for (size_t t = 0; t < sw->windowSize; t++) {     // For each time step
size_t ringIdx = (startPos + t) % sw->windowSize;  // Chronological order
```

```
float v = sw->buffer[c * sw->windowSize + ringIdx];
size_t outIdx = (c * sw->windowSize + t) * sizeof(float);
writeFloatToByteArray(v, &out->data[outIdx]);  // Write to tensor
}
}
}
// Output tensor shape: [1, 9, 128] (1 sample, 9 channels, 128 time steps)
// Total bytes: 9 * 128 * 4 = 4608 bytes
```

32.5 Missing RigL Code

SlidingWindow.c does not need RigL changes. It produces dense input tensors: all 9*128 = 1152 values are filled from the ring buffer. The sparse forward pass in Linear and Conv1d layers then processes these dense inputs, skipping inactive weights via the mask check. The sensor data itself is never sparse -- every time step provides real sensor readings.

TIP For 50 Hz HAR with 128-sample windows and 50% overlap: slidingWindowPush() is called 50 times/second (every 20ms from DMA ISR), and slidingWindowGet() + forward pass runs every 64 samples = every 1.28 seconds. A Cortex-M7 forward pass for a sparse model (90% sparsity, 16-channel Conv1d + Linear) takes approximately 20-50ms, well within the 1280ms budget.

Chapter 33

Comparison.h / Comparison.c

Tensor Comparison Operations for Mask and Threshold Logic

33.1 Usage of This File

Comparison.c implements element-wise comparison operations between tensors and scalars. These are used in: ReLU threshold comparisons (is x > 0?), MaxPool argmax tracking (which element is the max?), gradient mask generation (is |weight| > threshold?), and early stopping conditions (has loss improved by more than epsilon?).

`// Element-wise comparisons:`	
`void greaterThan(tensor_t *a, float b, tensor_t *out);`	`// out[i] = a[i] > b`
`void lessThan(tensor_t *a, float b, tensor_t *out);`	`// out[i] = a[i] < b`
`void greaterThanTensor(tensor_t *a, tensor_t *b, tensor_t *out);`	`// out[i] = a[i] > b[i]`
`// Reduction comparisons:`	
`size_t argmax(tensor_t *t);`	`// Index of maximum element`
`size_t argmin(tensor_t *t);`	`// Index of minimum element`
`bool allGreaterThan(tensor_t *t, float thresh);`	`// All elements > thresh?`

33.2 Interactions with Other Files and STM32

Comparison.c is called by MaxPool1d.c (argmax in backward), Relu.c (threshold comparison), and the RigL implementation (comparing |weight| to threshold for DROP, |grad| to threshold for GROW). argmax() is also used in evaluation code to convert Softmax probability vectors to predicted class labels.

CONCEPT argmax() is the key function for classification inference: given softmax output probabilities [0.05, 0.08, 0.72, 0.07, 0.04, 0.04] for 6 HAR classes, argmax returns index 2 (sitting) as the predicted activity. This is a single linear scan O(n) over the 6 probabilities.

33.3 Line-by-Line Explanation

argmax() -- Predicted Class

`size_t argmax(tensor_t *t) {`	
`size_t n = calcNumberOfElementsByTensor(t);`	
`size_t maxIdx = 0;`	
`float maxVal = readBytesAsFloat(&t->data[0]);`	
`for (size_t i = 1; i < n; i++) {`	

```
float v = readBytesAsFloat(&t->data[i*4]);
if (v > maxVal) { maxVal = v; maxIdx = i; }
}
return maxIdx;                                  // Class prediction
}

// HAR inference:
softmaxForward(&softmaxLayer, &linear_out, &probs);
size_t pred = argmax(&probs);                   // 0=walking, 2=sitting, etc.
```

greaterThan() -- Threshold Mask

```
void greaterThan(tensor_t *a, float thresh, tensor_t *out) {
size_t n = calcNumberOfElementsByTensor(a);
for (size_t i = 0; i < n; i++) {
float v = readBytesAsFloat(&a->data[i*4]);
float result = (v > thresh) ? 1.0f : 0.0f;      // Binary mask
writeFloatToByteArray(result, &out->data[i*4]);
}
}
// Used in ReLU backward: mask = greaterThan(fwdInput, 0.0f)
```

33.4 RigL Usage of Comparisons

Comparison operations are central to the RigL DROP/GROW steps. The DROP step uses findAbsKthSmallestActive() to find the K-th smallest active weight magnitude -- effectively a threshold computation. The GROW step uses the gradient magnitudes and finds the K-th largest inactive gradient.

RigL: RigL Threshold Comparisons	DROP: threshold = |weight[k-smallest-active]|. For each active weight: if |weight[i]| < threshold, set mask[i]=0 (DROP). GROW: threshold = |grad[k-largest-inactive]|. For each inactive weight: if |grad[i]| > threshold, set mask[i]=1 (GROW).

```
// Conceptual RigL mask update using comparisons:
float dropThresh = findAbsKthSmallestActive(weights, mask,
K);
float growThresh = findAbsKthLargestInactive(grads, mask, K);
for (size_t i = 0; i < n; i++) {
bool active = tensorBoolGet(mask, i);
float w = fabsf(readBytesAsFloat(&weights->data[i*4]));
float g = fabsf(readBytesAsFloat(&grads->data[i*4]));
if (active && w < dropThresh) tensorBoolSet(mask, i, false);  // DROP
if (!active && g > growThresh) tensorBoolSet(mask, i, true);  // GROW
}
```

Chapter 34

RowBroadcast.h / RowBroadcast.c

Row-Wise Broadcasting for Bias Addition and Normalization

34.1 Usage of This File

RowBroadcast.c implements broadcasting operations where a 1D vector is added to (or multiplied by) every row of a 2D matrix. The primary use case is bias addition in linear layers: adding a [features] bias vector to each row of a [batch, features] activation matrix. Without broadcasting, bias addition would require an explicit loop over the batch dimension, duplicating the bias vector batch times.

Column broadcasting (addColBroadcast, mulColBroadcast) handles the transpose case: adding a [batch] vector to every column of a [batch, features] matrix. This appears in normalization: adding per-sample means back after normalization.

```
// Row broadcasting: same vector added to every row
void addRowBroadcast(tensor_t *matrix, tensor_t *row, tensor_t *out);   // out[b,f] = mat[b,f]+row[f]
void mulRowBroadcast(tensor_t *matrix, tensor_t *row, tensor_t *out);   // out[b,f] = mat[b,f]*row[f]

// Column broadcasting: same vector added to every column
void addColBroadcast(tensor_t *matrix, tensor_t *col, tensor_t *out);   // out[b,f] = mat[b,f]+col[b]
void mulColBroadcast(tensor_t *matrix, tensor_t *col, tensor_t *out);   // out[b,f] = mat[b,f]*col[b]

// In-place variants (output = input, saves memory allocation):
void addRowBroadcastInplace(tensor_t *matrix, tensor_t *row);   // matrix += row broadcast
void mulRowBroadcastInplace(tensor_t *matrix, tensor_t *row);   // matrix *= row broadcast
```

34.2 Memory Efficiency Analysis

Broadcasting avoids materializing an expanded bias matrix. Instead of storing a [batch, features] copy of the bias (batch * features floats), only the [features] bias vector is needed. The expansion happens on-the-fly during the broadcast loop.

```
// Memory comparison for bias addition:
// batch=32, features=64:
// Expanded approach: [32, 64] float matrix = 32*64*4 = 8192 bytes
// Broadcast approach: [64] float vector = 64*4 = 256 bytes
// Savings: 7936 bytes = 31x less memory for the bias buffer
```

```
// On STM32F746ZG with 320 KB SRAM:
// 8 KB just for expanded bias is 2.5% of total SRAM
// 256 bytes is 0.08% -- negligible
// The difference matters when multiple layers stack in SRAM
```

CONCEPT Broadcasting is more memory-efficient than pre-expanding the bias vector. For batch=32, features=64: expanded = 8192 bytes vs. broadcast = 256 bytes -- 32x savings. This is why PyTorch, NumPy, and ODT all use broadcasting semantics: memory is the primary constraint, not compute.

34.3 Interactions with Other Files and STM32

RowBroadcast.c is called by: Linear.c (addRowBroadcast for bias addition after matmul), LayerNorm.c (mulRowBroadcast for gamma scaling, addRowBroadcast for beta shift), Conv1d.c (addColBroadcast for per-channel bias). It depends on Tensor.h and DTypes.h (readBytesAsFloat, writeFloatToByteArray). No quantization logic is in RowBroadcast -- it operates on FLOAT32 values only.

On STM32, addRowBroadcast() with batch=32 and features=64 performs 32*64=2048 FADD operations. At Cortex-M7 throughput of ~1 FADD/cycle (pipelined), this takes about 2048 cycles = ~9.5 microseconds at 216 MHz. Negligible compared to the matmul (~200,000 cycles for dense, ~20,000 for 90% sparse).

34.4 Line-by-Line Explanation

addRowBroadcast() -- Bias Addition

The outer loop iterates over batch samples, the inner loop over features. For each (b, f) pair, the same bias[f] is added regardless of which sample b is being processed. This matches the linear layer semantics: each output neuron has one bias value added to all samples.

```
void addRowBroadcast(tensor_t *matrix, tensor_t *row,
tensor_t *out) {
size_t rows = getDimensionsByIndex(matrix, 0);          // Batch size (e.g., 32)
size_t cols = getDimensionsByIndex(matrix, 1);          // Feature count (e.g., 64)
for (size_t r = 0; r < rows; r++) {                     // For each sample in batch
for (size_t c = 0; c < cols; c++) {                     // For each feature
size_t idx = r * cols + c;                              // Flat index
float m = readBytesAsFloat(&matrix->data[idx * 4]);     // Matrix value
float v = readBytesAsFloat(&row->data[c * 4]);          // Same bias for ALL rows
writeFloatToByteArray(m + v, &out->data[idx * 4]);      // Write result
}
}
}
// Called as: addRowBroadcast(&matmul_out, &bias, &layer_out)   // In Linear.c
// matmul_out[r,c] + bias[c] -> layer_out[r,c]                  // bias[c] same for all r
```

mulRowBroadcast() -- Gamma Scaling (LayerNorm)

```
void mulRowBroadcast(tensor_t *matrix, tensor_t *row,
tensor_t *out) {
size_t rows = getDimensionsByIndex(matrix, 0);          // Batch size
size_t cols = getDimensionsByIndex(matrix, 1);          // Features
for (size_t r = 0; r < rows; r++) {
for (size_t c = 0; c < cols; c++) {
size_t idx = r * cols + c;
float m = readBytesAsFloat(&matrix->data[idx * 4]);     // Normalized value
float g = readBytesAsFloat(&row->data[c * 4]);          // gamma[c]
writeFloatToByteArray(m * g, &out->data[idx * 4]);      // x_hat * gamma
}
}
}
// Called by LayerNorm: mulRowBroadcast(x_hat, gamma, scaled)
// Then: addRowBroadcast(scaled, beta, out) to complete
y=gamma*x_hat+beta
```

addColBroadcast() -- Column Broadcasting

addColBroadcast() adds a per-sample scalar to every feature of that sample. The col vector has shape [batch]: one value per row. This is used in LayerNorm backward to add the per-sample mean gradient back to the propagated loss.

```
void addColBroadcast(tensor_t *matrix, tensor_t *col,
tensor_t *out) {
size_t rows = getDimensionsByIndex(matrix, 0);          // Batch
size_t cols = getDimensionsByIndex(matrix, 1);          // Features
for (size_t r = 0; r < rows; r++) {
float offset = readBytesAsFloat(&col->data[r * 4]);     // One offset per sample
for (size_t c = 0; c < cols; c++) {
size_t idx = r * cols + c;
float m = readBytesAsFloat(&matrix->data[idx * 4]);
writeFloatToByteArray(m + offset, &out->data[idx * 4]); // Same offset, all cols
}
}
}
// Col vector: col[r] is added to ALL features of sample r
```

34.5 Missing RigL Code

RowBroadcast.c does not need RigL changes. Bias vectors are always dense -- sparsifying bias weights would prevent each output neuron from having an independent offset, removing the bias correction capability. The RigL mask is applied only to the weight matrix, not to the bias.

RigL: Bias always dense	addRowBroadcast() always adds the full [features] bias to every row of the matmul output, regardless of the weight sparsity. After the sparse matmul (which zeros out inactive weight contributions), the bias adds back a fixed offset per output neuron. This is correct: even a neuron with 90% of its inputs zeroed still benefits from a bias term.

TIP	Performance tip: addRowBroadcast() with cols=64 and rows=1 (single sample, batch=1) is equivalent to a simple vectorAdd. On STM32 with SIMD (VLDR+VFADD), 64 floats can be processed in ~128 cycles. For batch=32, total: ~4096 cycles = 19 microseconds at 216 MHz -- negligible compared to the ~10ms forward pass.

Chapter 35

JacobiEig.h / JacobiEig.c

Jacobi Eigenvalue Decomposition for PPCA

35.1 Usage of This File

JacobiEig.c implements the classical Jacobi algorithm for computing eigenvalues and eigenvectors of small symmetric matrices. It is used exclusively by PpcaReplay.c to perform PCA on the sample covariance matrix of a class. The Jacobi method iteratively applies Givens rotations to zero off-diagonal elements, converging when all off-diagonal elements are below a tolerance.

The Jacobi algorithm is O(n^3) per sweep, typically converging in 5-10 sweeps. For an NxN matrix, total work is O(N^3) floating-point operations. For N=64 (PPCA on Conv1d features), that is ~260K FP ops -- roughly the same as one forward pass through a 64->64 linear layer.

```
void jacobiEig(tensor_t *matrix, // NxN symmetric matrix (modified in-place)
float *eigenvalues, // Output: N eigenvalues (sorted descending)
tensor_t *eigvecs, // Output: NxN matrix, columns are eigenvectors
size_t maxSweeps, // Max Jacobi iterations (default 100)
float tol); // Convergence tolerance (default 1e-6)

// Convenience: find top K eigenvectors only
void jacobiEigTopK(tensor_t *matrix, float *vals, tensor_t *vecs, size_t K);
```

35.2 Interactions with Other Files and STM32

JacobiEig.c depends on DTypes.h (readBytesAsFloat, writeFloatToByteArray), Tensor.h, and standard math (fabsf, sqrtf). It is called only by PpcaReplay.c. The algorithm modifies the input matrix in-place (destroying it during eigendecomposition), so a copy must be made if the original covariance matrix is needed afterward.

WARNING	JacobiEig requires the input matrix to be allocated in SRAM. For N=64: 64*64*4 = 16 KB. For N=128: 65 KB -- half of available SRAM. This is why PPCA should be applied to small feature vectors, not raw 1152-D sensor inputs (1152*1152*4 = 5.3 MB -- impossible on MCU).

CONCEPT	Jacobi Givens rotation: for each off-diagonal element A[p,q], compute the rotation angle theta = 0.5*atan(2*A[p,q]/(A[p,p]-A[q,q])). Apply a 2x2 Givens rotation to rows/columns p and q. Repeat for all pairs until max(|off-diagonal|) < tol. The resulting diagonal elements are the eigenvalues.

35.3 Line-by-Line Explanation

Jacobi Sweep -- Givens Rotations

```
void jacobiEig(tensor_t *A, float *vals, tensor_t *V, ...) {
size_t N = getDimensionsByIndex(A, 0);
// Initialize eigenvector matrix V = I (identity)
initIdentity(V);
for (size_t sweep = 0; sweep < maxSweeps; sweep++) {
// Check convergence: sum of |off-diagonal| elements
float offDiagSum = computeOffDiagSum(A, N);
if (offDiagSum < tol) break;                    // Converged
// Apply Givens rotation to each (p,q) pair:
for (size_t p = 0; p < N-1; p++) {
for (size_t q = p+1; q < N; q++) {
float apq = getElem(A, p, q);
if (fabsf(apq) < 1e-10f) continue;              // Skip tiny elements
float theta = 0.5f * atanf(2*apq / (getElem(A,p,p) -
getElem(A,q,q)));
float c = cosf(theta), s = sinf(theta);
applyGivens(A, V, p, q, c, s, N);               // Update A and V
}
}
}
// Extract diagonal as eigenvalues, sort descending:
for (size_t i = 0; i < N; i++)
vals[i] = getElem(A, i, i);
sortEigenvaluesDescending(vals, V, N);
}
```

35.4 Missing RigL Code

JacobiEig.c does not need RigL changes. It operates on dense symmetric covariance matrices for PPCA fitting. The eigendecomposition is a one-time operation at continual learning checkpoint boundaries, not called during the per-sample training loop.

TIP Convergence check: for PPCA with K=3 components on N=32 features, Jacobi typically converges in 3-5 sweeps (< 5000 operations). The one-time PPCA fitting cost is amortized over thousands of synthetic replay samples, making the per-sample cost negligible.

Chapter 36

CSVHelper.h / CSVHelper.c

CSV File I/O for Training Logs and HAR Dataset Loading

36.1 Usage of This File

CSVHelper.c provides simple CSV reading and writing utilities for the STM32 and desktop development environments. On desktop (where the training loop runs for initial testing), CSVHelper loads the HAR dataset CSV files and logs training metrics (loss, accuracy per epoch). On STM32, it may interface with an SD card or UART log output.

The HAR dataset is originally distributed as CSV files with one sample per row, 561 features per row (after feature extraction) or as raw sensor CSV files with 9 channels and timestamp column. CSVHelper handles both formats via configurable column selection and delimiter settings.

```
typedef struct csvReader {
FILE *fp; // File pointer
char delimiter; // ',' or ' ' or TAB
size_t numCols; // Expected columns per row
char *lineBuffer; // Pre-allocated line buffer
size_t bufferSize; // sizeof(lineBuffer)
} csvReader_t;

void csvReaderOpen(csvReader_t *r, const char *path, char
delim);
bool csvReaderReadRow(csvReader_t *r, float *values, size_t
n);                                   // Read one row -> float[n]
void csvReaderClose(csvReader_t *r);
void csvWriteMetrics(FILE *fp, size_t epoch,
float loss, float acc);               // Log training metrics
```

36.2 Interactions with Other Files and STM32

CSVHelper.c depends only on stdio.h and stdlib.h (for strtof). On STM32, FILE operations require a POSIX filesystem (e.g., FatFS with SD card via SPI). In the typical ODT setup, CSVHelper is used only on the host PC for dataset loading and metric logging, while the STM32 uses the NPYLoader in DataLoader.c to read preprocessed binary .npy files.

WARNING CSVHelper is NOT memory-safe for MCU without careful buffer sizing. The HAR raw CSV files have lines up to 500 characters long (9 channels * 13 chars/value). The line buffer must be at least 500 bytes. With a 320 KB SRAM limit and multiple active buffers, pre-allocating a 4 KB line buffer is safe.

36.3 Line-by-Line Explanation

csvReaderReadRow() -- Parse One CSV Line

```
bool csvReaderReadRow(csvReader_t *r, float *values, size_t n) {
if (fgets(r->lineBuffer, r->bufferSize, r->fp) == NULL)
return false;                                          // EOF
char *ptr = r->lineBuffer;
for (size_t i = 0; i < n; i++) {
char *end;
values[i] = strtof(ptr, &end);                         // Parse float
if (end == ptr) return false;                          // Parse error
ptr = end;
if (*ptr == r->delimiter) ptr++;                       // Skip delimiter
}
return true;
}
```

csvWriteMetrics() -- Training Log

```
void csvWriteMetrics(FILE *fp, size_t epoch,
float loss, float acc) {
fprintf(fp, '%zu,%.6f,%.4f ', epoch, loss, acc);
fflush(fp);                                            // Ensure write even if process dies
}

// Usage:
// FILE *log = fopen("training_log.csv", "w");
// fprintf(log, "epoch,loss,accuracy\n");              // Header
// for each epoch: csvWriteMetrics(log, epoch, loss, acc);
```

36.4 Missing RigL Code

CSVHelper.c should log additional RigL-specific metrics during training: current sparsity level per layer (fraction of mask bits that are 0), total active weight count, and the alpha decay value. These metrics are critical for diagnosing RigL convergence.

RigL: Add RigL logging	Extend csvWriteMetrics() to accept additional columns: sparsity per layer, active weights count, alpha(t) value. This allows plotting the sparsity schedule convergence and confirming that DROP/GROW is working correctly.

```
// Extended metric logging with RigL:
void csvWriteMetricsRigL(FILE *fp, size_t step,
float loss, float acc, float alpha,
```

size_t *activeWeights, size_t numLayers) {	
fprintf(fp, "%zu,%.6f,%.4f,%.4f", step, loss, acc, alpha);	
for (size_t l = 0; l < numLayers; l++)	
fprintf(fp, ",%zu", activeWeights[l]);	
fprintf(fp, "\n");	
fflush(fp);	
}	

Chapter 37

TrainingLoopApi.h / TrainingLoopApi.c

High-Level Training Loop Orchestration

37.1 Usage of This File

TrainingLoopApi.c provides the high-level training loop that orchestrates DataLoader iteration, forward pass, loss computation, backward pass, and optimizer update. It is the top-level module that ties together all ODT components into a complete training pipeline. The API is designed to be simple enough for embedded systems engineers without deep ML background.

The module supports both float32 and integer (FQT) training modes, mini-batch or stochastic gradient descent, learning rate scheduling, and optional RigL mask updates. On STM32, the training loop runs directly on the device with live sensor data, updating the model in response to the current user environment.

```
typedef struct trainingConfig {
layer_t **model; // Array of layer pointers
size_t numLayers;
dataLoader_t *loader; // Training data source
optimizer_t *opt; // SGD or AdamW
lrScheduler_t *sched; // Optional LR scheduler
size_t epochs;
size_t batchSize;
bool useRigL; // Enable RigL mask updates
size_t rigLInterval; // T_rigL: steps between updates
size_t rigLTEnd; // T_end: total steps for schedule
float rigLAlphaInit; // Initial alpha for cosine decay
} trainingConfig_t;

void trainingRun(trainingConfig_t *cfg);                    // Run full training
void trainingEpoch(trainingConfig_t *cfg, size_t epoch);    // One epoch
float evaluateAccuracy(trainingConfig_t *cfg);              // Test set eval
```

37.2 Interactions with Other Files and STM32

TrainingLoopApi.c is the most comprehensive file in ODT -- it includes virtually everything: Layer.h, Linear.h, DataLoader.h, CrossEntropy.h, Sgd.h/AdamW.h, LrScheduler.h, and optionally RigL (via Layer.h rigLStep). The training loop is the integration test for all ODT components.

On STM32F746ZG, the training loop must manage time carefully. A full epoch over the HAR training set (6617 samples) with batch_size=1 and a 3-layer sparse model takes approximately 6617 * 50ms forward+backward = 330 seconds = 5.5 minutes. This is feasible for initial training; production systems may train offline and deploy only the pretrained weights.

CONCEPT	Mini-batch training on MCU: with 320 KB SRAM and a 4608-byte sample, a batch_size=4 requires 4*4608 = 18 KB for input data alone. Layer activations add ~16 KB. With optimizer state (Adam: 2x weights ~26 KB), batch_size=4 is feasible. batch_size=1 (SGD) is safest for SRAM budget.

37.3 Line-by-Line Explanation

trainingEpoch() -- One Full Epoch

```
void trainingEpoch(trainingConfig_t *cfg, size_t epoch) {
float totalLoss = 0.0f;
size_t correct = 0, total = 0;
size_t step = epoch * cfg->loader->numSamples /
cfg->batchSize;

dataLoaderShuffle(cfg->loader);                    // Shuffle before each epoch
while (dataLoaderHasNext(cfg->loader)) {
// Load batch:
dataLoaderGetBatch(cfg->loader, &inputBatch, &labelBatch,
cfg->batchSize);

// Forward pass through all layers:
tensor_t *cur = &inputBatch;
for (size_t l = 0; l < cfg->numLayers; l++) {
layerFunctions[cfg->model[l]->type].forward(
cfg->model[l], cur, &activations[l]);
cur = &activations[l];
}

// Loss and backward:
float loss = crossEntropyForwardFloat(cur, &labelBatch);
totalLoss += loss;
crossEntropyBackwardFloat(cur, &labelBatch,
&lossGrad[cfg->numLayers]);
for (int l = cfg->numLayers-1; l >= 0; l--) {
layerFunctions[cfg->model[l]->type].backward(
cfg->model[l], &activations[l-1], &lossGrad[l+1],
&lossGrad[l]);
}

// Optimizer update:
optimizerStep(cfg->opt, cfg->model, cfg->numLayers);

// Optional RigL mask update:
if (cfg->useRigL)
rigLStep(cfg->model, cfg->numLayers, step,
```

```
cfg->rigLTEnd, cfg->rigLInterval, cfg->rigLAlphaInit);
step++;
}

// LR schedule update:
if (cfg->sched) lrSchedulerStep(cfg->sched, epoch);
PRINT_INFO("Epoch %zu: loss=%.4f", epoch, totalLoss/total);
}
```

37.4 Missing RigL Code

TrainingLoopApi.c currently does not call rigLStep() -- the RigL field is missing from trainingConfig_t in the current implementation. Adding RigL requires: (1) adding useRigL, rigLInterval, rigLTEnd, rigLAlphaInit fields to trainingConfig_t, and (2) calling rigLStep() at the appropriate point in the training loop.

RigL: Integration Point	rigLStep() must be called AFTER the optimizer step but BEFORE the next forward pass. The order matters: update weights with SGD, then update the mask with RigL. This ensures that the gradient used for GROW reflects the current weight update.

TIP	Debugging RigL training: log active weight count per layer every epoch. A correctly working RigL maintains constant total sparsity while allowing mask positions to change. If sparsity is monotonically increasing (only dropping, never growing), check that K>0 in both DROP and GROW steps and that gradient accumulation is working correctly.

Chapter 38

UserAPI.h: Model Building, Initialization, and Deployment

Putting It All Together: From Sensor to Prediction

38.1 Usage of This File

UserAPI.h is the top-level interface that end users interact with. It provides model construction helpers, initialization routines, and deployment inference functions. The HAR use case is fully supported: build a 6-class activity recognition model, train it on the STM32, and deploy in streaming inference mode using SlidingWindow for real-time predictions.

The UserAPI abstracts tensor management, quantization configuration, and vtable setup. A user can build a complete training pipeline with about 30 lines of C by calling the high-level model-building functions, passing layerSpec_t configuration structs, and calling trainingRun().

```
model_t *buildSequentialModel(layerSpec_t *specs, size_t n);  // Build all layers
void modelInitWeights(model_t *m, weightInitScheme_t s);  // Kaiming/Xavier init
void modelSetTrainingMode(model_t *m, bool training);  // Toggle dropout
size_t modelPredict(model_t *m, tensor_t *input);  // Returns class index
void modelSave(model_t *m, const char *path);  // Serialize to .odts
model_t *modelLoad(const char *path);  // Load from .odts
```

layerSpec_t -- Declarative Layer Specification

```
typedef struct layerSpec {
layerType_t type;
size_t inFeatures, outFeatures;  // For LINEAR
size_t kernelSize, stride, padding;  // For CONV1D
float dropoutRate;  // For DROPOUT
bool sparse;  // Enable RigL mask
float sparsity;  // Target sparsity, e.g. 0.9
} layerSpec_t;
```

38.2 Complete HAR Pipeline Example

The following example builds a complete HAR model with Conv1d front-end, pooling, and sparse linear classification head. This demonstrates the full UserAPI from model construction through training and deployment.

```
// 1. Define model architecture:
layerSpec_t specs[] = {
```

```
{CONV1D, .inFeatures=9, .outFeatures=16, .kernelSize=3},  // Temporal features
{MAXPOOL1D, .kernelSize=4, .stride=4},  // 4x downsampling
{RELU},  // Non-linearity
{FLATTEN},  // [16,31] -> [496]
{LINEAR, .inFeatures=496, .outFeatures=64,
.sparse=true, .sparsity=0.9f},  // 90% sparse
{RELU},
{LINEAR, .inFeatures=64, .outFeatures=6},  // 6-class output
{SOFTMAX},
};
model_t *model = buildSequentialModel(specs, 8);  // Build all layers
modelInitWeights(model, KAIMING_UNIFORM);  // Initialize weights
```

Training Configuration with RigL

```
// 2. Configure training pipeline:
dataLoader_t *loader = npyLoaderCreate(
TRAIN_X_PATH, TRAIN_Y_PATH, BATCH_SIZE_1);  // Stream one sample
optimizer_t *opt = sgdOptimizerCreate(0.01f, 0.0f);  // lr=0.01
lrScheduler_t *sched = cosineAnnealingCreate(0.01f, 0.0f,
50);  // 50 epochs
trainingConfig_t cfg = {
.model=model->layers, .numLayers=model->numLayers,
.loader=loader, .opt=opt, .sched=sched,
.epochs=50, .batchSize=1,
.useRigL=true, .rigLInterval=100,
.rigLTEnd=50000, .rigLAlphaInit=0.3f,
};
trainingRun(&cfg);  // Train for 50 epochs
```

38.3 Interactions with Other Files and STM32

UserAPI.h includes virtually all ODT headers: Layer.h, Linear.h, Conv1d.h, Relu.h, Softmax.h, Flatten.h, MaxPool1d.h, Dropout.h, TrainingLoopApi.h, Serialize.h, DataLoader.h, and SlidingWindow.h. It is the final integration layer of the entire library.

Memory budget for the complete HAR system on STM32F746ZG (320 KB SRAM): sparse model 13 KB, activations 4 KB, DataLoader buffer 5 KB, PPCA replay (6 classes, K=2) 72 KB, SlidingWindow ring buffer 5 KB. Total: ~99 KB. Leaves 221 KB headroom for stack and temporary allocations.

CONCEPT Deployment mode: after training, call modelSetTrainingMode(model, false) to disable dropout, then modelSave() to write the .odts checkpoint. At runtime, modelLoad() restores weights and mask from flash. modelPredict() runs the sparse forward pass in ~5ms for a 90% sparse 3-layer model at 216 MHz.

38.4 Missing RigL Code in UserAPI

buildSequentialModel() must be extended to handle the sparse=true field in layerSpec_t: (1) allocate a BOOL tensor of size inFeatures*outFeatures bits, (2) call bernoulliFillMask() to initialize at the target sparsity, (3) set linearConfig->weightMask to the allocated mask tensor.

RigL: Complete RigL UserAPI Integration	buildSequentialModel() with sparse=true: allocate mask tensor (inFeat*outFeat bits), bernoulliFillMask(mask, 1.0f - sparsity) for initial random sparsity, set linearConfig->weightMask=mask. After buildSequentialModel(), the RigL training loop is fully functional.

```
// UserAPI internals for sparse LINEAR layer:
if (spec->sparse) {
size_t maskBits = inFeat * outFeat;
tensor_t *mask = createBoolTensor(maskBits);          // Bit-packed BOOL tensor
bernoulliFillMask(mask, 1.0f - spec->sparsity);       // e.g., 10% active bits
zeroInactiveWeights(linearCfg, mask);                 // Enforce sparsity at init
linearCfg->weightMask = mask;                         // Attach mask to layer
}
```

TIP	For HAR deployment: the trained model (13 KB) fits in internal flash (1 MB on STM32F746ZG). modelLoad() at reset takes ~2ms. SlidingWindow accumulates 128 samples at 50 Hz (2.56s window), then triggers modelPredict() which runs in ~5ms for a 90% sparse model. End-to-end latency from full window to prediction: <10ms.

Chapter 39

RigL Complete Implementation

rigLStep(), K-Selection, Mask-Aware Matmul and SGD -- Full C Code

39.1 Overview: Components to Build

Six components must be added/modified: (1) findAbsKthSmallestActive() in MinMax.c; (2) findAbsKthLargestInactive() in MinMax.c; (3) weightMask field in Linear.h; (4) mask-aware inner loop in Matmul.c; (5) mask-aware update in Sgd.c; (6) rigLStep() in new RigL.h/RigL.c; (7) serializeSparsity() fix in Serialize.c.

RigL: Implementation	All 7 components are provided with complete C code and line-by-line explanation. After implementing, ODT supports full RigL sparse training on STM32 Nucleo-F746ZG.

39.2 Step 1: MinMax.c - findAbsKthSmallestActive()

Finds K-th smallest absolute weight value among ACTIVE (mask=1) weights. Uses partial selection sort: O(n*K). Called in DROP step to set the deactivation threshold.

```
float findAbsKthSmallestActive(tensor_t *weights, tensor_t *mask, size_t K) {
size_t n = calcNumberOfElementsByTensor(weights);
float *w = (float *)weights->data;                    // Direct float access
size_t count = 0;                                     // Count active weights
for (size_t i = 0; i < n; i++)
if (tensorBoolGet(mask, i)) count++;
if (K >= count) return 1e38f;                         // K>=count: keep all
float *vals = malloc(count * sizeof(float));
if (!vals) { exit(1); }
size_t idx = 0;
for (size_t i = 0; i < n; i++)
if (tensorBoolGet(mask, i)) vals[idx++] = fabsf(w[i]); // Collect active |w|
for (size_t i = 0; i <= K && i < count; i++) {         // Partial selection sort
size_t minIdx = i;
for (size_t j = i+1; j < count; j++)
if (vals[j] < vals[minIdx]) minIdx = j;               // Find min
float tmp = vals[i]; vals[i] = vals[minIdx]; vals[minIdx] = tmp;  // Swap
}
float thresh = vals[K]; free(vals); return thresh;    // K-th smallest
```

}	

CONCEPT	Example: active |w| = [0.01, 0.05, 0.03, 0.12, 0.08], K=2. After partial sort: [0.01, 0.03, 0.05, ...]. thresh = vals[2] = 0.05. Weights with |w|<=0.05 are dropped: 0.01 and 0.03. Exactly 2 weights dropped = K.

39.3 Step 2: MinMax.c - findAbsKthLargestInactive()

`float findAbsKthLargestInactive(tensor_t *grads, tensor_t *mask, size_t K) {`	
`size_t n = calcNumberOfElementsByTensor(grads);`	
`float *g = (float *)grads->data;`	
`size_t count = 0;`	`// Count inactive (mask=0) weights`
`for (size_t i = 0; i < n; i++)`	
`if (!tensorBoolGet(mask, i)) count++;`	
`if (K >= count) return 0.0f;`	`// K>=count: grow all`
`float *vals = malloc(count * sizeof(float));`	
`if (!vals) { exit(1); }`	
`size_t idx = 0;`	
`for (size_t i = 0; i < n; i++)`	
`if (!tensorBoolGet(mask, i)) vals[idx++] = fabsf(g[i]);`	`// Collect inactive |grad|`
`for (size_t i = 0; i <= K && i < count; i++) {`	`// Partial sort descending`
`size_t maxIdx = i;`	
`for (size_t j = i+1; j < count; j++)`	
`if (vals[j] > vals[maxIdx]) maxIdx = j;`	`// Find max`
`float tmp = vals[i]; vals[i] = vals[maxIdx]; vals[maxIdx] = tmp;`	`// Swap`
`}`	
`float thresh = vals[K]; free(vals); return thresh;`	`// K-th largest`
`}`	

39.4 Step 3: Linear.h - Add weightMask Field

`// Add to linearConfig_t in Linear.h:`	
`typedef struct linearConfig {`	
`parameter_t *weights;`	
`parameter_t *bias;`	
`arithmetic_t forwardMath, weightGradMath, biasGradMath, propLossMath;`	
`quantization_t *outputQ, *propLossQ;`	
`outputMode_t weightGradAccMode, biasGradAccMode;`	
`bool ownsQuantizations;`	

`tensor_t *weightMask; // NEW: NULL=dense, BOOL=RigL sparse mask`	`// ADDED`
`} linearConfig_t;`	
`// In linearInitConfig(): cfg->weightMask = NULL;`	`// Default: dense`

39.5 Step 4: Matmul.c - Mask-Aware Inner Loop

`// In matmulFloatCore inner loop:`	
`for (size_t i = 0; i < aColumns; i++) {`	`// Inner loop`
`size_t flatIdx = rowIndex * aColumns + i;`	`// Flat weight index`
`if (weightMask != NULL && !tensorBoolGet(weightMask, flatIdx))`	
`continue; // Skip inactive weight`	`// 90% sparse: skip 90% of muls`
`float aVal = readBytesAsFloat(&A->data[aByteIdx]);`	
`float bVal = readBytesAsFloat(&B->data[bByteIdx]);`	
`result += aVal * bVal; // Only active weights contribute`	
`}`	

39.6 Step 5: Sgd.c - Mask-Aware Update

`// Modified sgdUpdateKernel:`	
`for (size_t i = 0; i < nElem; i++) {`	
`if (mask != NULL && !tensorBoolGet(mask, i)) {`	`// Check mask`
`out[i] = 0.0f; grad[i] = 0.0f; continue;`	`// Force inactive to zero`
`}`	
`float g = grad[i] + ctx->weightDecay * param[i];`	
`out[i] = param[i] - ctx->lr * g;`	`// Normal SGD`
`}`	

39.7 Step 6: Complete rigLStep()

`void rigLStep(layer_t **model, size_t numLayers,`	
`float sparsityTarget, size_t step, size_t totalSteps) {`	
`// Cosine decay for swap fraction`	
`float prog = (float)step / (float)(totalSteps > 0 ? totalSteps : 1);`	
`float alpha = 0.3f * 0.5f * (1.0f + cosf(3.14159265f * prog));`	`// alpha_init=0.3`
`for (size_t l = 0; l < numLayers; l++) {`	
`if (model[l]->type != LINEAR) continue;`	`// Only linear layers`
`linearConfig_t *cfg = model[l]->config->linear;`	
`tensor_t *mask = cfg->weightMask;`	

```
if (mask == NULL) continue;                                   // Dense layer: skip
tensor_t *weights = cfg->weights->param;
tensor_t *grads = cfg->weights->grad;
size_t n = calcNumberOfElementsByTensor(weights);
float *w = (float*)weights->data; float *g =
(float*)grads->data;
size_t numActive = 0;
for (size_t i=0;i<n;i++) if(tensorBoolGet(mask,i))            // Count active
numActive++;
size_t K = (size_t)(alpha * (float)numActive);                // Weights to swap
if (K == 0) continue;                                         // Nothing to swap

// DROP: deactivate K smallest |w|
float dropThresh = findAbsKthSmallestActive(weights, mask,
K);
for (size_t i=0;i<n;i++) {
if (tensorBoolGet(mask,i) && fabsf(w[i])<=dropThresh)
{ tensorBoolSet(mask,i,false); w[i]=0.0f; }                   // Deactivate
}
// GROW: activate K largest |grad| among inactive
float growThresh = findAbsKthLargestInactive(grads, mask, K);
for (size_t i=0;i<n;i++) {
if (!tensorBoolGet(mask,i) && fabsf(g[i])>=growThresh)
{ tensorBoolSet(mask,i,true); w[i]=0.0f; }                    // Activate at zero
}
// Zero gradients of still-inactive weights
for (size_t i=0;i<n;i++)
if (!tensorBoolGet(mask,i)) g[i]=0.0f;
}                                                             // End layer loop
}
```

39.8 Step 7: serializeSparsity() Fix

```
void serializeSparsity(tensor_t *mask, FILE *fp) {
if (mask == NULL || mask->quantization->type != BOOL) {
uint8_t noMask = 0; fwrite(&noMask,1,1,fp); return;           // No mask: write 0
}
uint8_t hasMask = 1; fwrite(&hasMask,1,1,fp);                 // Has mask: write 1
size_t n = calcNumberOfElementsByTensor(mask);
size_t bytes = (n+7)/8;                                       // Bit-packed size
fwrite(mask->data, 1, bytes, fp);                             // Write bit-packed mask
}
```

39.9 Complete Training Loop

```
#define RIGL_INTERVAL 100 // Update mask every 100 steps
size_t step = 0;
for (size_t epoch = 0; epoch < epochs; epoch++) {
for (size_t i = 0; i < trainSize; i++) {
batch_t *b = loader.getBatch(&loader, i);                    // 4.5 KB HAR sample
forward(model, b->input); backward(model, b->label);
if (step % RIGL_INTERVAL == 0)
rigLStep(model, nLayers, 0.9f, step, totalSteps);            // Update mask
optimizerFunctions[SGD_M].step(&optim);                      // Mask-aware update
step++;
}
lrSchedulerStep(&sched);                                     // Cosine LR decay
}
```

TIP Run rigLStep() BEFORE the optimizer step. This ensures that newly grown weights (w=0) get their first gradient update in the same step they are activated. If rigLStep() runs AFTER the optimizer, the new weights miss their first update.

39.10 Expected Results

After implementing all 7 components, expected behavior: (1) mask sparsity stays near 90% throughout training (K dropped = K grown); (2) training loss decreases normally despite sparsity; (3) mask evolves -- weights move from low-gradient inactive positions to high-gradient positions; (4) model can be saved/loaded with mask via serializeSparsity().

WARNING For the HAR dataset, the first linear layer (1152->64) needs 72 MB for weights in float32 BEFORE sparsity. The RigL mask saves the mask overhead (9 KB), but the full weight tensor still needs to fit in SRAM. Use a smaller model (e.g., Conv1d feature extractor) to make the linear layers MCU-feasible.